

Correctness of a Sparse-Merkle-Tree Verification-Circuit Compiler

Jinwook Kim

August 29, 2026

Abstract

A verification-circuit compiler turns sparse-Merkle-tree operations (membership, non-membership, single-leaf update) into layered arithmetic circuits whose acceptance is meant to coincide with the native semantics of the operation: the circuit accepts a witness encoding exactly when the operation holds between the claimed roots. This entry mechanizes that claim and its composition with the GKR protocol assembly of the parent session.

The development covers: the native semantics of sparse-Merkle-tree operations (path verification against abstract leaf and node hashes); a deep-embedded model of the compiler emitting one residual output layer (equality, booleanity and tag residuals) over an evaluation stack computing the in-circuit hash images and key-bit multiplexers; Theorem A, the deterministic equivalence between circuit acceptance and the native semantics, split into an acceptance-to-constraint stage and a constraint-to-semantics stage, packaged as soundness and completeness of a verifier model that mirrors the public-input checks of the surrounding verifier; an injectivity lemma for the fixed-width leaf pre-image encoding, reducing leaf-hash collisions to collisions of the underlying sponge; and Theorem C, the composition bounding the probability that the GKR chain accepts a semantically invalid operation, for any adversary, by $(r + \sum_i (2s_i d + 1))/|\mathbb{F}|$.

No collision resistance or injectivity of the hashes is assumed anywhere in Theorem A: the equivalence is deterministic, with the hashes as abstract parameters. The hash sub-circuits of the evaluation stack are specified by a generator-correctness assumption and remain otherwise abstract.

Every locale of the development is inhabited by a concrete instance, so none of the theorems is vacuously true. In particular the compiler model is interpreted over the KoalaBear prime field $\text{GF}(2^{31} - 2^{24} + 1)$, the wire field of the production prover, with an explicit gate list whose evaluation provably equals the specification vectors; Theorem A is exercised there on a concrete membership operation, and Theorem C fires with each of its premises discharged for a concrete, semantically unsatisfiable update, the representative degree bound being supplied by the polynomial interpolation of the parent session.

Contents

1	Leaf states and their field encoding	2
2	Instance parameters	2
3	SMT semantics locale	2
4	Operation semantics: the meaning standard for Theorem A	3

5	Sanity instance (non-vacuity witness)	3
6	Sponge geometry	4
7	The lossless fold	5
7.1	Lane layout (pin against the Rust test <i>leaf-fold-layout-tag-len-verbatim</i>) . . .	6
8	Injectivity (lemma (b))	7
9	Collision-resistance reduction	8
9.1	The encode domain	8
10	Firing instances (non-vacuity witnesses)	9
10.1	The two collision classes the fold closes	10
10.2	The reduction fires	10
11	Operation kinds	11
12	List segment utilities	11
13	Constraint descriptors and the residual output layer	12
13.1	Evaluation of the residual layer	12
14	Key-bit multiplexer (Rust: <code>build_mux</code>)	15
15	Kind tags and key bits	16
16	Zero padding	16
17	The compiler model locale (representation layer)	17
17.1	Input-vector layout (Rust: <code>InputLayout</code> , same segment order)	17
17.2	Evaluation-stack output specification	18
17.3	Residual constraint descriptors (the constraint SET of <code>compile.rs</code>)	20
17.4	Constraint conjunction (the A-i target semantics)	20
17.5	Witness encoding (Rust: <code>build_input_vector</code>)	21
18	The full compiler model (evaluation stack as abstract parameter)	22
19	Slot lemmas for the evaluation-stack specification vector	23
19.1	Update slots	28
20	Stage A-i: acceptance is the constraint conjunction	33
21	Stage A-ii: the constraint conjunction is the SMT semantics	37
21.1	Theory-level helpers	37
21.2	Segment values of the read-only encoding	38
21.3	The read-only constraint conjunction on the honest encoding	41
21.4	Segment values of the Update encoding	43
21.5	The Update constraint conjunction on the honest encoding	47

22 Theorem A: the verifier model and the compilation soundness	49
23 The composition locale (v0.1 pipeline)	53
24 Theorem C	54
25 The KoalaBear prime	57
26 The field type	58
27 Field structure (finite integral domain argument)	60
28 Concrete facts the instantiations consume	63
29 The KoalaBear toy hash pair and representations	64
30 Quadratic wire descriptors and their gate layers	65
31 Small list-segment computation lemmas	68
32 The concrete evaluation stacks (depth 1, one-lane digests)	68
32.1 Layout arithmetic at the concrete parameters	69
32.2 The explicit specification vectors at the concrete layout	70
32.3 The stack layers compute the specification vectors	70
33 The compiler model is inhabited over KoalaBear	72
34 Activation examples	73
35 Lemma (b) over the production field	74
36 The composed locale is inhabited over KoalaBear + mpoly	74
37 The activation scenario	74
37.1 Semantic unsatisfiability (the nosem premise)	75
37.2 Public-input, echo and length premises	75
37.3 Chain shape (the wf premise, discharged by computation)	76
38 Theorem C, activated	76

```

theory SMT-Semantics
  imports GKR-Protocol.Layered-Circuit
begin

```

1 Leaf states and their field encoding

```

datatype 'v leaf-state =
  Empty
  | Occupied 'v
  | Tombstone

```

Domain-separated encoding: a one-element kind tag (0 = Empty, 1 = Occupied, 2 = Tombstone) followed by the payload encoding for *Occupied*. The payload encoder is a parameter (the registry contract fixes the concrete convention; the prover is generic in it).

```

fun encode :: ('v  $\Rightarrow$  'f list)  $\Rightarrow$  'v leaf-state  $\Rightarrow$  'f::comm-ring-1 list where
  encode encp Empty = [0]
  | encode encp (Occupied v) = 1 # encp v
  | encode encp Tombstone = [2]

```

```

lemma encode-nonempty: encode encp l  $\neq$  []
by (cases l) simp-all

```

lemma encode-tag-separates:

— The tag lane alone separates the three states ($2 \neq 0$ and $2 \neq 1$ require odd characteristic; KoalaBear has odd characteristic. We keep the lemma over a ring with the needed inequalities as explicit premises, discharged trivially in concrete instances.)

```

fixes encp :: 'v  $\Rightarrow$  'f::comm-ring-1 list
assumes (2 :: 'f)  $\neq$  0 (2 :: 'f)  $\neq$  1
shows hd (encode encp Empty)  $\neq$  hd (encode encp Tombstone)
  and hd (encode encp (Occupied v))  $\neq$  hd (encode encp Tombstone)
  and hd (encode encp Empty)  $\neq$  hd (encode encp (Occupied v))
using assms by simp-all

```

2 Instance parameters

```

record smt-params =
  depth :: nat          — tree depth; Rust default 24 (genesis parameter)
  leaf-max-fields :: nat — structural bound of the leaf hash domain

```

type-synonym asset-id = nat

— Sequential AssetID key (monotone counter from the RWA Registry). Uniqueness is enforced by the verified state transition, not here.

3 SMT semantics locale

```

locale smt-semantics =
  fixes params :: smt-params
  and h-leaf :: 'v leaf-state  $\Rightarrow$  'd
  and h-node :: 'd  $\Rightarrow$  'd  $\Rightarrow$  'd

```

begin

Merkle path verification: recompute the root from a leaf digest, the key bits (low bit = deepest level) and the sibling chain. Rust: *MerklePath::compute-root* - spec-literal transcription.

```
fun path-root :: 'd  $\Rightarrow$  nat  $\Rightarrow$  'd list  $\Rightarrow$  'd where
  path-root leaf key [] = leaf
| path-root leaf key (s # ss) =
  path-root (if key mod 2 = 0 then h-node leaf s else h-node s leaf)
    (key div 2) ss
```

```
definition path-ok :: 'v leaf-state  $\Rightarrow$  asset-id  $\Rightarrow$  'd list  $\Rightarrow$  'd  $\Rightarrow$  bool where
  path-ok leaf key siblings root  $\longleftrightarrow$ 
    length siblings = depth params  $\wedge$ 
    key < 2  $\wedge$  depth params  $\wedge$ 
    path-root (h-leaf leaf) key siblings = root
```

4 Operation semantics: the meaning standard for Theorem A

end

```
datatype 'v smt-op =
  Membership asset-id 'v
| NonMembership asset-id
| Update asset-id 'v leaf-state 'v leaf-state
— Rust: SmtOperation - same three constructors
```

context *smt-semantics*
begin

smt-valid: what it MEANS for an operation to hold between two roots with a witness (leaf state + sibling chain). This is the single semantic anchor; the compiled circuit is verified AGAINST this meaning, never against itself. Clauses literally in sync with *smt-valid-native* (smt.rs).

```
fun smt-valid :: 'v smt-op  $\Rightarrow$  'd  $\Rightarrow$  'd  $\Rightarrow$  ('v leaf-state  $\times$  'd list)  $\Rightarrow$  bool where
  smt-valid (Membership k v) root root' (leaf, sib)  $\longleftrightarrow$ 
    root' = root  $\wedge$  leaf = Occupied v  $\wedge$  path-ok leaf k sib root
| smt-valid (NonMembership k) root root' (leaf, sib)  $\longleftrightarrow$ 
    root' = root  $\wedge$  (leaf = Empty  $\vee$  leaf = Tombstone)  $\wedge$  path-ok leaf k sib root
| smt-valid (Update k old new) root root' (leaf, sib)  $\longleftrightarrow$ 
    leaf = old  $\wedge$  path-ok old k sib root  $\wedge$  path-ok new k sib root'
— Update re-uses the SAME sibling chain for the old and new root: single-leaf update semantics.
  Batched multi-leaf updates compose sequentially (Theorem D territory).
```

end

5 Sanity instance (non-vacuity witness)

A depth-1 instance over the integers with a transparent "hash" (injective pairing on small naturals encoded as ints) witnessing that the locale is inhabited and *smt-valid* actually discriminates: a valid membership holds and a mismatched root fails.

```

definition toy-params :: smt-params where
  toy-params = ⟨ depth = 1, leaf-max-fields = 4 ⟩

definition toy-h-leaf :: int leaf-state ⇒ int where
  toy-h-leaf l = (case l of Empty ⇒ 0 | Occupied v ⇒ 100 + v | Tombstone ⇒ 1)

definition toy-h-node :: int ⇒ int ⇒ int where
  toy-h-node a b = 1000 + 65536 * a + b

interpretation toy: smt-semantics toy-params toy-h-leaf toy-h-node
  by unfold-locales

lemma toy-membership-valid:
  toy.smt-valid (Membership 0 7) (toy-h-node (toy-h-leaf (Occupied 7)) 3)
    (toy-h-node (toy-h-leaf (Occupied 7)) 3)
    (Occupied 7, [3])
proof –
  have toy.path-ok (Occupied 7) 0 [3] (toy-h-node (toy-h-leaf (Occupied 7)) 3)
    unfolding toy.path-ok-def
    by (simp add: toy.path-root.simps toy-params-def)
  then show ?thesis by (simp add: toy.smt-valid.simps)
qed

lemma toy-membership-wrong-root-invalid:
  ¬ toy.smt-valid (Membership 0 7) 42 42 (Occupied 7, [3])
proof –
  have ¬ toy.path-ok (Occupied 7) 0 [3] 42
    unfolding toy.path-ok-def
    by (simp add: toy.path-root.simps toy-params-def
      toy-h-node-def toy-h-leaf-def)
  then show ?thesis by (simp add: toy.smt-valid.simps)
qed

lemma toy-nonmembership-rejects-occupied:
  ¬ toy.smt-valid (NonMembership 0) r r (Occupied 7, [3])
  by (simp add: toy.smt-valid.simps)

end

```

```

theory Leaf-Fold
  imports SMT-Semantics
begin

```

6 Sponge geometry

Rust: *LEAF-SPONGE-RATE* = 8 - lanes absorbed per permutation of the rate-8 sponge. The pre-image width is the encoding bound plus one bookkeeping lane (the length lane; the tag lane replaces *enc[0]*, it does not add width) rounded UP to whole rate blocks, so the sponge absorbs every lane in exact chunks.

```

definition LEAF-SPONGE-RATE :: nat where

```

$LEAF-SPONGE-RATE = 8$

Rust: *leaf-pre-width* - literally $(leaf-max-fields + 1).div_ceil(LEAF-SPONGE-RATE) * LEAF-SPONGE-RATE$ with *div_ceil* unfolded to its standard-library definition $d + (if\ r = 0\ then\ 0\ else\ 1)$ where d/r are quotient/remainder. (The Rust parameter name *leaf-max-fields* is taken by the *smt-params* record selector of *SMT-Semantics*, so the bound is named *max-fields* throughout this theory.)

definition *leaf-pre-width* :: *nat* $\Rightarrow$ *nat* **where**

leaf-pre-width *max-fields* =
 $((max-fields + 1) \text{ div } LEAF-SPONGE-RATE$
 $+ (if\ (max-fields + 1) \text{ mod } LEAF-SPONGE-RATE = 0\ then\ 0\ else\ 1))$
 $* LEAF-SPONGE-RATE$

lemma *leaf-pre-width-ge*:

$max-fields + 1 \leq leaf-pre-width\ max-fields$

proof (*cases* $(max-fields + 1) \text{ mod } 8 = 0$)

case *True*

have $max-fields + 1$

$= (max-fields + 1) \text{ div } 8 * 8 + (max-fields + 1) \text{ mod } 8$

by *simp*

with *True* **show** *?thesis*

by (*simp* *add*: *leaf-pre-width-def* *LEAF-SPONGE-RATE-def*)

next

case *False*

then have *w*: *leaf-pre-width* *max-fields*

$= (max-fields + 1) \text{ div } 8 * 8 + 8$

by (*simp* *add*: *leaf-pre-width-def* *LEAF-SPONGE-RATE-def* *algebra-simps*)

have $max-fields + 1$

$= (max-fields + 1) \text{ div } 8 * 8 + (max-fields + 1) \text{ mod } 8$

by *simp*

moreover have $(max-fields + 1) \text{ mod } 8 < 8$ **by** *simp*

ultimately show *?thesis* **using** *w* **by** *linarith*

qed

lemma *leaf-pre-width-rate-blocks*:

leaf-pre-width *max-fields* *mod* *LEAF-SPONGE-RATE* = 0

— The sponge absorbs the pre-image in exact rate blocks (the Rust *debug-assert* in *hash-leaf-pre*).

by (*simp* *add*: *leaf-pre-width-def*)

lemma *leaf-pre-width-rounds-to-rate-blocks*:

— Pin against the Rust unit test *leaf-pre-width-rounds-to-rate-blocks*: 31 + 1 bookkeeping lanes round to exactly four rate-8 blocks.

leaf-pre-width 31 = 32

leaf-pre-width 7 = 8

leaf-pre-width 15 = 16

leaf-pre-width 23 = 24

by (*simp-all* *add*: *leaf-pre-width-def* *LEAF-SPONGE-RATE-def*)

7 The lossless fold

Rust: *leaf-fold* (*hash.rs*) - the fixed-width, lossless pre-image of an encoding of length $1..leaf-max-fields$. The Rust Err branches (empty / oversized) are the domain premise here; on the Ok domain

the layout is literally $[tag, length, verbatim\ rest, zero\ padding]$.

definition *leaf-fold* :: 'f::comm-ring-1 list $\Rightarrow$ nat $\Rightarrow$ 'f list **where**
leaf-fold enc max-fields =
 [enc ! 0, of-nat (length enc)]
 @ tl enc
 @ replicate (leaf-pre-width max-fields - (length enc + 1)) 0

lemma *leaf-fold-length*:
assumes *len-lb*: $1 \leq \text{length enc}$
and *len-ub*: $\text{length enc} \leq \text{max-fields}$
shows $\text{length (leaf-fold enc max-fields)} = \text{leaf-pre-width max-fields}$
proof –
have $\text{length enc} + 1 \leq \text{leaf-pre-width max-fields}$
using *len-ub leaf-pre-width-ge[of max-fields]* **by** *linarith*
with *len-lb* **show** *?thesis*
unfolding *leaf-fold-def* **by** *simp*
qed

7.1 Lane layout (pin against the Rust test *leaf-fold-layout-tag-len-verbatim*)

lemma *leaf-fold-tag-lane* [*simp*]:
leaf-fold enc max-fields ! 0 = enc ! 0
by (*simp add: leaf-fold-def*)

lemma *leaf-fold-length-lane* [*simp*]:
leaf-fold enc max-fields ! 1 = of-nat (length enc)
by (*simp add: leaf-fold-def*)

lemma *leaf-fold-verbatim-lane*:
assumes *i*: $i < \text{length enc} - 1$
shows $\text{leaf-fold enc max-fields ! (i + 2)} = \text{enc ! (i + 1)}$
proof –
have $\text{leaf-fold enc max-fields ! Suc (Suc i)}$
 = $(\text{tl enc @ replicate (leaf-pre-width max-fields - (length enc + 1)) 0}) ! i$
by (*simp add: leaf-fold-def*)
also have $\dots = \text{tl enc ! i}$
using *i* **by** (*simp add: nth-append*)
also have $\dots = \text{enc ! Suc i}$
using *i* **by** (*simp add: nth-tl*)
finally have $\text{leaf-fold enc max-fields ! Suc (Suc i)} = \text{enc ! Suc i}$.
then show *?thesis* **by** (*simp add: eval-nat-numeral*)
qed

lemma *leaf-fold-pad-suffix*:
 — Everything beyond the tag/length/verbatim prefix is the zero padding (the "zeros beyond" clause of the Rust layout test). Per-lane readings follow via $?i < ?n \implies \text{replicate } ?n ?x ! ?i = ?x$.
assumes *len-lb*: $1 \leq \text{length enc}$
shows $\text{drop (length enc + 1) (leaf-fold enc max-fields)}$
 = $\text{replicate (leaf-pre-width max-fields - (length enc + 1)) 0}$
proof –
have *split*: $\text{leaf-fold enc max-fields}$
 = $(\text{enc ! 0} \# \text{of-nat (length enc)} \# \text{tl enc})$
 @ $\text{replicate (leaf-pre-width max-fields - (length enc + 1)) 0}$

```

    by (simp add: leaf-fold-def)
  have len-pre: length (enc ! 0 # of-nat (length enc) # tl enc)
    = length enc + 1
    using len-lb by simp
  from split len-pre show ?thesis
    by (metis append-eq-conv-conj)
qed

```

8 Injectivity (lemma (b))

Distinct in-bound encodings never share a pre-image. Proof route: the length lane forces equal lengths (via the *of-nat* premise), the padding then coincides, so the verbatim region and the tag lane pin every element (list extensionality).

The *of-nat* premise is exactly what the length lane needs and no more: over KoalaBear it holds because the bound (default 31) is far below *p*; see the modeling notes in the file header for why it must be stated as injectivity-up-to-the-bound rather than $\text{max-fields} < \text{CARD}(f)$.

theorem *leaf-fold-inj*:

```

  fixes e1 e2 :: 'f::comm-ring-1 list
  assumes len1: 1 ≤ length e1 length e1 ≤ max-fields
    and len2: 1 ≤ length e2 length e2 ≤ max-fields
    and of-nat-inj:  $\bigwedge a b. \llbracket a \leq \text{max-fields}; b \leq \text{max-fields};$ 
       $(\text{of-nat } a :: 'f) = \text{of-nat } b \rrbracket \implies a = b$ 
    and eq: leaf-fold e1 max-fields = leaf-fold e2 max-fields
  shows e1 = e2

```

proof —

```

  have ne1: e1 ≠ [] and ne2: e2 ≠ []
    using len1(1) len2(1) by auto
  — Lane 0: the tag.
  have tag: e1 ! 0 = e2 ! 0
    using eq unfolding leaf-fold-def by simp
  — Lane 1: the length lane, then the of-nat premise.
  have lenlane: (of-nat (length e1) :: 'f) = of-nat (length e2)
    using eq unfolding leaf-fold-def by simp
  have len-eq: length e1 = length e2
    by (rule of-nat-inj[OF len1(2) len2(2) lenlane])
  — Lanes 2...: verbatim region plus identical padding.
  have rest: tl e1 @ replicate (leaf-pre-width max-fields - (length e1 + 1)) 0
    = tl e2 @ replicate (leaf-pre-width max-fields - (length e2 + 1)) 0
    using eq unfolding leaf-fold-def by simp
  from rest len-eq have tl-eq: tl e1 = tl e2 by simp
  — List extensionality.
  obtain a1 t1 where e1c: e1 = a1 # t1 using ne1 by (cases e1) auto
  obtain a2 t2 where e2c: e2 = a2 # t2 using ne2 by (cases e2) auto
  from tag e1c e2c have a1 = a2 by simp
  moreover from tl-eq e1c e2c have t1 = t2 by simp
  ultimately show e1 = e2 using e1c e2c by simp

```

qed

9 Collision-resistance reduction

The structural seam $h\text{-leaf} = \text{sponge } o \text{ leaf-fold}$ on the in-bound domain (Rust: *HashGadget::hash-leaf* = *hash-leaf-pre* *o* *leaf-fold*; the equation is restricted to the *Ok* domain because Rust *hash-leaf* errs outside it). The sponge is a fully abstract parameter: no collision-resistance assumption is placed on it. The reduction turns an $h\text{-leaf}$ collision on distinct in-bound encodings into an explicit sponge collision pair, which is precisely the interface the system-level "root binds state" interpretation consumes (Poseidon2 hardness stays a black-box assumption).

```

locale leaf-hash-structure =
  fixes h-leaf :: 'f::comm-ring-1 list  $\Rightarrow$  'd
    and sponge :: 'f list  $\Rightarrow$  'd
    and max-fields :: nat
  assumes h-leaf-decomp:
     $\bigwedge enc. \llbracket 1 \leq \text{length } enc; \text{length } enc \leq \text{max-fields} \rrbracket$ 
       $\implies h\text{-leaf } enc = \text{sponge } (\text{leaf-fold } enc \text{ max-fields})$ 
    and of-nat-inj-bound:
       $\bigwedge a \ b. \llbracket a \leq \text{max-fields}; b \leq \text{max-fields};$ 
         $(\text{of-nat } a :: 'f) = \text{of-nat } b \rrbracket \implies a = b$ 
begin

theorem h-leaf-collision-to-sponge-collision:
  assumes e1:  $1 \leq \text{length } e1$   $\text{length } e1 \leq \text{max-fields}$ 
    and e2:  $1 \leq \text{length } e2$   $\text{length } e2 \leq \text{max-fields}$ 
    and neg:  $e1 \neq e2$ 
    and collide:  $h\text{-leaf } e1 = h\text{-leaf } e2$ 
  shows  $\exists p1 \ p2. p1 \neq p2 \wedge \text{sponge } p1 = \text{sponge } p2$ 
proof -
  have fold-neg:  $\text{leaf-fold } e1 \text{ max-fields} \neq \text{leaf-fold } e2 \text{ max-fields}$ 
  proof
    assume feq:  $\text{leaf-fold } e1 \text{ max-fields} = \text{leaf-fold } e2 \text{ max-fields}$ 
    have  $e1 = e2$ 
    proof (rule leaf-fold-inj[OF e1 e2 - feq])
      fix a b
      assume  $a \leq \text{max-fields}$  and  $b \leq \text{max-fields}$ 
      and  $(\text{of-nat } a :: 'f) = \text{of-nat } b$ 
      then show  $a = b$  by (rule of-nat-inj-bound)
    qed
    with neg show False ..
  qed
  have  $\text{sponge } (\text{leaf-fold } e1 \text{ max-fields}) = \text{sponge } (\text{leaf-fold } e2 \text{ max-fields})$ 
    using collide h-leaf-decomp[OF e1] h-leaf-decomp[OF e2] by simp
  with fold-neg show ?thesis by blast
qed

end

```

9.1 The encode domain

Bridge to the leaf-state encoding of *SMT-Semantics*: every *encode* result is nonempty (tag lane first), and it stays within the bound whenever the payload encoder respects it - so encoded leaf states live in the injectivity domain of the fold.

lemma *encode-length-lb*: $1 \leq \text{length } (\text{encode } \text{encp } l)$
by (*cases l*) *simp-all*

lemma *encode-length-ub*:
assumes $\bigwedge v. 1 + \text{length } (\text{encp } v) \leq \text{max-fields}$
and $1 \leq \text{max-fields}$
shows $\text{length } (\text{encode } \text{encp } l) \leq \text{max-fields}$
using *assms* **by** (*cases l*) *simp-all*

context *leaf-hash-structure*
begin

corollary *encoded-state-collision-to-sponge-collision*:

— Leaf-state level reading of the reduction: two leaf states whose encodings differ (e.g. any two distinct states under an injective payload encoder) cannot collide under *h-leaf* without exhibiting a sponge collision.

fixes *encp* :: '*v* $\Rightarrow$ '*f* list
assumes *encp-bound*: $\bigwedge v. 1 + \text{length } (\text{encp } v) \leq \text{max-fields}$
and *max-pos*: $1 \leq \text{max-fields}$
and *neq*: $\text{encode } \text{encp } l1 \neq \text{encode } \text{encp } l2$
and *collide*: $\text{h-leaf } (\text{encode } \text{encp } l1) = \text{h-leaf } (\text{encode } \text{encp } l2)$
shows $\exists p1\ p2. p1 \neq p2 \wedge \text{sponge } p1 = \text{sponge } p2$
by (*rule h-leaf-collision-to-sponge-collision*[*OF*
 $\text{encode-length-lb } \text{encode-length-ub}$ [*OF encp-bound max-pos*]
 $\text{encode-length-lb } \text{encode-length-ub}$ [*OF encp-bound max-pos*]
neq collide])

end

10 Firing instances (non-vacuity witnesses)

The theorems actually fire. The *of-nat* premise is discharged over a characteristic-zero ring (int, by simp); over the concrete KoalaBear prime field it holds because the bound (default 31) is far below $p = 2^{31} - 2^{24} + 1$ (discharged at instantiation).

lemma *leaf-fold-inj-int*:
fixes *e1 e2* :: *int* list
assumes $1 \leq \text{length } e1$ $\text{length } e1 \leq \text{max-fields}$
and $1 \leq \text{length } e2$ $\text{length } e2 \leq \text{max-fields}$
and $\text{leaf-fold } e1\ \text{max-fields} = \text{leaf-fold } e2\ \text{max-fields}$
shows $e1 = e2$
by (*rule leaf-fold-inj*[*OF* *assms*(1,2,3,4) - *assms*(5)]) *simp*

lemma *leaf-fold-inj-fires*:

— A pre-image determines its in-bound encoding uniquely.

fixes *e* :: *int* list
assumes $1 \leq \text{length } e$ **and** $\text{length } e \leq 31$
and $\text{leaf-fold } e\ 31 = \text{leaf-fold } [1, 42]\ 31$
shows $e = [1, 42]$
by (*rule leaf-fold-inj-int*[*OF* *assms*(1,2) - - *assms*(3)]) *simp-all*

10.1 The two collision classes the fold closes

(a) The wrap-collision class (the audited CRITICAL break). The pre-fix fold wrapped the encoding onto 16 lanes additively ($s[1 + (j \bmod 15)] += v$), so encodings agreeing on all wrap-slot SUMS shared a pre-image: below, rest positions 0 and 15 landed in the same slot and $5 + 0 = 3 + 2$. The lossless fold separates the pair in the first verbatim lane. (Pin against the Rust property test, class (a).)

definition *former-wrap-a* :: int list **where**
former-wrap-a = 1 # 5 # replicate 14 0 @ [0]

definition *former-wrap-b* :: int list **where**
former-wrap-b = 1 # 3 # replicate 14 0 @ [2]

lemma *former-wrap-class-separated*:
former-wrap-a $\neq$ *former-wrap-b*
leaf-fold *former-wrap-a* 31 $\neq$ *leaf-fold* *former-wrap-b* 31
by (*simp-all* add: *former-wrap-a-def* *former-wrap-b-def* *leaf-fold-def*)

(b) The trailing-zero / length-shift class - the reason the length lane exists. [1, 7] and [1, 7, 0] agree on the tag lane and on the whole verbatim-plus-padding region; ONLY the length lane separates their pre-images. Without it the fold would identify them, and *h-leaf* would collide without any sponge collision - the forgery class of the audit. (Pin against the Rust property test, class (b).)

lemma *zero-extension-class-separated*:
leaf-fold [1, 7] 31 $\neq$ (*leaf-fold* [1, 7, 0] 31 :: int list)
by (*simp* add: *leaf-fold-def*)

10.2 The reduction fires

With a toy, deliberately WEAK sponge (the lane sum) an *h-leaf* collision on distinct in-bound encodings materializes an explicit sponge collision pair: [1, 5, 0] and [1, 3, 2] have equal lane sums, and by injectivity their pre-images are distinct - exactly such a pair. Note what this shows: the reduction places NO collision-resistance assumption on the sponge; hardness lives in the Poseidon2 black box, structure lives here.

interpretation *toy-leaf-hash*: *leaf-hash-structure*
 $\lambda enc. \text{sum-list } (\text{leaf-fold } enc \ 31) \text{ sum-list} :: \text{int list} \Rightarrow \text{int } 31$
by *unfold-locales simp-all*

lemma *toy-sponge-collision-exhibited*:
 $\exists p1 \ p2 :: \text{int list}. p1 \neq p2 \wedge \text{sum-list } p1 = \text{sum-list } p2$
proof (*rule* *toy-leaf-hash.h-leaf-collision-to-sponge-collision*)
show $1 \leq \text{length } [1 :: \text{int}, 5, 0]$ **and** $\text{length } [1 :: \text{int}, 5, 0] \leq 31$
and $1 \leq \text{length } [1 :: \text{int}, 3, 2]$ **and** $\text{length } [1 :: \text{int}, 3, 2] \leq 31$
by *simp-all*
show $[1 :: \text{int}, 5, 0] \neq [1, 3, 2]$ **by** *simp*
show $\text{sum-list } (\text{leaf-fold } [1 :: \text{int}, 5, 0] \ 31) = \text{sum-list } (\text{leaf-fold } [1 :: \text{int}, 3, 2] \ 31)$
by (*simp* add: *leaf-fold-def* *sum-list-replicate*)
qed
end

```

theory Compiler-Model
  imports SMT-Semantics
begin

```

11 Operation kinds

Circuit structure depends only on the operation KIND (witness-independent compile cache - the Module-3 batching premise). Rust: *SmtOpKind*.

```

datatype op-kind = KMembership | KNonMembership | KUpdate

```

```

fun kind-of :: 'v smt-op  $\Rightarrow$  op-kind where
  kind-of (Membership k v) = KMembership
| kind-of (NonMembership k) = KNonMembership
| kind-of (Update k old new) = KUpdate

```

12 List segment utilities

```

definition seg :: nat  $\Rightarrow$  nat  $\Rightarrow$  'a list  $\Rightarrow$  'a list where
  seg a w xs = take w (drop a xs)

```

```

lemma length-seg:  $a + w \leq \text{length } xs \implies \text{length } (\text{seg } a \ w \ xs) = w$ 
by (simp add: seg-def)

```

```

lemma seg-nth:  $i < w \implies a + w \leq \text{length } xs \implies \text{seg } a \ w \ xs ! i = xs ! (a + i)$ 
by (simp add: seg-def add.commute)

```

```

lemma seg-append-in:  $a + w \leq \text{length } xs \implies \text{seg } a \ w \ (xs @ ys) = \text{seg } a \ w \ xs$ 
by (simp add: seg-def)

```

Indexing a concatenation of uniform-length blocks.

```

lemma concat-map-uniform-length:
  assumes  $\bigwedge j. j < n \implies \text{length } (f \ j) = W$ 
  shows  $\text{length } (\text{concat } (\text{map } f \ [0 ..< n])) = n * W$ 
  using assms by (induction n) (auto simp: length-concat)

```

```

lemma concat-map-uniform-nth:
  assumes uni:  $\bigwedge j. j < n \implies \text{length } (f \ j) = W$ 
  and l:  $l < n$  and i:  $i < W$ 
  shows  $\text{concat } (\text{map } f \ [0 ..< n]) ! (l * W + i) = f \ l ! i$ 
proof -
  have split:  $[0 ..< n] = [0 ..< l] @ l \# [Suc \ l ..< n]$ 
  proof -
    from l have n-decomp:  $n = l + Suc \ (n - Suc \ l)$  by arith
    have  $[0 ..< l + Suc \ (n - Suc \ l)] = [0 ..< l] @ [l ..< l + Suc \ (n - Suc \ l)]$ 
    by (rule upt-add-eq-append) simp
    moreover have  $[l ..< l + Suc \ (n - Suc \ l)] = l \# [Suc \ l ..< l + Suc \ (n - Suc \ l)]$ 
    by (rule upt-conv-Cons) simp
    ultimately have  $[0 ..< l + Suc \ (n - Suc \ l)]$ 
       $= [0 ..< l] @ l \# [Suc \ l ..< l + Suc \ (n - Suc \ l)]$  by simp

```

```

    then show ?thesis using n-decomp by simp
qed
have len-pre: length (concat (map f [0 ..< l])) = l * W
  using uni l by (intro concat-map-uniform-length) auto
have concat (map f [0 ..< n]) = concat (map f [0 ..< l]) @ f l @ concat (map f [Suc l ..< n])
  by (simp add: split)
then show ?thesis
  using len-pre i uni[OF l]
  by (simp add: nth-append)
qed

```

13 Constraint descriptors and the residual output layer

A residual constraint over the layer below (the evaluation-stack top): $RDiff\ a\ b$ vanishes iff wire a equals wire b ; $RBool\ a$ is the key-bit booleanity residual $s^*(s-1)$; $RTag\ a$ is the NonMembership tag residual $tag^*(tag-2)$ (roots exactly 0,2 in an integral domain). Rust: the residual nodes pushed to ‘outputs’ in compile.rs.

datatype *res-desc* =

```

  RDiff nat nat
| RBool nat
| RTag nat

```

fun *res-gates* :: *nat* $\Rightarrow$ *res-desc* $\Rightarrow$ '*f::comm-ring-1 gate list* **where**

```

  res-gates z (RDiff a b) =
    [ (| g-kind = GLin, g-out = z, g-in1 = a, g-in2 = a, g-coeff = 1 |),
      (| g-kind = GLin, g-out = z, g-in1 = b, g-in2 = b, g-coeff = - 1 |) ]
| res-gates z (RBool a) =
    [ (| g-kind = GMul, g-out = z, g-in1 = a, g-in2 = a, g-coeff = 1 |),
      (| g-kind = GLin, g-out = z, g-in1 = a, g-in2 = a, g-coeff = - 1 |) ]
| res-gates z (RTag a) =
    [ (| g-kind = GMul, g-out = z, g-in1 = a, g-in2 = a, g-coeff = 1 |),
      (| g-kind = GLin, g-out = z, g-in1 = a, g-in2 = a, g-coeff = - 2 |) ]

```

fun *res-sem* :: '*f::comm-ring-1 list* $\Rightarrow$ *res-desc* $\Rightarrow$ '*f* **where**

```

  res-sem below (RDiff a b) = below ! a - below ! b
| res-sem below (RBool a) = below ! a * below ! a - below ! a
| res-sem below (RTag a) = below ! a * below ! a - 2 * below ! a

```

The residual layer: one output wire per descriptor, generated by enumeration. Width is padded to the next power of two implicitly by taking *layer-width-bits* = *length ds* ($n < 2^{\wedge} n$), a modeling freedom - padding wires carry no gates and no constants, hence evaluate to 0 and never obstruct acceptance.

definition *mk-res-layer* :: *res-desc list* $\Rightarrow$ '*f::comm-ring-1 layer* **where**

```

  mk-res-layer ds =
    (| layer-width-bits = length ds,
      layer-gates = concat (map ( $\lambda(z, rd).$  res-gates z rd) (List.enumerate 0 ds)),
      layer-consts = [] |)

```

13.1 Evaluation of the residual layer

lemma *res-gates-out*: $g \in \text{set } (\text{res-gates } z \text{ rd}) \implies g\text{-out } g = z$

```

by (cases rd) auto

lemma filter-res-gates-same:
  filter ( $\lambda g. g\text{-out } g = z$ ) (res-gates z rd) = res-gates z rd
  by (cases rd) auto

lemma filter-res-gates-other:
   $k \neq z \implies \text{filter } (\lambda g. g\text{-out } g = z) (\text{res-gates } k \text{ rd}) = []$ 
  by (cases rd) auto

lemma filter-enumerate-res-gates-miss:
  assumes  $z < n \vee n + \text{length } ds \leq z$ 
  shows filter ( $\lambda g. g\text{-out } g = z$ )
    (concat (map ( $\lambda(k, rd). \text{res-gates } k \text{ rd}$ ) (List.enumerate n ds))) = []
  using assms
proof (induction ds arbitrary: n)
  case Nil then show ?case by simp
next
  case (Cons rd ds)
  then have  $z \neq n$  and rest:  $z < \text{Suc } n \vee \text{Suc } n + \text{length } ds \leq z$  by auto
  with Cons.IH[OF rest] show ?case
    by (simp add: filter-res-gates-other)
qed

lemma filter-enumerate-res-gates:
  assumes  $n \leq z < n + \text{length } ds$ 
  shows filter ( $\lambda g. g\text{-out } g = z$ )
    (concat (map ( $\lambda(k, rd). \text{res-gates } k \text{ rd}$ ) (List.enumerate n ds)))
    = (res-gates z (ds ! (z - n)) :: 'f::comm-ring-1 gate list)
  using assms
proof (induction ds arbitrary: n)
  case Nil then show ?case by simp
next
  case (Cons rd ds)
  show ?case
  proof (cases n = z)
    case True
    have filter ( $\lambda g. g\text{-out } g = z$ )
      (concat (map ( $\lambda(k, rd). \text{res-gates } k \text{ rd}$ ) (List.enumerate (Suc n) ds))) = []
    using True by (intro filter-enumerate-res-gates-miss) simp
    with True show ?thesis
      by (simp add: filter-res-gates-same)
  next
    case False
    with Cons.prem1 have le:  $\text{Suc } n \leq z$  and lt:  $z < \text{Suc } n + \text{length } ds$  by auto
    have ih: filter ( $\lambda g. g\text{-out } g = z$ )
      (concat (map ( $\lambda(k, rd). \text{res-gates } k \text{ rd}$ ) (List.enumerate (Suc n) ds)))
      = (res-gates z (ds ! (z - Suc n)) :: 'f gate list)
    by (rule Cons.IH[OF le lt])
    have idx:  $(rd \# ds) ! (z - n) = ds ! (z - \text{Suc } n)$ 
    using le by (simp add: Suc-diff-le nth-Cons')
    from False show ?thesis
      by (simp add: filter-res-gates-other ih idx)
  end
end

```

qed
qed

lemma *filter-enumerate-res-gates-beyond*:

assumes $n + \text{length } ds \leq z$

shows *filter* ($\lambda g. g\text{-out } g = z$)

(*concat* (*map* ($\lambda(k, rd). \text{res-gates } k \text{ } rd$) (*List.enumerate* $n \text{ } ds$))) = []

using *assms* **by** (*intro filter-enumerate-res-gates-miss*) *simp*

lemma *res-gates-contrib-sum*:

sum-list (*map* ($\lambda g. \text{gate-contrib } g \text{ below}$) (*res-gates* $z \text{ } rd$)) = *res-sem below rd*

by (*cases rd*) (*simp-all add: gate-contrib-def algebra-simps*)

lemma *mk-res-layer-width*: *layer-width* (*mk-res-layer* ds) = $2^{\text{length } ds}$

by (*simp add: mk-res-layer-def layer-width-def*)

lemma *res-index-width*:

$z < \text{length } ds \implies z < \text{layer-width } (\text{mk-res-layer } ds)$

unfolding *mk-res-layer-width* **by** (*meson less-exp less-trans*)

lemma *mk-res-layer-gates*:

layer-gates (*mk-res-layer* ds)

= *concat* (*map* ($\lambda(z, rd). \text{res-gates } z \text{ } rd$) (*List.enumerate* $0 \text{ } ds$))

by (*simp add: mk-res-layer-def*)

lemma *mk-res-layer-consts*: *const-at* (*mk-res-layer* ds) $z = 0$

by (*simp add: mk-res-layer-def const-at-def*)

lemma *mk-res-layer-eval-in*:

assumes $z: z < \text{length } ds$

shows *layer-eval* (*mk-res-layer* ds) *below* ! $z = \text{res-sem below } (ds \text{ } ! z)$

proof –

have *flt*: *filter* ($\lambda g. g\text{-out } g = z$) (*layer-gates* (*mk-res-layer* ds))
= *res-gates* $z \text{ } (ds \text{ } ! z)$

using *filter-enumerate-res-gates[of 0 z ds]* z

by (*simp add: mk-res-layer-gates*)

show *?thesis*

using *layer-eval-nth[OF res-index-width[OF z]]*

by (*simp add: flt mk-res-layer-consts res-gates-contrib-sum*)

qed

lemma *mk-res-layer-eval-pad*:

fixes *below* :: '*f*::comm-ring-1 list

assumes *ge*: $\text{length } ds \leq z$

and *zw*: $z < \text{layer-width } (\text{mk-res-layer } ds :: 'f \text{ layer})$

shows *layer-eval* (*mk-res-layer* ds) *below* ! $z = 0$

proof –

have *flt*: *filter* ($\lambda g. g\text{-out } g = z$) (*layer-gates* (*mk-res-layer* $ds :: 'f \text{ layer}$))) = []

using *filter-enumerate-res-gates-beyond[of 0 ds z]* *ge*

by (*simp add: mk-res-layer-gates*)

show *?thesis*

by (*simp add: layer-eval-nth[OF zw] flt mk-res-layer-consts*)

qed

Acceptance of a residual layer over a fixed below-vector.

lemma *mk-res-layer-accept-iff*:

$(\forall v \in \text{set } (\text{layer-eval } (\text{mk-res-layer } ds) \text{ below}). v = 0)$

$\longleftrightarrow (\forall rd \in \text{set } ds. \text{res-sem below } rd = 0)$

proof

assume L : $\forall v \in \text{set } (\text{layer-eval } (\text{mk-res-layer } ds) \text{ below}). v = 0$

show $\forall rd \in \text{set } ds. \text{res-sem below } rd = 0$

proof

fix rd **assume** $rd \in \text{set } ds$

then obtain z **where** z : $z < \text{length } ds$ $ds ! z = rd$

by (*meson in-set-conv-nth*)

have $\text{layer-eval } (\text{mk-res-layer } ds) \text{ below } ! z$

$\in \text{set } (\text{layer-eval } (\text{mk-res-layer } ds) \text{ below})$

using $\text{res-index-width}[OF\ z(1)]$ **by** (*intro nth-mem*) *simp*

then have zero : $\text{layer-eval } (\text{mk-res-layer } ds) \text{ below } ! z = 0$

using L **by** *blast*

show $\text{res-sem below } rd = 0$

using $\text{zero } z(2)$ **by** (*simp add: mk-res-layer-eval-in[OF z(1)]*)

qed

next

assume R : $\forall rd \in \text{set } ds. \text{res-sem below } rd = 0$

show $\forall v \in \text{set } (\text{layer-eval } (\text{mk-res-layer } ds) \text{ below}). v = 0$

proof

fix v **assume** $v \in \text{set } (\text{layer-eval } (\text{mk-res-layer } ds) \text{ below})$

then obtain z **where** z : $z < \text{length } (\text{layer-eval } (\text{mk-res-layer } ds) \text{ below})$

and v : $v = \text{layer-eval } (\text{mk-res-layer } ds) \text{ below } ! z$

unfolding *in-set-conv-nth* **by** *blast*

show $v = 0$

proof (*cases* $z < \text{length } ds$)

case *True*

have ev : $\text{layer-eval } (\text{mk-res-layer } ds) \text{ below } ! z = \text{res-sem below } (ds ! z)$

by (*rule mk-res-layer-eval-in[OF True]*)

have $\text{res-sem below } (ds ! z) = 0$

using R *nth-mem[OF True]* **by** *blast*

with $ev\ v$ **show** *?thesis* **by** *simp*

next

case *False*

then have $\text{layer-eval } (\text{mk-res-layer } ds) \text{ below } ! z = 0$

using z **by** (*simp add: mk-res-layer-eval-pad*)

with v **show** *?thesis* **by** *simp*

qed

qed

qed

14 Key-bit multiplexer (Rust: build_mux)

$\text{left}[i] = \text{acc}[i] + s * (\text{sib}[i] - \text{acc}[i])$, $\text{right}[i] = \text{sib}[i] + s * (\text{acc}[i] - \text{sib}[i])$: selector 0 keeps (acc, sib), selector 1 swaps to (sib, acc) - literally the compress order of *MerklePath::compute-root* (key low bit first).

definition *mux-left* :: $'f::\text{comm-ring-1} \Rightarrow 'f\ \text{list} \Rightarrow 'f\ \text{list} \Rightarrow 'f\ \text{list}$ **where**

mux-left $s\ as\ ss = \text{map2 } (\lambda a\ sb. a + s * (sb - a))\ as\ ss$

definition *mux-right* :: 'f::comm-ring-1 $\Rightarrow$ 'f list $\Rightarrow$ 'f list $\Rightarrow$ 'f list **where**
mux-right s as ss = map2 ($\lambda a sb. sb + s * (a - sb)$) as ss

lemma *mux-zero*:
assumes *length as = length ss*
shows *mux-left 0 as ss = as and mux-right 0 as ss = ss*
using *assms*
unfolding *mux-left-def mux-right-def*
by (*induction as ss rule: list-induct2*) *auto*

lemma *mux-one*:
assumes *length as = length ss*
shows *mux-left 1 as ss = ss and mux-right 1 as ss = as*
using *assms*
unfolding *mux-left-def mux-right-def*
by (*induction as ss rule: list-induct2*) *auto*

15 Kind tags and key bits

Leaf-encoding tag lane (domain separation): 0/1/2.

fun *leaf-tag-val* :: 'v leaf-state $\Rightarrow$ 'f::comm-ring-1 **where**
leaf-tag-val Empty = 0
| *leaf-tag-val (Occupied v) = 1*
| *leaf-tag-val Tombstone = 2*

Key bit *l* as a field element (Rust: (*key.0* >> *l*) & 1).

definition *kbit* :: nat $\Rightarrow$ nat $\Rightarrow$ 'f::comm-ring-1 **where**
kbit key l = of-nat ((key div 2 $\wedge$ l) mod 2)

lemma *kbit-cases*: *kbit key l = 0 $\vee$ kbit key l = 1*
proof –
have (*key div 2 $\wedge$ l*) mod 2 = 0 $\vee$ (*key div 2 $\wedge$ l*) mod 2 = 1 **by** *auto*
then show ?thesis **by** (*auto simp: kbit-def*)
qed

lemma *kbit-bool*: *kbit key l * (kbit key l - 1) = 0*
proof –
have (*key div 2 $\wedge$ l*) mod 2 = 0 $\vee$ (*key div 2 $\wedge$ l*) mod 2 = 1 **by** *auto*
then show ?thesis **unfolding** *kbit-def* **by** (*elim disjE*) *simp-all*
qed

16 Zero padding

definition *pad-to* :: nat $\Rightarrow$ 'f::comm-ring-1 list $\Rightarrow$ 'f list **where**
pad-to n xs = xs @ replicate (n - length xs) 0

lemma *length-pad-to*: *length xs $\leq$ n $\implies$ length (pad-to n xs) = n*
by (*simp add: pad-to-def*)

lemma *pad-to-nth*: *i < length xs $\implies$ pad-to n xs ! i = xs ! i*

by (simp add: pad-to-def nth-append)

lemma *seg-pad-to*: $a + w \leq \text{length } xs \implies \text{seg } a \ w \ (\text{pad-to } n \ xs) = \text{seg } a \ w \ xs$
 by (simp add: pad-to-def seg-append-in)

17 The compiler model locale (representation layer)

Extends the frozen SMT semantics with the field-level REPRESENTATION of digests and leaf pre-images the circuit operates on:

digest-repr a digest as W field lanes (Rust: Digest<BaseField>, W=8); injective - two digests are equal iff their lane vectors are (the wire-level equality tests decide digest equality) *leaf-repr* the FIXED lossless leaf pre-image (Rust: *leaf-fold* of the leaf encoding, L = *leaf-pre-width* lanes); lane 0 carries the encoding tag (domain separation). Injectivity of *leaf-fold* is lemma (b) territory (*Leaf-Fold.thy*) and is NOT consumed here - Theorem A only needs the tag lane. *hl* / *hn* the lane-level hash maps the in-circuit hash sub-circuits compute (Rust: *hash-leaf-pre* / compress gadgets); their agreement with the abstract *h-leaf* / *h-node* is the generator-correctness boundary (black-box hash layers).

No collision resistance and no injectivity of *h-leaf*/*h-node* is assumed (Theorem A is a deterministic equivalence).

locale *compiler-model-base* = *smt-semantics params h-leaf h-node*
 for *params* :: *smt-params*
 and *h-leaf* :: '*v* leaf-state $\Rightarrow$ '*d*
 and *h-node* :: '*d* $\Rightarrow$ '*d* +
 fixes *W* :: nat and *L* :: nat
 and *digest-repr* :: '*d* $\Rightarrow$ '*f*::idom list
 and *leaf-repr* :: '*v* leaf-state $\Rightarrow$ '*f* list
 and *hl* :: '*f* list $\Rightarrow$ '*f* list
 and *hn* :: '*f* list $\Rightarrow$ '*f* list
 assumes *L-pos*: $0 < L$
 and *digest-len* [simp]: $\bigwedge x. \text{length } (\text{digest-repr } x) = W$
 and *digest-inj*: $\bigwedge x \ y. \text{digest-repr } x = \text{digest-repr } y \implies x = y$
 and *leaf-len* [simp]: $\bigwedge lf. \text{length } (\text{leaf-repr } lf) = L$
 and *leaf-tag*: $\bigwedge lf. \text{leaf-repr } lf ! 0 = \text{leaf-tag-val } lf$
 and *hl-len* [simp]: $\bigwedge xs. \text{length } (\text{hl } xs) = W$
 and *hn-len* [simp]: $\bigwedge a \ b. \text{length } (\text{hn } a \ b) = W$
 and *hl-correct*: $\bigwedge lf. \text{hl } (\text{leaf-repr } lf) = \text{digest-repr } (\text{h-leaf } lf)$
 and *hn-correct*: $\bigwedge a \ b. \text{hn } (\text{digest-repr } a) \ (\text{digest-repr } b) = \text{digest-repr } (\text{h-node } a \ b)$
 begin

abbreviation *dd* :: nat **where** *dd* $\equiv$ *depth params*

17.1 Input-vector layout (Rust: InputLayout, same segment order)

Read-only ops (Membership / NonMembership).

definition *ro-lp* :: nat **where** *ro-lp* = 0
definition *ro-acc* :: nat $\Rightarrow$ nat **where** *ro-acc* *l* = *L* + *W* * *l*
definition *ro-sib* :: nat $\Rightarrow$ nat **where** *ro-sib* *l* = *L* + *W* * (*dd* + 1) + *W* * *l*
definition *ro-kb* :: nat $\Rightarrow$ nat **where** *ro-kb* *l* = *L* + *W* * (2 * *dd* + 1) + *l*
definition *ro-root* :: nat **where** *ro-root* = *L* + *W* * (2 * *dd* + 1) + *dd*

definition $ro\text{-}vd :: nat$ **where** $ro\text{-}vd = ro\text{-}root + W$

definition $ro\text{-}width :: nat$ **where** $ro\text{-}width = ro\text{-}vd + W$

Update (two leaf pre-images, two accumulator chains, two roots).

definition $up\text{-}lp :: nat$ **where** $up\text{-}lp = 0$

definition $up\text{-}lp2 :: nat$ **where** $up\text{-}lp2 = L$

definition $up\text{-}acc :: nat \Rightarrow nat$ **where** $up\text{-}acc\ l = 2 * L + W * l$

definition $up\text{-}acc2 :: nat \Rightarrow nat$ **where** $up\text{-}acc2\ l = 2 * L + W * (dd + 1) + W * l$

definition $up\text{-}sib :: nat \Rightarrow nat$ **where** $up\text{-}sib\ l = 2 * L + 2 * W * (dd + 1) + W * l$

definition $up\text{-}kb :: nat \Rightarrow nat$ **where** $up\text{-}kb\ l = 2 * L + W * (3 * dd + 2) + l$

definition $up\text{-}root :: nat$ **where** $up\text{-}root = 2 * L + W * (3 * dd + 2) + dd$

definition $up\text{-}root2 :: nat$ **where** $up\text{-}root2 = up\text{-}root + W$

definition $up\text{-}vd :: nat$ **where** $up\text{-}vd = up\text{-}root2 + W$

definition $up\text{-}width :: nat$ **where** $up\text{-}width = up\text{-}vd + W$

Input vectors are zero-padded to a power of two. The model takes $input\text{-}width\text{-}bits = width$ itself ($n < 2^n$) - a deliberately generous padding; wire-width economy is a refinement concern, not semantics.

fun $in\text{-}width :: op\text{-}kind \Rightarrow nat$ **where**

$in\text{-}width\ KMembership = ro\text{-}width$

| $in\text{-}width\ KNonMembership = ro\text{-}width$

| $in\text{-}width\ KUpdate = up\text{-}width$

definition $iwb :: op\text{-}kind \Rightarrow nat$ **where** $iwb\ k = in\text{-}width\ k$

definition $ivlen :: op\text{-}kind \Rightarrow nat$ **where** $ivlen\ k = 2^{iwb\ k}$

lemma $in\text{-}width\text{-}le\text{-}ivlen$: $in\text{-}width\ k \leq ivlen\ k$

by ($simp\ add$: $ivlen\text{-}def\ iwb\text{-}def\ less\text{-}imp\text{-}le[OF\ less\text{-}exp]$)

17.2 Evaluation-stack output specification

$ev\text{-}ro\ xs$ / $ev\text{-}up\ xs$: the vector the hash/mux evaluation stack must produce on top of input vector xs - the in-circuit hash values, the mux images feeding them, and the carried inputs the residual layer compares against. The mux arithmetic is EXPLICIT here (the key-bit mux is a soundness-critical constraint structure); only the hash maps hl/hn are abstract.

PADDING (non-vacuity fix): $layer\text{-}eval$ always produces a vector of power-of-two length ($layer\text{-}width = 2^{layer\text{-}width\text{-}bits}$), so the stack specification target must itself have power-of-two length or the $stack\text{-}spec$ assumptions would be UNSATISFIABLE - the core (unpadded) lengths of the two vectors are locked in the open interval $(len\text{-}ro, 2 * len\text{-}ro)$ relative to each other, so no width parameters can make both powers of two. The specification therefore zero-pads the core vector to width $2^{core\text{-}length}$ (the same $n < 2^n$ modeling freedom as iwb and $mk\text{-}res\text{-}layer$), mirroring the Rust builder whose layers are power-of-two wide with unused wires at zero.

definition $ro\text{-}m\text{-}xl :: 'f\ list \Rightarrow nat \Rightarrow 'f\ list$ **where**

$ro\text{-}m\text{-}xl\ xs\ l = mux\text{-}left\ (xs\ !\ ro\text{-}kb\ l)\ (seg\ (ro\text{-}acc\ l)\ W\ xs)\ (seg\ (ro\text{-}sib\ l)\ W\ xs)$

definition $ro\text{-}m\text{-}xr :: 'f\ list \Rightarrow nat \Rightarrow 'f\ list$ **where**

$ro\text{-}m\text{-}xr\ xs\ l = mux\text{-}right\ (xs\ !\ ro\text{-}kb\ l)\ (seg\ (ro\text{-}acc\ l)\ W\ xs)\ (seg\ (ro\text{-}sib\ l)\ W\ xs)$

definition $ev-ro-core :: 'f list \Rightarrow 'f list$ **where**

```

ev-ro-core xs =
  hl (seg ro-lp L xs)
  @ concat (map ( $\lambda l. hn (ro-mxl xs l) (ro-mxr xs l)$ ) [0 ..< dd])
  @ concat (map ( $\lambda l. seg (ro-acc l) W xs$ ) [0 ..< Suc dd])
  @ map ( $\lambda l. xs ! ro-kb l$ ) [0 ..< dd]
  @ seg ro-root W xs
  @ seg ro-vd W xs
  @ [xs ! ro-lp]

```

definition $ev-hl :: nat$ **where** $ev-hl = 0$

definition $ev-hn :: nat \Rightarrow nat$ **where** $ev-hn l = W + W * l$

definition $ev-acc :: nat \Rightarrow nat$ **where** $ev-acc l = W * (dd + 1) + W * l$

definition $ev-kb :: nat \Rightarrow nat$ **where** $ev-kb l = W * (2 * dd + 2) + l$

definition $ev-root :: nat$ **where** $ev-root = W * (2 * dd + 2) + dd$

definition $ev-vd :: nat$ **where** $ev-vd = ev-root + W$

definition $ev-tag :: nat$ **where** $ev-tag = ev-vd + W$

definition $ev-ro-len :: nat$ **where** $ev-ro-len = ev-tag + 1$

definition $ev-ro :: 'f list \Rightarrow 'f list$ **where**

```

ev-ro xs = pad-to (2 ^ ev-ro-len) (ev-ro-core xs)

```

definition $up-mxl :: 'f list \Rightarrow (nat \Rightarrow nat) \Rightarrow nat \Rightarrow 'f list$ **where**

```

up-mxl xs acc l = mux-left (xs ! up-kb l) (seg (acc l) W xs) (seg (up-sib l) W xs)

```

definition $up-mxr :: 'f list \Rightarrow (nat \Rightarrow nat) \Rightarrow nat \Rightarrow 'f list$ **where**

```

up-mxr xs acc l = mux-right (xs ! up-kb l) (seg (acc l) W xs) (seg (up-sib l) W xs)

```

definition $ev-up-core :: 'f list \Rightarrow 'f list$ **where**

```

ev-up-core xs =
  hl (seg up-lp L xs)
  @ hl (seg up-lp2 L xs)
  @ concat (map ( $\lambda l. hn (up-mxl xs up-acc l) (up-mxr xs up-acc l)$ ) [0 ..< dd])
  @ concat (map ( $\lambda l. hn (up-mxl xs up-acc2 l) (up-mxr xs up-acc2 l)$ ) [0 ..< dd])
  @ concat (map ( $\lambda l. seg (up-acc l) W xs$ ) [0 ..< Suc dd])
  @ concat (map ( $\lambda l. seg (up-acc2 l) W xs$ ) [0 ..< Suc dd])
  @ map ( $\lambda l. xs ! up-kb l$ ) [0 ..< dd]
  @ seg up-root W xs
  @ seg up-root2 W xs
  @ seg up-vd W xs

```

definition $ev2-hl :: nat$ **where** $ev2-hl = 0$

definition $ev2-hl2 :: nat$ **where** $ev2-hl2 = W$

definition $ev2-hn :: nat \Rightarrow nat$ **where** $ev2-hn l = 2 * W + W * l$

definition $ev2-hn2 :: nat \Rightarrow nat$ **where** $ev2-hn2 l = 2 * W + W * dd + W * l$

definition $ev2-acc :: nat \Rightarrow nat$ **where** $ev2-acc l = W * (2 * dd + 2) + W * l$

definition $ev2-acc2 :: nat \Rightarrow nat$ **where** $ev2-acc2 l = W * (3 * dd + 3) + W * l$

definition $ev2-kb :: nat \Rightarrow nat$ **where** $ev2-kb l = W * (4 * dd + 4) + l$

definition $ev2-root :: nat$ **where** $ev2-root = W * (4 * dd + 4) + dd$

definition $ev2-root2 :: nat$ **where** $ev2-root2 = ev2-root + W$

definition $ev2-vd :: nat$ **where** $ev2-vd = ev2-root2 + W$

definition $ev\text{-}up\text{-}len :: nat$ **where** $ev\text{-}up\text{-}len = ev2\text{-}vd + W$

definition $ev\text{-}up :: 'f\ list \Rightarrow 'f\ list$ **where**
 $ev\text{-}up\ xs = pad\text{-}to\ (2 \wedge ev\text{-}up\text{-}len)\ (ev\text{-}up\text{-}core\ xs)$

17.3 Residual constraint descriptors (the constraint SET of compile.rs)

Read-only ops: leaf-hash binding, dd compression transitions, root binding, value-digest binding ($acc\text{-}0 = vd$: the leaf being authenticated), key-bit booleanity, and - NonMembership only - the tag residual.

definition $descs\text{-}ro :: op\text{-}kind \Rightarrow res\text{-}desc\ list$ **where**

$descs\text{-}ro\ k =$
 $map\ (\lambda i. RDiff\ (ev\text{-}hl + i)\ (ev\text{-}acc\ 0 + i))\ [0 \dots W]$
 $@\ concat\ (map\ (\lambda l. map\ (\lambda i. RDiff\ (ev\text{-}hn\ l + i)\ (ev\text{-}acc\ (Suc\ l) + i))\ [0 \dots W])\ [0 \dots dd])$
 $@\ map\ (\lambda i. RDiff\ (ev\text{-}acc\ dd + i)\ (ev\text{-}root + i))\ [0 \dots W]$
 $@\ map\ (\lambda i. RDiff\ (ev\text{-}acc\ 0 + i)\ (ev\text{-}vd + i))\ [0 \dots W]$
 $@\ map\ (\lambda l. RBool\ (ev\text{-}kb\ l))\ [0 \dots dd]$
 $@\ (if\ k = KNonMembership\ then\ [RTag\ ev\text{-}tag]\ else\ [])$

Update: both paths (old leaf to root, new leaf to root2) over the SAME siblings and key bits; $value\text{-}digest$ binds the NEW leaf (S-6): $acc2\text{-}0 = vd$.

definition $descs\text{-}up :: res\text{-}desc\ list$ **where**

$descs\text{-}up =$
 $map\ (\lambda i. RDiff\ (ev2\text{-}hl + i)\ (ev2\text{-}acc\ 0 + i))\ [0 \dots W]$
 $@\ map\ (\lambda i. RDiff\ (ev2\text{-}hl2 + i)\ (ev2\text{-}acc2\ 0 + i))\ [0 \dots W]$
 $@\ concat\ (map\ (\lambda l. map\ (\lambda i. RDiff\ (ev2\text{-}hn\ l + i)\ (ev2\text{-}acc\ (Suc\ l) + i))\ [0 \dots W])\ [0 \dots dd])$
 $@\ concat\ (map\ (\lambda l. map\ (\lambda i. RDiff\ (ev2\text{-}hn2\ l + i)\ (ev2\text{-}acc2\ (Suc\ l) + i))\ [0 \dots W])\ [0 \dots dd])$
 $@\ map\ (\lambda i. RDiff\ (ev2\text{-}acc\ dd + i)\ (ev2\text{-}root + i))\ [0 \dots W]$
 $@\ map\ (\lambda i. RDiff\ (ev2\text{-}acc2\ dd + i)\ (ev2\text{-}root2 + i))\ [0 \dots W]$
 $@\ map\ (\lambda i. RDiff\ (ev2\text{-}acc2\ 0 + i)\ (ev2\text{-}vd + i))\ [0 \dots W]$
 $@\ map\ (\lambda l. RBool\ (ev2\text{-}kb\ l))\ [0 \dots dd]$

fun $descs :: op\text{-}kind \Rightarrow res\text{-}desc\ list$ **where**

$descs\ KUpdate = descs\text{-}up$

| $descs\ k = descs\text{-}ro\ k$

17.4 Constraint conjunction (the A-i target semantics)

definition $cons\text{-}ro :: op\text{-}kind \Rightarrow 'f\ list \Rightarrow bool$ **where**

$cons\text{-}ro\ k\ xs \longleftrightarrow$
 $hl\ (seg\ ro\text{-}lp\ L\ xs) = seg\ (ro\text{-}acc\ 0)\ W\ xs$
 $\wedge\ (\forall l < dd. hn\ (ro\text{-}mxl\ xs\ l)\ (ro\text{-}mxr\ xs\ l) = seg\ (ro\text{-}acc\ (Suc\ l))\ W\ xs)$
 $\wedge\ seg\ (ro\text{-}acc\ dd)\ W\ xs = seg\ ro\text{-}root\ W\ xs$
 $\wedge\ seg\ (ro\text{-}acc\ 0)\ W\ xs = seg\ ro\text{-}vd\ W\ xs$
 $\wedge\ (\forall l < dd. xs\ !\ ro\text{-}kb\ l * (xs\ !\ ro\text{-}kb\ l - 1) = 0)$
 $\wedge\ (k = KNonMembership \longrightarrow xs\ !\ ro\text{-}lp * (xs\ !\ ro\text{-}lp - 2) = 0)$

definition $cons\text{-}up :: 'f\ list \Rightarrow bool$ **where**

$cons\text{-}up\ xs \longleftrightarrow$
 $hl\ (seg\ up\text{-}lp\ L\ xs) = seg\ (up\text{-}acc\ 0)\ W\ xs$
 $\wedge\ hl\ (seg\ up\text{-}lp2\ L\ xs) = seg\ (up\text{-}acc2\ 0)\ W\ xs$
 $\wedge\ (\forall l < dd. hn\ (up\text{-}mxl\ xs\ up\text{-}acc\ l)\ (up\text{-}mxr\ xs\ up\text{-}acc\ l) = seg\ (up\text{-}acc\ (Suc\ l))\ W\ xs)$

$$\begin{aligned}
& \wedge (\forall l < dd. \text{hn } (up\text{-}mxl \text{ } xs \text{ } up\text{-}acc2 \text{ } l) (up\text{-}mxr \text{ } xs \text{ } up\text{-}acc2 \text{ } l) = \text{seg } (up\text{-}acc2 \text{ } (Suc \text{ } l)) \text{ } W \text{ } xs) \\
& \wedge \text{seg } (up\text{-}acc \text{ } dd) \text{ } W \text{ } xs = \text{seg } up\text{-}root \text{ } W \text{ } xs \\
& \wedge \text{seg } (up\text{-}acc2 \text{ } dd) \text{ } W \text{ } xs = \text{seg } up\text{-}root2 \text{ } W \text{ } xs \\
& \wedge \text{seg } (up\text{-}acc2 \text{ } 0) \text{ } W \text{ } xs = \text{seg } up\text{-}vd \text{ } W \text{ } xs \\
& \wedge (\forall l < dd. xs ! up\text{-}kb \text{ } l * (xs ! up\text{-}kb \text{ } l - 1) = 0)
\end{aligned}$$

17.5 Witness encoding (Rust: build_input_vector)

The honest accumulator chain along one path: $acc\text{-}0$ is the leaf digest, $acc\text{-}\{l+1\}$ compresses $acc\text{-}l$ with sibling l in key-bit order - exactly *MerklePath::compute-root*'s recursion retaining intermediates (Rust: *acc-chain*, *witness.rs*).

```

fun acc-list :: 'd  $\Rightarrow$  nat  $\Rightarrow$  'd list  $\Rightarrow$  'd list where
  acc-list a key [] = [a]
| acc-list a key (s # ss) =
  a # acc-list (if key mod 2 = 0 then h-node a s else h-node s a) (key div 2) ss

```

```

lemma length-acc-list [simp]: length (acc-list a key ss) = Suc (length ss)
by (induction ss arbitrary: a key) auto

```

```

lemma acc-list-0 [simp]: acc-list a key ss ! 0 = a
by (cases ss) auto

```

```

lemma acc-list-ne [simp]: acc-list a key ss  $\neq$  []
by (cases ss) auto

```

```

lemma acc-list-last: last (acc-list a key ss) = path-root a key ss
by (induction a key ss rule: acc-list.induct) auto

```

```

lemma acc-list-step:
  assumes l < length ss
  shows acc-list a key ss ! Suc l =
    (if (key div 2 ^ l) mod 2 = 0
     then h-node (acc-list a key ss ! l) (ss ! l)
     else h-node (ss ! l) (acc-list a key ss ! l))
  using assms

```

```

proof (induction ss arbitrary: a key l)
  case Nil then show ?case by simp
next
  case (Cons s ss)
  show ?case
  proof (cases l)
    case 0 then show ?thesis by simp
  next
    case (Suc l')
    with Cons.prem1 have l' < length ss by simp
    from Cons.IH[OF this] Suc show ?thesis
    by (simp add: div-mult2-eq [symmetric] mult.commute)
  qed
qed

```

Read-only input encoding: leaf pre-image, honest accumulator chain, siblings, key bits, public root and value digest - the segment order of *InputLayout* - zero-padded to the power-of-two

input width.

definition $enc-ro-core :: 'v\ leaf-state \Rightarrow nat \Rightarrow 'd\ list \Rightarrow 'd \Rightarrow 'd \Rightarrow 'f\ list$ **where**

```

  enc-ro-core leaf key sibs root vd =
    leaf-repr leaf
  @ concat (map ( $\lambda l.$  digest-repr (acc-list (h-leaf leaf) key sibs ! l)) [0 ..< Suc dd])
  @ concat (map ( $\lambda l.$  digest-repr (sibs ! l)) [0 ..< dd])
  @ map (kbit key) [0 ..< dd]
  @ digest-repr root
  @ digest-repr vd

```

definition $enc-ro :: op-kind \Rightarrow 'v\ leaf-state \Rightarrow nat \Rightarrow 'd\ list \Rightarrow 'd \Rightarrow 'd \Rightarrow 'f\ list$ **where**

```

  enc-ro k leaf key sibs root vd = pad-to (ivlen k) (enc-ro-core leaf key sibs root vd)

```

Update input encoding. The primary path carries the operation's OLD leaf, the second path the NEW leaf, over the same siblings and key bits (Rust: *build-input-vector* ignores the witness leaf for Update and uses the operation's old leaf - mirrored literally here; the agreement of the witness leaf with the old leaf is a well-formedness premise of Theorem A, matching *smt-valid's* $leaf = old$ clause).

definition $enc-up-core :: 'v\ leaf-state \Rightarrow 'v\ leaf-state \Rightarrow nat \Rightarrow 'd\ list \Rightarrow 'd \Rightarrow 'd \Rightarrow 'd \Rightarrow 'f\ list$ **where**

```

  enc-up-core old new key sibs root root' vd =
    leaf-repr old
  @ leaf-repr new
  @ concat (map ( $\lambda l.$  digest-repr (acc-list (h-leaf old) key sibs ! l)) [0 ..< Suc dd])
  @ concat (map ( $\lambda l.$  digest-repr (acc-list (h-leaf new) key sibs ! l)) [0 ..< Suc dd])
  @ concat (map ( $\lambda l.$  digest-repr (sibs ! l)) [0 ..< dd])
  @ map (kbit key) [0 ..< dd]
  @ digest-repr root
  @ digest-repr root'
  @ digest-repr vd

```

definition $enc-up :: 'v\ leaf-state \Rightarrow 'v\ leaf-state \Rightarrow nat \Rightarrow 'd\ list \Rightarrow 'd \Rightarrow 'd \Rightarrow 'd \Rightarrow 'f\ list$ **where**

```

  enc-up old new key sibs root root' vd =
    pad-to (ivlen KUpdate) (enc-up-core old new key sibs root root' vd)

```

The witness encoder of Theorem A (Rust: *build_input_vector*).

definition $encode-witness :: 'v\ smt-op \Rightarrow 'd \Rightarrow 'd \Rightarrow 'd \Rightarrow ('v\ leaf-state \times 'd\ list) \Rightarrow 'f\ list$ **where**

```

  encode-witness op root root' vd w = (case op of
    | Membership k v  $\Rightarrow$  enc-ro KMembership (fst w) k (snd w) root vd
    | NonMembership k  $\Rightarrow$  enc-ro KNonMembership (fst w) k (snd w) root vd
    | Update k old new  $\Rightarrow$  enc-up old new k (snd w) root root' vd)

```

end

18 The full compiler model (evaluation stack as abstract parameter)

HASH SUB-CIRCUIT BOUNDARY: *stack k* is the layer list computing the hash values and mux images. Its only specification is generator correctness (*stack-spec**): on an input vector of the declared width, evaluating the stack yields exactly *ev-ro/ev-up*. The gate-level

Poseidon2 arithmetization inside the stack is NOT proven here - it is pinned by the code-level cross-check tests (compile.rs: the *poseidon2-*-matches-native* family) and remains a blackbox behind this interface. This is a specification assumption, not a strength assumption: no collision resistance, no injectivity of the hashes.

```

locale compiler-model = compiler-model-base params h-leaf h-node W L digest-repr leaf-repr hl hn
  for params :: smt-params
  and h-leaf :: 'v leaf-state  $\Rightarrow$  'd
  and h-node :: 'd  $\Rightarrow$  'd  $\Rightarrow$  'd
  and W L :: nat
  and digest-repr :: 'd  $\Rightarrow$  'f::idom list
  and leaf-repr :: 'v leaf-state  $\Rightarrow$  'f list
  and hl hn +
  fixes stack :: op-kind  $\Rightarrow$  'f layer list
  assumes stack-spec-ro:
     $\bigwedge k$  xs. k  $\neq$  KUpdate  $\impl$  length xs = ivlen k  $\impl$ 
      foldr layer-eval (stack k) xs = ev-ro xs
  and stack-spec-up:
     $\bigwedge$  xs. length xs = ivlen KUpdate  $\impl$ 
      foldr layer-eval (stack KUpdate) xs = ev-up xs
begin

```

The model compiler: one residual output layer over the evaluation stack. Rust: *compile* (compile.rs) - the constraint set is identical; the layer decomposition differs (a refinement-map freedom).

```

definition compile :: op-kind  $\Rightarrow$  'f layered-circuit where
  compile k =
    ( $\mid$  circ-layers = mk-res-layer (descs k)  $\#$  stack k,
     input-width-bits = iwb k  $\mid$ )

```

```

lemma compile-output:
  assumes length xs = ivlen k
  shows circuit-output (compile k) xs
    = layer-eval (mk-res-layer (descs k)) (foldr layer-eval (stack k) xs)
  by (simp add: circuit-output-foldr compile-def)

end

```

end

```

theory Compiler-Correctness
  imports Compiler-Model
begin

```

```

context compiler-model-base
begin

```

19 Slot lemmas for the evaluation-stack specification vector

```

lemma ro-seg-bounds:
  ro-lp + L  $\leq$  ro-width

```

$l \leq dd \implies \text{ro-acc } l + W \leq \text{ro-width}$
 $l < dd \implies \text{ro-sib } l + W \leq \text{ro-width}$
 $l < dd \implies \text{ro-kb } l < \text{ro-width}$
 $\text{ro-root} + W \leq \text{ro-width}$
 $\text{ro-vd} + W \leq \text{ro-width}$

proof –

have $d1: W * (2 * dd + 1) = 2 * (W * dd) + W$
by (*simp add: algebra-simps*)
have $d2: W * (dd + 1) = W * dd + W$
by (*simp add: algebra-simps*)
show $\text{ro-lp} + L \leq \text{ro-width}$
unfolding *ro-lp-def ro-width-def ro-vd-def ro-root-def* **by** *linarith*
show $\text{ro-acc } l + W \leq \text{ro-width}$ **if** $l: l \leq dd$
proof –
have $m: W * l \leq W * dd$ **using** l **by** (*rule mult-le-mono2*)
show *?thesis*
using m $d1$
unfolding *ro-acc-def ro-width-def ro-vd-def ro-root-def*
by *linarith*

qed

show $\text{ro-sib } l + W \leq \text{ro-width}$ **if** $l: l < dd$

proof –

have $W * l \leq W * dd$ **using** *less-imp-le[OF l]* **by** (*rule mult-le-mono2*)
then have $W * (dd + 1) + W * l \leq W * (dd + 1) + W * dd$
by (*rule add-left-mono*)
also have $W * (dd + 1) + W * dd = W * (2 * dd + 1)$
by (*simp add: algebra-simps*)
finally have $m: W * (dd + 1) + W * l \leq W * (2 * dd + 1)$.
show *?thesis*
using m
unfolding *ro-sib-def ro-width-def ro-vd-def ro-root-def*
by *linarith*

qed

show $\text{ro-kb } l < \text{ro-width}$ **if** $l: l < dd$

using l

unfolding *ro-kb-def ro-width-def ro-vd-def ro-root-def*

by *linarith*

show $\text{ro-root} + W \leq \text{ro-width}$

unfolding *ro-width-def ro-vd-def* **by** *linarith*

show $\text{ro-vd} + W \leq \text{ro-width}$

unfolding *ro-width-def* **by** *linarith*

qed

lemma *length-seg-ro [simp]*:

assumes $\text{ro-width} \leq \text{length } xs$
shows $\text{length } (\text{seg ro-lp } L \text{ } xs) = L$
and $l \leq dd \implies \text{length } (\text{seg } (\text{ro-acc } l) \text{ } W \text{ } xs) = W$
and $l < dd \implies \text{length } (\text{seg } (\text{ro-sib } l) \text{ } W \text{ } xs) = W$
and $\text{length } (\text{seg ro-root } W \text{ } xs) = W$
and $\text{length } (\text{seg ro-vd } W \text{ } xs) = W$

proof –

show $\text{length } (\text{seg ro-lp } L \text{ } xs) = L$
by (*rule length-seg*) (*rule le-trans[OF ro-seg-bounds(1) assms]*)

```

show length (seg (ro-acc l) W xs) = W if l: l ≤ dd
  by (rule length-seg) (rule le-trans[OF ro-seg-bounds(2)] [OF l] assms)
show length (seg (ro-sib l) W xs) = W if l: l < dd
  by (rule length-seg) (rule le-trans[OF ro-seg-bounds(3)] [OF l] assms)
show length (seg ro-root W xs) = W
  by (rule length-seg) (rule le-trans[OF ro-seg-bounds(5)] assms)
show length (seg ro-vd W xs) = W
  by (rule length-seg) (rule le-trans[OF ro-seg-bounds(6)] assms)
qed

```

Uniform block lengths inside *ev-ro*.

```

lemma ev-ro-block-lens:
  assumes ro-width ≤ length xs
  shows length (concat (map (λl. hn (ro-mxl xs l) (ro-mxr xs l)) [0 ..< dd])) = dd * W
    and length (concat (map (λl. seg (ro-acc l) W xs) [0 ..< Suc dd])) = Suc dd * W
  using assms
  by (auto intro!: concat-map-uniform-length simp: length-seg ro-seg-bounds
    intro: order-trans)

```

The core specification vector has exactly the slot-layout length, and the padded vector its power-of-two width.

```

lemma length-ev-ro-core:
  assumes len: ro-width ≤ length xs
  shows length (ev-ro-core xs) = ev-ro-len
  using ev-ro-block-lens[OF len] length-seg-ro[OF len]
  by (simp add: ev-ro-core-def ev-ro-len-def ev-tag-def ev-vd-def ev-root-def
    algebra-simps)

```

```

lemma ev-ro-core-le-pad: length (ev-ro-core xs) ≤ 2 ^ ev-ro-len ⇒
  length (ev-ro xs) = 2 ^ ev-ro-len
  by (simp add: ev-ro-def length-pad-to)

```

```

lemma length-ev-ro:
  assumes len: ro-width ≤ length xs
  shows length (ev-ro xs) = 2 ^ ev-ro-len
  by (rule ev-ro-core-le-pad)
    (simp add: length-ev-ro-core[OF len] less-imp-le[OF less-exp])

```

```

lemma block-index-lt: l < n ⇒ i < W ⇒ W * l + i < n * W

```

```

proof –
  assume l: l < n and i: i < W
  have W * l + i < W * Suc l using i by simp
  also have ... ≤ W * n using Suc-leI[OF l] by (rule mult-le-mono2)
  finally show ?thesis by (simp add: mult.commute)
qed

```

```

lemma ev-ro-nth-hl:
  assumes ro-width ≤ length xs and i: i < W
  shows ev-ro xs ! (ev-hl + i) = hl (seg ro-lp L xs) ! i
  using i by (simp add: ev-ro-def pad-to-def ev-ro-core-def ev-hl-def nth-append)

```

```

lemma ev-ro-nth-hn:

```

assumes $len: ro\text{-}width \leq length\ xs$ **and** $l: l < dd$ **and** $i: i < W$
shows $ev\text{-}ro\ xs \ ! \ (ev\text{-}hn\ l + i) = hn\ (ro\text{-}mxl\ xs\ l)\ (ro\text{-}mxr\ xs\ l) \ ! \ i$
proof –
have $lt: W * l + i < dd * W$ **using** $block\text{-}index\text{-}lt\ l\ i$ **by** $simp$
have $ev\text{-}ro\ xs \ ! \ (ev\text{-}hn\ l + i)$
 $= concat\ (map\ (\lambda j. hn\ (ro\text{-}mxl\ xs\ j)\ (ro\text{-}mxr\ xs\ j))\ [0 ..< dd]) \ ! \ (W * l + i)$
using lt **by** $(simp\ add: ev\text{-}ro\text{-}def\ pad\text{-}to\text{-}def\ ev\text{-}ro\text{-}core\text{-}def\ ev\text{-}hn\text{-}def\ nth\text{-}append\ ev\text{-}ro\text{-}block\text{-}lens[OF\ len]\ add.assoc)$
also have $\dots = hn\ (ro\text{-}mxl\ xs\ l)\ (ro\text{-}mxr\ xs\ l) \ ! \ i$
using $l\ i$ **by** $(subst\ mult.commute[of\ W\ l])\ (intro\ concat\text{-}map\text{-}uniform\text{-}nth;\ simp)$
finally show $?thesis$.
qed

lemma $ev\text{-}ro\text{-}nth\text{-}acc$:
assumes $len: ro\text{-}width \leq length\ xs$ **and** $l: l \leq dd$ **and** $i: i < W$
shows $ev\text{-}ro\ xs \ ! \ (ev\text{-}acc\ l + i) = seg\ (ro\text{-}acc\ l)\ W\ xs \ ! \ i$
proof –
have $lt: W * l + i < Suc\ dd * W$
by $(rule\ block\text{-}index\text{-}lt[OF\ le\text{-}imp\text{-}less\text{-}Suc[OF\ l]\ i])$
have $idx: ev\text{-}acc\ l + i$
 $= length\ (hl\ (seg\ ro\text{-}lp\ L\ xs))$
 $+ (length\ (concat\ (map\ (\lambda l. hn\ (ro\text{-}mxl\ xs\ l)\ (ro\text{-}mxr\ xs\ l))\ [0 ..< dd])))$
 $+ (W * l + i)$
using $ev\text{-}ro\text{-}block\text{-}lens[OF\ len]$ **by** $(simp\ add: ev\text{-}acc\text{-}def\ algebra\text{-}simps)$
have $lt2: W * l + i < length\ (concat\ (map\ (\lambda l. seg\ (ro\text{-}acc\ l)\ W\ xs)\ [0 ..< Suc\ dd]))$
using $ev\text{-}ro\text{-}block\text{-}lens(2)[OF\ len]\ lt$ **by** $simp$
have $ev\text{-}ro\ xs \ ! \ (ev\text{-}acc\ l + i)$
 $= concat\ (map\ (\lambda j. seg\ (ro\text{-}acc\ j)\ W\ xs)\ [0 ..< Suc\ dd]) \ ! \ (W * l + i)$
unfolding $ev\text{-}ro\text{-}def\ pad\text{-}to\text{-}def\ ev\text{-}ro\text{-}core\text{-}def\ idx\ append\text{-}assoc$
by $(simp\ only: nth\text{-}append\text{-}length\text{-}plus)\ (simp\ only: nth\text{-}append\ if\text{-}P[OF\ lt2])$
also have $\dots = seg\ (ro\text{-}acc\ l)\ W\ xs \ ! \ i$
using $l\ i\ len$
by $(subst\ mult.commute[of\ W\ l])\ (intro\ concat\text{-}map\text{-}uniform\text{-}nth;\ simp\ add: le\text{-}imp\text{-}less\text{-}Suc)$
finally show $?thesis$.
qed

lemma $ev\text{-}ro\text{-}nth\text{-}kb$:
assumes $len: ro\text{-}width \leq length\ xs$ **and** $l: l < dd$
shows $ev\text{-}ro\ xs \ ! \ (ev\text{-}kb\ l) = xs \ ! \ ro\text{-}kb\ l$
proof –
have $idx: ev\text{-}kb\ l$
 $= length\ (hl\ (seg\ ro\text{-}lp\ L\ xs))$
 $+ (length\ (concat\ (map\ (\lambda l. hn\ (ro\text{-}mxl\ xs\ l)\ (ro\text{-}mxr\ xs\ l))\ [0 ..< dd])))$
 $+ (length\ (concat\ (map\ (\lambda l. seg\ (ro\text{-}acc\ l)\ W\ xs)\ [0 ..< Suc\ dd])))$
 $+ l)$
using $ev\text{-}ro\text{-}block\text{-}lens[OF\ len]$ **by** $(simp\ add: ev\text{-}kb\text{-}def\ algebra\text{-}simps)$
show $?thesis$
unfolding $ev\text{-}ro\text{-}def\ pad\text{-}to\text{-}def\ ev\text{-}ro\text{-}core\text{-}def\ idx\ append\text{-}assoc$
by $(simp\ only: nth\text{-}append\text{-}length\text{-}plus)\ (simp\ add: nth\text{-}append\ l\ nth\text{-}upt)$
qed

lemma $ev\text{-}ro\text{-}nth\text{-}root$:

assumes $len: ro\text{-}width \leq length\ xs$ **and** $i: i < W$
shows $ev\text{-}ro\ xs \ ! \ (ev\text{-}root + i) = seg\ ro\text{-}root\ W\ xs \ ! \ i$
proof –
have $idx: ev\text{-}root + i$
 $= length\ (hl\ (seg\ ro\text{-}lp\ L\ xs))$
 $+ (length\ (concat\ (map\ (\lambda l. hn\ (ro\text{-}mxl\ xs\ l)\ (ro\text{-}mxr\ xs\ l))\ [0 \ ..< \ dd])))$
 $+ (length\ (concat\ (map\ (\lambda l. seg\ (ro\text{-}acc\ l)\ W\ xs)\ [0 \ ..< \ Suc\ dd])))$
 $+ (length\ (map\ (\lambda l. xs \ ! \ ro\text{-}kb\ l)\ [0 \ ..< \ dd])$
 $+ i)))$
using $ev\text{-}ro\text{-}block\text{-}lens[OF\ len]$ **by** $(simp\ add: ev\text{-}root\text{-}def\ algebra\text{-}simps)$
show $?thesis$
unfolding $ev\text{-}ro\text{-}def\ pad\text{-}to\text{-}def\ ev\text{-}ro\text{-}core\text{-}def\ idx\ append\text{-}assoc$
by $(simp\ only: nth\text{-}append\text{-}length\text{-}plus)$
 $(simp\ add: nth\text{-}append\ length\text{-}seg\text{-}ro(4)[OF\ len]\ i)$
qed

lemma $ev\text{-}ro\text{-}nth\text{-}vd$:
assumes $len: ro\text{-}width \leq length\ xs$ **and** $i: i < W$
shows $ev\text{-}ro\ xs \ ! \ (ev\text{-}vd + i) = seg\ ro\text{-}vd\ W\ xs \ ! \ i$
proof –
have $idx: ev\text{-}vd + i$
 $= length\ (hl\ (seg\ ro\text{-}lp\ L\ xs))$
 $+ (length\ (concat\ (map\ (\lambda l. hn\ (ro\text{-}mxl\ xs\ l)\ (ro\text{-}mxr\ xs\ l))\ [0 \ ..< \ dd])))$
 $+ (length\ (concat\ (map\ (\lambda l. seg\ (ro\text{-}acc\ l)\ W\ xs)\ [0 \ ..< \ Suc\ dd])))$
 $+ (length\ (map\ (\lambda l. xs \ ! \ ro\text{-}kb\ l)\ [0 \ ..< \ dd])$
 $+ (length\ (seg\ ro\text{-}root\ W\ xs)$
 $+ i))))$
using $ev\text{-}ro\text{-}block\text{-}lens[OF\ len]\ length\text{-}seg\text{-}ro(4)[OF\ len]$
by $(simp\ add: ev\text{-}vd\text{-}def\ ev\text{-}root\text{-}def\ algebra\text{-}simps)$
show $?thesis$
unfolding $ev\text{-}ro\text{-}def\ pad\text{-}to\text{-}def\ ev\text{-}ro\text{-}core\text{-}def\ idx\ append\text{-}assoc$
by $(simp\ only: nth\text{-}append\text{-}length\text{-}plus)$
 $(simp\ add: nth\text{-}append\ length\text{-}seg\text{-}ro(5)[OF\ len]\ i)$
qed

lemma $ev\text{-}ro\text{-}nth\text{-}tag$:
assumes $len: ro\text{-}width \leq length\ xs$
shows $ev\text{-}ro\ xs \ ! \ ev\text{-}tag = xs \ ! \ ro\text{-}lp$
proof –
have $idx: ev\text{-}tag$
 $= length\ (hl\ (seg\ ro\text{-}lp\ L\ xs))$
 $+ (length\ (concat\ (map\ (\lambda l. hn\ (ro\text{-}mxl\ xs\ l)\ (ro\text{-}mxr\ xs\ l))\ [0 \ ..< \ dd])))$
 $+ (length\ (concat\ (map\ (\lambda l. seg\ (ro\text{-}acc\ l)\ W\ xs)\ [0 \ ..< \ Suc\ dd])))$
 $+ (length\ (map\ (\lambda l. xs \ ! \ ro\text{-}kb\ l)\ [0 \ ..< \ dd])$
 $+ (length\ (seg\ ro\text{-}root\ W\ xs)$
 $+ (length\ (seg\ ro\text{-}vd\ W\ xs)$
 $+ 0))))$
using $ev\text{-}ro\text{-}block\text{-}lens[OF\ len]\ length\text{-}seg\text{-}ro(4)[OF\ len]\ length\text{-}seg\text{-}ro(5)[OF\ len]$
by $(simp\ add: ev\text{-}tag\text{-}def\ ev\text{-}vd\text{-}def\ ev\text{-}root\text{-}def\ algebra\text{-}simps)$
show $?thesis$
unfolding $ev\text{-}ro\text{-}def\ pad\text{-}to\text{-}def\ ev\text{-}ro\text{-}core\text{-}def\ idx\ append\text{-}assoc$
by $(simp\ only: nth\text{-}append\text{-}length\text{-}plus)\ simp$
qed

19.1 Update slots

lemma *up-seg-bounds*:

$up\text{-}lp + L \leq up\text{-}width$
 $up\text{-}lp2 + L \leq up\text{-}width$
 $l \leq dd \implies up\text{-}acc\ l + W \leq up\text{-}width$
 $l \leq dd \implies up\text{-}acc2\ l + W \leq up\text{-}width$
 $l < dd \implies up\text{-}sib\ l + W \leq up\text{-}width$
 $l < dd \implies up\text{-}kb\ l < up\text{-}width$
 $up\text{-}root + W \leq up\text{-}width$
 $up\text{-}root2 + W \leq up\text{-}width$
 $up\text{-}vd + W \leq up\text{-}width$

proof –

have $d2: W * (dd + 1) = W * dd + W$
by (*simp add: algebra-simps*)
have $d3: W * (3 * dd + 2) = 3 * (W * dd) + 2 * W$
by (*simp add: algebra-simps*)
have $d4: 2 * W * (dd + 1) = 2 * (W * dd) + 2 * W$
by (*simp add: algebra-simps*)
show $up\text{-}lp + L \leq up\text{-}width$
unfolding *up-lp-def up-width-def up-vd-def up-root2-def up-root-def* **by** *linarith*
show $up\text{-}lp2 + L \leq up\text{-}width$
unfolding *up-lp2-def up-width-def up-vd-def up-root2-def up-root-def* **by** *linarith*
show $up\text{-}acc\ l + W \leq up\text{-}width$ **if** $l: l \leq dd$
proof –
have $m: W * l \leq W * dd$ **using** l **by** (*rule mult-le-mono2*)
show *?thesis*
using $m\ d3$
unfolding *up-acc-def up-width-def up-vd-def up-root2-def up-root-def*
by *linarith*

qed

show $up\text{-}acc2\ l + W \leq up\text{-}width$ **if** $l: l \leq dd$

proof –

have $W * l \leq W * dd$ **using** l **by** (*rule mult-le-mono2*)
then have $W * (dd + 1) + W * l \leq W * (dd + 1) + W * dd$
by (*rule add-left-mono*)
also have $W * (dd + 1) + W * dd = W * (2 * dd + 1)$
by (*simp add: algebra-simps*)
also have $W * (2 * dd + 1) \leq W * (3 * dd + 2)$
by (*rule mult-le-mono2*) *linarith*
finally have $m: W * (dd + 1) + W * l \leq W * (3 * dd + 2)$.
show *?thesis*
using m
unfolding *up-acc2-def up-width-def up-vd-def up-root2-def up-root-def*
by *linarith*

qed

show $up\text{-}sib\ l + W \leq up\text{-}width$ **if** $l: l < dd$

proof –

have $m: W * l \leq W * dd$ **using** *less-imp-le[OF l]* **by** (*rule mult-le-mono2*)
show *?thesis*
using $m\ d3\ d4$
unfolding *up-sib-def up-width-def up-vd-def up-root2-def up-root-def*
by *linarith*

```

qed
show up-kb l < up-width if l: l < dd
  using l
  unfolding up-kb-def up-width-def up-vd-def up-root2-def up-root-def
  by linarith
show up-root + W ≤ up-width
  unfolding up-width-def up-vd-def up-root2-def by linarith
show up-root2 + W ≤ up-width
  unfolding up-width-def up-vd-def by linarith
show up-vd + W ≤ up-width
  unfolding up-width-def by linarith
qed

```

```

lemma length-seg-up [simp]:
  assumes up-width ≤ length xs
  shows length (seg up-lp L xs) = L
    and length (seg up-lp2 L xs) = L
    and l ≤ dd ⇒ length (seg (up-acc l) W xs) = W
    and l ≤ dd ⇒ length (seg (up-acc2 l) W xs) = W
    and l < dd ⇒ length (seg (up-sib l) W xs) = W
    and length (seg up-root W xs) = W
    and length (seg up-root2 W xs) = W
    and length (seg up-vd W xs) = W
proof -
  show length (seg up-lp L xs) = L
    by (rule length-seg) (rule le-trans[OF up-seg-bounds(1) assms])
  show length (seg up-lp2 L xs) = L
    by (rule length-seg) (rule le-trans[OF up-seg-bounds(2) assms])
  show length (seg (up-acc l) W xs) = W if l: l ≤ dd
    by (rule length-seg) (rule le-trans[OF up-seg-bounds(3)[OF l] assms])
  show length (seg (up-acc2 l) W xs) = W if l: l ≤ dd
    by (rule length-seg) (rule le-trans[OF up-seg-bounds(4)[OF l] assms])
  show length (seg (up-sib l) W xs) = W if l: l < dd
    by (rule length-seg) (rule le-trans[OF up-seg-bounds(5)[OF l] assms])
  show length (seg up-root W xs) = W
    by (rule length-seg) (rule le-trans[OF up-seg-bounds(7) assms])
  show length (seg up-root2 W xs) = W
    by (rule length-seg) (rule le-trans[OF up-seg-bounds(8) assms])
  show length (seg up-vd W xs) = W
    by (rule length-seg) (rule le-trans[OF up-seg-bounds(9) assms])
qed

```

```

lemma ev-up-block-lens:
  assumes up-width ≤ length xs
  shows length (concat (map (λl. hn (up-mxl xs up-acc l) (up-mxr xs up-acc l)) [0 ..< dd])) = dd *
W
  and length (concat (map (λl. hn (up-mxl xs up-acc2 l) (up-mxr xs up-acc2 l)) [0 ..< dd])) = dd *
W
  and length (concat (map (λl. seg (up-acc l) W xs) [0 ..< Suc dd])) = Suc dd * W
  and length (concat (map (λl. seg (up-acc2 l) W xs) [0 ..< Suc dd])) = Suc dd * W
  using assms
  by (auto intro!: concat-map-uniform-length simp: length-seg up-seg-bounds
    intro: order-trans)

```

lemma *length-ev-up-core*:
assumes *len*: $\text{up-width} \leq \text{length } xs$
shows $\text{length } (\text{ev-up-core } xs) = \text{ev-up-len}$
using *ev-up-block-lens*[*OF len*] *length-seg-up*[*OF len*]
by (*simp add*: *ev-up-core-def ev-up-len-def ev2-vd-def ev2-root2-def ev2-root-def*
algebra-simps)

lemma *length-ev-up*:
assumes *len*: $\text{up-width} \leq \text{length } xs$
shows $\text{length } (\text{ev-up } xs) = 2^{\wedge} \text{ev-up-len}$
by (*simp add*: *ev-up-def length-pad-to length-ev-up-core*[*OF len*]
less-imp-le[*OF less-exp*])

lemma *ev-up-nth-hl*:
assumes *len*: $\text{up-width} \leq \text{length } xs$ **and** *i*: $i < W$
shows $\text{ev-up } xs ! (\text{ev2-hl} + i) = \text{hl } (\text{seg up-lp } L \text{ } xs) ! i$
and $\text{ev-up } xs ! (\text{ev2-hl2} + i) = \text{hl } (\text{seg up-lp2 } L \text{ } xs) ! i$
using *i* **by** (*simp-all add*: *ev-up-def pad-to-def ev-up-core-def ev2-hl-def*
ev2-hl2-def nth-append)

lemma *ev-up-nth-hn*:
assumes *len*: $\text{up-width} \leq \text{length } xs$ **and** *l*: $l < dd$ **and** *i*: $i < W$
shows $\text{ev-up } xs ! (\text{ev2-hn } l + i) = \text{hn } (\text{up-mxl } xs \text{ up-acc } l) (\text{up-mxr } xs \text{ up-acc } l) ! i$
proof –
have *lt*: $W * l + i < dd * W$ **using** *block-index-lt l i* **by** *simp*
have $\text{ev-up } xs ! (\text{ev2-hn } l + i)$
 $= \text{concat } (\text{map } (\lambda j. \text{hn } (\text{up-mxl } xs \text{ up-acc } j) (\text{up-mxr } xs \text{ up-acc } j)) [0 ..< dd]) ! (W * l + i)$
using *lt* **by** (*simp add*: *ev-up-def pad-to-def ev-up-core-def ev2-hn-def nth-append*
ev-up-block-lens[*OF len*] *algebra-simps*)
also have $\dots = \text{hn } (\text{up-mxl } xs \text{ up-acc } l) (\text{up-mxr } xs \text{ up-acc } l) ! i$
using *l i* **by** (*subst mult.commute*[*of W l*]) (*intro concat-map-uniform-nth; simp*)
finally show *?thesis* .
qed

lemma *ev-up-nth-hn2*:
assumes *len*: $\text{up-width} \leq \text{length } xs$ **and** *l*: $l < dd$ **and** *i*: $i < W$
shows $\text{ev-up } xs ! (\text{ev2-hn2 } l + i) = \text{hn } (\text{up-mxl } xs \text{ up-acc2 } l) (\text{up-mxr } xs \text{ up-acc2 } l) ! i$
proof –
have *lt*: $W * l + i < dd * W$ **using** *block-index-lt l i* **by** *simp*
have $\text{ev-up } xs ! (\text{ev2-hn2 } l + i)$
 $= \text{concat } (\text{map } (\lambda j. \text{hn } (\text{up-mxl } xs \text{ up-acc2 } j) (\text{up-mxr } xs \text{ up-acc2 } j)) [0 ..< dd]) ! (W * l + i)$
using *lt* **by** (*simp add*: *ev-up-def pad-to-def ev-up-core-def ev2-hn2-def nth-append*
ev-up-block-lens[*OF len*] *algebra-simps*)
also have $\dots = \text{hn } (\text{up-mxl } xs \text{ up-acc2 } l) (\text{up-mxr } xs \text{ up-acc2 } l) ! i$
using *l i* **by** (*subst mult.commute*[*of W l*]) (*intro concat-map-uniform-nth; simp*)
finally show *?thesis* .
qed

lemma *ev-up-nth-acc*:
assumes *len*: $\text{up-width} \leq \text{length } xs$ **and** *l*: $l \leq dd$ **and** *i*: $i < W$
shows $\text{ev-up } xs ! (\text{ev2-acc } l + i) = \text{seg } (\text{up-acc } l) \text{ } W \text{ } xs ! i$
proof –

have $lt: W * l + i < \text{Suc } dd * W$
by (rule *block-index-lt*[*OF le-imp-less-Suc*[*OF l*] *i*])
have $idx: \text{ev2-acc } l + i$
 $= \text{length } (hl \text{ (seg up-lp } L \text{ xs)})$
 $+ (\text{length } (hl \text{ (seg up-lp2 } L \text{ xs)}))$
 $+ (\text{length } (\text{concat } (\text{map } (\lambda l. hn \text{ (up-mxl xs up-acc } l) \text{ (up-mxr xs up-acc } l)) [0 \dots dd])))$
 $+ (\text{length } (\text{concat } (\text{map } (\lambda l. hn \text{ (up-mxl xs up-acc2 } l) \text{ (up-mxr xs up-acc2 } l)) [0 \dots dd])))$
 $+ (W * l + i))$
using *ev-up-block-lens*[*OF len*] **by** (*simp add: ev2-acc-def algebra-simps*)
have $lt2: W * l + i < \text{length } (\text{concat } (\text{map } (\lambda l. \text{seg } (\text{up-acc } l) \text{ } W \text{ xs}) [0 \dots \text{Suc } dd])))$
using *ev-up-block-lens*(3)[*OF len*] *lt* **by** *simp*
have $\text{ev-up xs} ! (\text{ev2-acc } l + i)$
 $= \text{concat } (\text{map } (\lambda j. \text{seg } (\text{up-acc } j) \text{ } W \text{ xs}) [0 \dots \text{Suc } dd]) ! (W * l + i)$
unfolding *ev-up-def pad-to-def ev-up-core-def idx append-assoc*
by (*simp only: nth-append-length-plus*) (*simp only: nth-append if-P*[*OF lt2*])
also have $\dots = \text{seg } (\text{up-acc } l) \text{ } W \text{ xs} ! i$
using *l i len*
by (*subst mult.commute*[*of W l*])
(*intro concat-map-uniform-nth; simp add: le-imp-less-Suc*)
finally show ?thesis .
qed

lemma *ev-up-nth-acc2*:

assumes $\text{len}: \text{up-width} \leq \text{length xs}$ **and** $l: l \leq dd$ **and** $i: i < W$

shows $\text{ev-up xs} ! (\text{ev2-acc2 } l + i) = \text{seg } (\text{up-acc2 } l) \text{ } W \text{ xs} ! i$

proof –

have $lt: W * l + i < \text{Suc } dd * W$
by (rule *block-index-lt*[*OF le-imp-less-Suc*[*OF l*] *i*])
have $idx: \text{ev2-acc2 } l + i$
 $= \text{length } (hl \text{ (seg up-lp } L \text{ xs)})$
 $+ (\text{length } (hl \text{ (seg up-lp2 } L \text{ xs)}))$
 $+ (\text{length } (\text{concat } (\text{map } (\lambda l. hn \text{ (up-mxl xs up-acc } l) \text{ (up-mxr xs up-acc } l)) [0 \dots dd])))$
 $+ (\text{length } (\text{concat } (\text{map } (\lambda l. hn \text{ (up-mxl xs up-acc2 } l) \text{ (up-mxr xs up-acc2 } l)) [0 \dots dd])))$
 $+ (\text{length } (\text{concat } (\text{map } (\lambda l. \text{seg } (\text{up-acc } l) \text{ } W \text{ xs}) [0 \dots \text{Suc } dd])))$
 $+ (W * l + i))$
using *ev-up-block-lens*[*OF len*] **by** (*simp add: ev2-acc2-def algebra-simps*)
have $lt2: W * l + i < \text{length } (\text{concat } (\text{map } (\lambda l. \text{seg } (\text{up-acc2 } l) \text{ } W \text{ xs}) [0 \dots \text{Suc } dd])))$
using *ev-up-block-lens*(4)[*OF len*] *lt* **by** *simp*
have $\text{ev-up xs} ! (\text{ev2-acc2 } l + i)$
 $= \text{concat } (\text{map } (\lambda j. \text{seg } (\text{up-acc2 } j) \text{ } W \text{ xs}) [0 \dots \text{Suc } dd]) ! (W * l + i)$
unfolding *ev-up-def pad-to-def ev-up-core-def idx append-assoc*
by (*simp only: nth-append-length-plus*) (*simp only: nth-append if-P*[*OF lt2*])
also have $\dots = \text{seg } (\text{up-acc2 } l) \text{ } W \text{ xs} ! i$
using *l i len*
by (*subst mult.commute*[*of W l*])
(*intro concat-map-uniform-nth; simp add: le-imp-less-Suc*)
finally show ?thesis .
qed

lemma *ev-up-nth-kb*:

assumes $\text{len}: \text{up-width} \leq \text{length xs}$ **and** $l: l < dd$

shows $\text{ev-up xs} ! (\text{ev2-kb } l) = \text{xs} ! \text{up-kb } l$

proof –

```

have idx: ev2-kb l
  = length (hl (seg up-lp L xs))
    + (length (hl (seg up-lp2 L xs))
      + (length (concat (map (λl. hn (up-mxl xs up-acc l) (up-mxr xs up-acc l)) [0 ..< dd])))
      + (length (concat (map (λl. hn (up-mxl xs up-acc2 l) (up-mxr xs up-acc2 l)) [0 ..< dd])))
      + (length (concat (map (λl. seg (up-acc l) W xs) [0 ..< Suc dd])))
      + (length (concat (map (λl. seg (up-acc2 l) W xs) [0 ..< Suc dd])))
      + l))))
using ev-up-block-lens[OF len] by (simp add: ev2-kb-def algebra-simps)
show ?thesis
unfolding ev-up-def pad-to-def ev-up-core-def idx append-assoc
by (simp only: nth-append-length-plus) (simp add: nth-append l nth-upt)
qed

```

```

lemma ev-up-nth-root:
  assumes len: up-width ≤ length xs and i: i < W
  shows ev-up xs ! (ev2-root + i) = seg up-root W xs ! i
proof –
  have idx: ev2-root + i
    = length (hl (seg up-lp L xs))
      + (length (hl (seg up-lp2 L xs))
        + (length (concat (map (λl. hn (up-mxl xs up-acc l) (up-mxr xs up-acc l)) [0 ..< dd])))
        + (length (concat (map (λl. hn (up-mxl xs up-acc2 l) (up-mxr xs up-acc2 l)) [0 ..< dd])))
        + (length (concat (map (λl. seg (up-acc l) W xs) [0 ..< Suc dd])))
        + (length (concat (map (λl. seg (up-acc2 l) W xs) [0 ..< Suc dd])))
        + (length (map (λl. xs ! up-kb l) [0 ..< dd])
          + i))))))
  using ev-up-block-lens[OF len] by (simp add: ev2-root-def algebra-simps)
show ?thesis
unfolding ev-up-def pad-to-def ev-up-core-def idx append-assoc
by (simp only: nth-append-length-plus)
    (simp add: nth-append length-seg-up(6)[OF len] i)
qed

```

```

lemma ev-up-nth-root2:
  assumes len: up-width ≤ length xs and i: i < W
  shows ev-up xs ! (ev2-root2 + i) = seg up-root2 W xs ! i
proof –
  have idx: ev2-root2 + i
    = length (hl (seg up-lp L xs))
      + (length (hl (seg up-lp2 L xs))
        + (length (concat (map (λl. hn (up-mxl xs up-acc l) (up-mxr xs up-acc l)) [0 ..< dd])))
        + (length (concat (map (λl. hn (up-mxl xs up-acc2 l) (up-mxr xs up-acc2 l)) [0 ..< dd])))
        + (length (concat (map (λl. seg (up-acc l) W xs) [0 ..< Suc dd])))
        + (length (concat (map (λl. seg (up-acc2 l) W xs) [0 ..< Suc dd])))
        + (length (map (λl. xs ! up-kb l) [0 ..< dd])
          + (length (seg up-root W xs)
            + i))))))
  using ev-up-block-lens[OF len] length-seg-up(6)[OF len]
by (simp add: ev2-root2-def ev2-root-def algebra-simps)
show ?thesis
unfolding ev-up-def pad-to-def ev-up-core-def idx append-assoc
by (simp only: nth-append-length-plus)

```

(simp add: nth-append length-seg-up(7)[OF len] i)

qed

lemma *ev-up-nth-vd*:

assumes *len*: $\text{up-width} \leq \text{length } xs$ **and** *i*: $i < W$

shows $\text{ev-up } xs ! (\text{ev2-vd} + i) = \text{seg up-vd } W xs ! i$

proof –

have *idx*: $\text{ev2-vd} + i$

$= \text{length } (\text{hl } (\text{seg up-lp } L xs))$

$+ (\text{length } (\text{hl } (\text{seg up-lp2 } L xs)))$

$+ (\text{length } (\text{concat } (\text{map } (\lambda l. \text{hn } (\text{up-mxl } xs \text{ up-acc } l) (\text{up-mxr } xs \text{ up-acc } l)) [0 ..< dd])))$

$+ (\text{length } (\text{concat } (\text{map } (\lambda l. \text{hn } (\text{up-mxl } xs \text{ up-acc2 } l) (\text{up-mxr } xs \text{ up-acc2 } l)) [0 ..< dd])))$

$+ (\text{length } (\text{concat } (\text{map } (\lambda l. \text{seg } (\text{up-acc } l) W xs) [0 ..< \text{Suc } dd])))$

$+ (\text{length } (\text{concat } (\text{map } (\lambda l. \text{seg } (\text{up-acc2 } l) W xs) [0 ..< \text{Suc } dd])))$

$+ (\text{length } (\text{map } (\lambda l. xs ! \text{up-kb } l) [0 ..< dd]))$

$+ (\text{length } (\text{seg up-root } W xs))$

$+ (\text{length } (\text{seg up-root2 } W xs))$

$+ i))))))$

using *ev-up-block-lens*[OF len] *length-seg-up*(6)[OF len] *length-seg-up*(7)[OF len]

by (simp add: ev2-vd-def ev2-root2-def ev2-root-def algebra-simps)

show ?thesis

unfolding *ev-up-def* *pad-to-def* *ev-up-core-def* *idx* *append-assoc*

by (simp only: nth-append-length-plus)

(simp add: nth-append length-seg-up(8)[OF len] i)

qed

20 Stage A-i: acceptance is the constraint conjunction

Vanishing of all residuals of *descs-ro* IS *cons-ro*.

lemma *res-sem-descs-ro-iff*:

assumes *len*: $\text{ro-width} \leq \text{length } xs$

shows $(\forall rd \in \text{set } (\text{descs-ro } k). \text{res-sem } (\text{ev-ro } xs) rd = 0) \longleftrightarrow \text{cons-ro } k xs$

proof –

let ?ev = *ev-ro xs*

have *leaf*: $(\forall rd \in \text{set } (\text{map } (\lambda i. \text{RDiff } (\text{ev-hl} + i) (\text{ev-acc } 0 + i)) [0 ..< W])).$

$\text{res-sem } ?ev rd = 0)$

$\longleftrightarrow \text{hl } (\text{seg ro-lp } L xs) = \text{seg } (\text{ro-acc } 0) W xs$

using *ev-ro-nth-hl*[OF len] *ev-ro-nth-acc*[OF len, of 0]

by (auto simp: list-eq-iff-nth-eq length-seg-ro[OF len])

have *chain*: $(\forall rd \in \text{set } (\text{concat } (\text{map } (\lambda l. \text{map } (\lambda i. \text{RDiff } (\text{ev-hn } l + i) (\text{ev-acc } (\text{Suc } l) + i)) [0 ..< W]) [0 ..< dd]))).$

$\text{res-sem } ?ev rd = 0)$

$\longleftrightarrow (\forall l < dd. \text{hn } (\text{ro-mxl } xs l) (\text{ro-mxr } xs l) = \text{seg } (\text{ro-acc } (\text{Suc } l)) W xs)$

proof –

have $(\forall rd \in \text{set } (\text{concat } (\text{map } (\lambda l. \text{map } (\lambda i. \text{RDiff } (\text{ev-hn } l + i) (\text{ev-acc } (\text{Suc } l) + i)) [0 ..< W]) [0 ..< dd]))).$

$\text{res-sem } ?ev rd = 0)$

$\longleftrightarrow (\forall l < dd. \forall i < W. ?ev ! (\text{ev-hn } l + i) = ?ev ! (\text{ev-acc } (\text{Suc } l) + i))$

by auto

also have $\dots \longleftrightarrow (\forall l < dd. \text{hn } (\text{ro-mxl } xs l) (\text{ro-mxr } xs l) = \text{seg } (\text{ro-acc } (\text{Suc } l)) W xs)$

proof (intro all-cong1 imp-cong[OF refl])

```

fix  $l$  assume  $l < dd$ 
then show  $(\forall i < W. \text{?ev} ! (ev-hn\ l + i) = \text{?ev} ! (ev-acc\ (Suc\ l) + i))$ 
   $\longleftrightarrow hn\ (ro-maxl\ xs\ l)\ (ro-mxr\ xs\ l) = seg\ (ro-acc\ (Suc\ l))\ W\ xs$ 
  using  $ev-ro-nth-hn[OF\ len]\ ev-ro-nth-acc[OF\ len]$ 
  by  $(auto\ simp: list-eq-iff-nth-eq\ length-seg-ro[OF\ len]\ Suc-le-eq)$ 
qed
finally show  $?thesis$  .
qed
have  $root: (\forall rd \in set\ (map\ (\lambda i. RDiff\ (ev-acc\ dd + i)\ (ev-root + i))\ [0 ..< W]).$ 
   $res-sem\ \text{?ev}\ rd = 0)$ 
   $\longleftrightarrow seg\ (ro-acc\ dd)\ W\ xs = seg\ ro-root\ W\ xs$ 
  using  $ev-ro-nth-acc[OF\ len, of\ dd]\ ev-ro-nth-root[OF\ len]$ 
  by  $(auto\ simp: list-eq-iff-nth-eq\ length-seg-ro[OF\ len])$ 
have  $vd: (\forall rd \in set\ (map\ (\lambda i. RDiff\ (ev-acc\ 0 + i)\ (ev-vd + i))\ [0 ..< W]).$ 
   $res-sem\ \text{?ev}\ rd = 0)$ 
   $\longleftrightarrow seg\ (ro-acc\ 0)\ W\ xs = seg\ ro-vd\ W\ xs$ 
  using  $ev-ro-nth-acc[OF\ len, of\ 0]\ ev-ro-nth-vd[OF\ len]$ 
  by  $(auto\ simp: list-eq-iff-nth-eq\ length-seg-ro[OF\ len])$ 
have  $bool: (\forall rd \in set\ (map\ (\lambda l. RBool\ (ev-kb\ l))\ [0 ..< dd]).\ res-sem\ \text{?ev}\ rd = 0)$ 
   $\longleftrightarrow (\forall l < dd. xs\ !\ ro-kb\ l * (xs\ !\ ro-kb\ l - 1) = 0)$ 
  using  $ev-ro-nth-kb[OF\ len]\ \text{by}\ (auto\ simp: algebra-simps)$ 
have  $tag: (\forall rd \in set\ (if\ k = KNonMembership\ then\ [RTag\ ev-tag]\ else\ []).$ 
   $res-sem\ \text{?ev}\ rd = 0)$ 
   $\longleftrightarrow (k = KNonMembership \longrightarrow xs\ !\ ro-lp * (xs\ !\ ro-lp - 2) = 0)$ 
  using  $ev-ro-nth-tag[OF\ len]\ \text{by}\ (auto\ simp: algebra-simps)$ 
show  $?thesis$ 
  unfolding  $descs-ro-def\ cons-ro-def\ set-append\ ball-Un$ 
  using  $leaf\ chain\ root\ vd\ bool\ tag\ \text{by}\ blast$ 
qed
end

context  $compiler-model$ 
begin

theorem  $A-i-ro$ :
  assumes  $k: k \neq KUpdate$  and  $len: length\ xs = ivlen\ k$ 
  shows  $circuit-accept\ (compile\ k)\ xs \longleftrightarrow cons-ro\ k\ xs$ 
proof –
  have  $wk: in-width\ k = ro-width$  using  $k$  by  $(cases\ k)\ auto$ 
  have  $lenw: ro-width \leq length\ xs$ 
  using  $len\ in-width-le-ivlen[of\ k]\ wk$  by  $simp$ 
  have  $dk: descs\ k = descs-ro\ k$  using  $k$  by  $(cases\ k)\ auto$ 
  have  $circuit-accept\ (compile\ k)\ xs$ 
   $\longleftrightarrow (\forall v \in set\ (layer-eval\ (mk-res-layer\ (descs-ro\ k))\ (ev-ro\ xs)).\ v = 0)$ 
  by  $(simp\ add: circuit-accept-def\ compile-output[OF\ len]\ stack-spec-ro[OF\ k\ len]\ dk)$ 
also have  $\dots \longleftrightarrow (\forall rd \in set\ (descs-ro\ k).\ res-sem\ (ev-ro\ xs)\ rd = 0)$ 
  by  $(rule\ mk-res-layer-accept-iff)$ 
also have  $\dots \longleftrightarrow cons-ro\ k\ xs$ 
  by  $(rule\ res-sem-descs-ro-iff[OF\ lenw])$ 
  finally show  $?thesis$  .
qed

```

lemma *res-sem-descs-up-iff*:

assumes *len*: $\text{up-width} \leq \text{length } xs$

shows $(\forall rd \in \text{set } \text{descs-up}. \text{res-sem } (\text{ev-up } xs) \text{ } rd = 0) \longleftrightarrow \text{cons-up } xs$

proof –

let $?ev = \text{ev-up } xs$

have *leaf*: $(\forall rd \in \text{set } (\text{map } (\lambda i. \text{RDiff } (\text{ev2-hl} + i) (\text{ev2-acc } 0 + i)) [0 ..< W]).$
 $\text{res-sem } ?ev \text{ } rd = 0)$

$\longleftrightarrow \text{hl } (\text{seg up-lp } L \text{ } xs) = \text{seg } (\text{up-acc } 0) \text{ } W \text{ } xs$

using *ev-up-nth-hl*[*OF len*] *ev-up-nth-acc*[*OF len*, *of 0*]

by (*auto simp: list-eq-iff-nth-eq length-seg-up*[*OF len*])

have *leaf2*: $(\forall rd \in \text{set } (\text{map } (\lambda i. \text{RDiff } (\text{ev2-hl2} + i) (\text{ev2-acc2 } 0 + i)) [0 ..< W]).$
 $\text{res-sem } ?ev \text{ } rd = 0)$

$\longleftrightarrow \text{hl } (\text{seg up-lp2 } L \text{ } xs) = \text{seg } (\text{up-acc2 } 0) \text{ } W \text{ } xs$

using *ev-up-nth-hl*[*OF len*] *ev-up-nth-acc2*[*OF len*, *of 0*]

by (*auto simp: list-eq-iff-nth-eq length-seg-up*[*OF len*])

have *chain*: $(\forall rd \in \text{set } (\text{concat } (\text{map } (\lambda l. \text{map } (\lambda i. \text{RDiff } (\text{ev2-hn } l + i) (\text{ev2-acc } (\text{Suc } l) + i))$
 $[0 ..< W]) [0 ..< dd])).$

$\text{res-sem } ?ev \text{ } rd = 0)$

$\longleftrightarrow (\forall l < dd. \text{hn } (\text{up-mxl } xs \text{ } \text{up-acc } l) (\text{up-mxr } xs \text{ } \text{up-acc } l)$
 $= \text{seg } (\text{up-acc } (\text{Suc } l)) \text{ } W \text{ } xs)$

proof –

have $(\forall rd \in \text{set } (\text{concat } (\text{map } (\lambda l. \text{map } (\lambda i. \text{RDiff } (\text{ev2-hn } l + i) (\text{ev2-acc } (\text{Suc } l) + i))$
 $[0 ..< W]) [0 ..< dd])).$

$\text{res-sem } ?ev \text{ } rd = 0)$

$\longleftrightarrow (\forall l < dd. \forall i < W. ?ev ! (\text{ev2-hn } l + i) = ?ev ! (\text{ev2-acc } (\text{Suc } l) + i))$

by *auto*

also have $\dots \longleftrightarrow (\forall l < dd. \text{hn } (\text{up-mxl } xs \text{ } \text{up-acc } l) (\text{up-mxr } xs \text{ } \text{up-acc } l)$
 $= \text{seg } (\text{up-acc } (\text{Suc } l)) \text{ } W \text{ } xs)$

proof (*intro all-cong1 imp-cong*[*OF refl*])

fix *l* **assume** $l < dd$

then show $(\forall i < W. ?ev ! (\text{ev2-hn } l + i) = ?ev ! (\text{ev2-acc } (\text{Suc } l) + i))$

$\longleftrightarrow \text{hn } (\text{up-mxl } xs \text{ } \text{up-acc } l) (\text{up-mxr } xs \text{ } \text{up-acc } l) = \text{seg } (\text{up-acc } (\text{Suc } l)) \text{ } W \text{ } xs$

using *ev-up-nth-hn*[*OF len*] *ev-up-nth-acc*[*OF len*]

by (*auto simp: list-eq-iff-nth-eq length-seg-up*[*OF len*] *Suc-le-eq*)

qed

finally show *?thesis* .

qed

have *chain2*: $(\forall rd \in \text{set } (\text{concat } (\text{map } (\lambda l. \text{map } (\lambda i. \text{RDiff } (\text{ev2-hn2 } l + i) (\text{ev2-acc2 } (\text{Suc } l) + i))$
 $[0 ..< W]) [0 ..< dd])).$

$\text{res-sem } ?ev \text{ } rd = 0)$

$\longleftrightarrow (\forall l < dd. \text{hn } (\text{up-mxl } xs \text{ } \text{up-acc2 } l) (\text{up-mxr } xs \text{ } \text{up-acc2 } l)$
 $= \text{seg } (\text{up-acc2 } (\text{Suc } l)) \text{ } W \text{ } xs)$

proof –

have $(\forall rd \in \text{set } (\text{concat } (\text{map } (\lambda l. \text{map } (\lambda i. \text{RDiff } (\text{ev2-hn2 } l + i) (\text{ev2-acc2 } (\text{Suc } l) + i))$
 $[0 ..< W]) [0 ..< dd])).$

$\text{res-sem } ?ev \text{ } rd = 0)$

$\longleftrightarrow (\forall l < dd. \forall i < W. ?ev ! (\text{ev2-hn2 } l + i) = ?ev ! (\text{ev2-acc2 } (\text{Suc } l) + i))$

by *auto*

also have $\dots \longleftrightarrow (\forall l < dd. \text{hn } (\text{up-mxl } xs \text{ } \text{up-acc2 } l) (\text{up-mxr } xs \text{ } \text{up-acc2 } l)$
 $= \text{seg } (\text{up-acc2 } (\text{Suc } l)) \text{ } W \text{ } xs)$

proof (*intro all-cong1 imp-cong*[*OF refl*])

fix *l* **assume** $l < dd$

then show $(\forall i < W. ?ev ! (\text{ev2-hn2 } l + i) = ?ev ! (\text{ev2-acc2 } (\text{Suc } l) + i))$

```

       $\longleftrightarrow \text{hn } (\text{up-mxl } xs \text{ up-acc2 } l) (\text{up-mxr } xs \text{ up-acc2 } l) = \text{seg } (\text{up-acc2 } (\text{Suc } l)) \text{ } W \text{ } xs$ 
    using ev-up-nth-hn2[OF len] ev-up-nth-acc2[OF len]
    by (auto simp: list-eq-iff-nth-eq length-seg-up[OF len] Suc-le-eq)
  qed
  finally show ?thesis .
qed
have root:  $(\forall rd \in \text{set } (\text{map } (\lambda i. \text{RDiff } (\text{ev2-acc } dd + i) (\text{ev2-root} + i)) [0 ..< W])).$ 
       $\text{res-sem } ?\text{ev } rd = 0$ 
       $\longleftrightarrow \text{seg } (\text{up-acc } dd) \text{ } W \text{ } xs = \text{seg } \text{up-root } W \text{ } xs$ 
    using ev-up-nth-acc[OF len, of dd] ev-up-nth-root[OF len]
    by (auto simp: list-eq-iff-nth-eq length-seg-up[OF len])
have root2:  $(\forall rd \in \text{set } (\text{map } (\lambda i. \text{RDiff } (\text{ev2-acc2 } dd + i) (\text{ev2-root2} + i)) [0 ..< W])).$ 
       $\text{res-sem } ?\text{ev } rd = 0$ 
       $\longleftrightarrow \text{seg } (\text{up-acc2 } dd) \text{ } W \text{ } xs = \text{seg } \text{up-root2 } W \text{ } xs$ 
    using ev-up-nth-acc2[OF len, of dd] ev-up-nth-root2[OF len]
    by (auto simp: list-eq-iff-nth-eq length-seg-up[OF len])
have vd:  $(\forall rd \in \text{set } (\text{map } (\lambda i. \text{RDiff } (\text{ev2-acc2 } 0 + i) (\text{ev2-vd} + i)) [0 ..< W])).$ 
       $\text{res-sem } ?\text{ev } rd = 0$ 
       $\longleftrightarrow \text{seg } (\text{up-acc2 } 0) \text{ } W \text{ } xs = \text{seg } \text{up-vd } W \text{ } xs$ 
    using ev-up-nth-acc2[OF len, of 0] ev-up-nth-vd[OF len]
    by (auto simp: list-eq-iff-nth-eq length-seg-up[OF len])
have bool:  $(\forall rd \in \text{set } (\text{map } (\lambda l. \text{RBool } (\text{ev2-kb } l)) [0 ..< dd])). \text{res-sem } ?\text{ev } rd = 0$ 
       $\longleftrightarrow (\forall l < dd. xs ! \text{up-kb } l * (xs ! \text{up-kb } l - 1) = 0)$ 
    using ev-up-nth-kb[OF len] by (auto simp: algebra-simps)
show ?thesis
  unfolding descs-up-def cons-up-def set-append ball-Un
  using leaf leaf2 chain chain2 root root2 vd bool by blast
qed

theorem A-i-up:
  assumes len:  $\text{length } xs = \text{ivlen } K\text{Update}$ 
  shows  $\text{circuit-accept } (\text{compile } K\text{Update}) \text{ } xs \longleftrightarrow \text{cons-up } xs$ 
proof -
  have lenw:  $\text{up-width} \leq \text{length } xs$ 
  using len in-width-le-ivlen[of KUpdate] by simp
  have circuit-accept (compile KUpdate) xs
     $\longleftrightarrow (\forall v \in \text{set } (\text{layer-eval } (\text{mk-res-layer } \text{descs-up}) (\text{ev-up } xs)). v = 0)$ 
  by (simp add: circuit-accept-def compile-output[OF len] stack-spec-up[OF len])
  also have ...  $\longleftrightarrow (\forall rd \in \text{set } \text{descs-up}. \text{res-sem } (\text{ev-up } xs) \text{ } rd = 0)$ 
  by (rule mk-res-layer-accept-iff)
  also have ...  $\longleftrightarrow \text{cons-up } xs$ 
  by (rule res-sem-descs-up-iff[OF lenw])
  finally show ?thesis .
qed

end

```

21 Stage A-ii: the constraint conjunction is the SMT semantics

Stage A-i reduced circuit acceptance to the constraint conjunctions *cons-ro/cons-up* over the evaluation-stack vector. Stage A-ii evaluates those conjunctions on the HONEST witness encoding (*enc-ro/enc-up*, the literal *build-input-vector* transcription): the hash-binding, chain and booleanity clauses hold by construction (the encoder computes the honest accumulator chain), so the conjunction collapses to exactly the semantic equations - path equality, value-digest binding and (NonMembership) the leaf-tag domain. Together with the facade public-input checks (the *verifier-accept* model below, Rust: *verify-sync-op*) this yields Theorem A.

21.1 Theory-level helpers

The operation's asset key. Rust: *op-key* (witness.rs).

```
fun op-key :: 'v smt-op ⇒ nat where
  op-key (Membership k v) = k
| op-key (NonMembership k) = k
| op-key (Update k old new) = k
```

The tag residual $tag * (tag - 2)$ vanishes on the leaf-tag lane exactly for the Empty/Tombstone states - the circuit-immanent NonMembership domain separation. Occupied has tag 1, and 1 is neither 0 nor 2 in ANY ring with $0 \neq 1$ ($1 = 2$ would force $0 = 1$), so no characteristic assumption is needed. Stated in the disjunctive form the simplifier's *mult-eq-0-iff* normalisation produces.

```
lemma leaf-tag-val-zero-or-two:
  ((leaf-tag-val lf :: 'f::comm-ring-1) = 0 ∨ (leaf-tag-val lf :: 'f) = 2)
  ⇔ (lf = Empty ∨ lf = Tombstone)
proof (cases lf)
  case (Occupied v)
  have neg2: (1 :: 'f) ≠ 2
  proof
    assume a: (1 :: 'f) = 2
    have (1 :: 'f) + 0 = 1 + 1 using a by (simp add: one-add-one)
    then have (0 :: 'f) = 1 by (rule add-left-imp-eq)
    then show False by simp
  qed
  then show ?thesis by (simp add: Occupied)
qed simp-all
```

Extracting one block out of an explicit append decomposition.

```
lemma seg-at:
  length pre = a ⇒ length blk = w ⇒ seg a w (pre @ blk @ post) = blk
by (simp add: seg-def)
```

```
lemma nth-at:
  length pre = a ⇒ i < length blk ⇒ (pre @ blk @ post) ! (a + i) = blk ! i
by (simp add: nth-append)
```

```
lemma concat-map-upt-split:
```

```

assumes  $l < n$ 
shows  $\text{concat } (\text{map } f [0 ..< n])$ 
       $= \text{concat } (\text{map } f [0 ..< l]) @ f\ l @ \text{concat } (\text{map } f [Suc\ l ..< n])$ 
proof –
  from assms have  $n\text{-decomp}$ :  $n = l + Suc\ (n - Suc\ l)$  by arith
  have  $[0 ..< l + Suc\ (n - Suc\ l)] = [0 ..< l] @ [l ..< l + Suc\ (n - Suc\ l)]$ 
    by (rule upt-add-eq-append) simp
  moreover have  $[l ..< l + Suc\ (n - Suc\ l)] = l \# [Suc\ l ..< l + Suc\ (n - Suc\ l)]$ 
    by (rule upt-conv-Cons) simp
  ultimately have  $[0 ..< n] = [0 ..< l] @ l \# [Suc\ l ..< n]$ 
    using  $n\text{-decomp}$  by simp
  then show ?thesis by simp
qed

context compiler-model-base
begin

```

21.2 Segment values of the read-only encoding

```

lemma length-enc-ro-core:
   $\text{length } (\text{enc-ro-core } \text{leaf } \text{key } \text{sibs } \text{root } \text{vd}) = \text{ro-width}$ 
proof –
  have acc:  $\text{length } (\text{concat } (\text{map } (\lambda l. \text{digest-repr } (\text{acc-list } (\text{h-leaf } \text{leaf}) \text{key } \text{sibs } !\ l))$ 
     $[0 ..< Suc\ dd])) = Suc\ dd * W$ 
    by (intro concat-map-uniform-length) simp
  have sib:  $\text{length } (\text{concat } (\text{map } (\lambda l. \text{digest-repr } (\text{sibs } !\ l)) [0 ..< dd])) = dd * W$ 
    by (intro concat-map-uniform-length) simp
  show ?thesis
    unfolding enc-ro-core-def ro-width-def ro-vd-def ro-root-def
    by (simp only: length-append acc sib length-map length-upt digest-len leaf-len)
      (simp add: algebra-simps)
qed

```

```

lemma length-enc-ro:
  assumes  $k \neq KUpdate$ 
  shows  $\text{length } (\text{enc-ro } k \text{ leaf } \text{key } \text{sibs } \text{root } \text{vd}) = \text{ivlen } k$ 
proof –
  have in-width  $k = \text{ro-width}$  using assms by (cases  $k$ ) auto
  then have  $\text{length } (\text{enc-ro-core } \text{leaf } \text{key } \text{sibs } \text{root } \text{vd}) \leq \text{ivlen } k$ 
    using in-width-le-ivlen[of  $k$ ] by (simp add: length-enc-ro-core)
  then show ?thesis by (simp add: enc-ro-def length-pad-to)
qed

```

```

lemma enc-ro-core-segs:
  fixes  $\text{leaf} :: 'v \text{ leaf-state}$  and  $\text{key} :: \text{nat}$  and  $\text{sibs} :: 'd \text{ list}$  and  $\text{root } \text{vd} :: 'd$ 
  defines  $xs \equiv \text{enc-ro-core } \text{leaf } \text{key } \text{sibs } \text{root } \text{vd}$ 
  shows  $\text{enc-ro-core-lp}$ :  $\text{seg ro-lp } L\ xs = \text{leaf-repr } \text{leaf}$ 
    and  $\text{enc-ro-core-acc}$ :
       $\bigwedge l. l \leq dd \implies \text{seg } (\text{ro-acc } l)\ W\ xs$ 
       $= \text{digest-repr } (\text{acc-list } (\text{h-leaf } \text{leaf}) \text{key } \text{sibs } !\ l)$ 
    and  $\text{enc-ro-core-sib}$ :
       $\bigwedge l. l < dd \implies \text{seg } (\text{ro-sib } l)\ W\ xs = \text{digest-repr } (\text{sibs } !\ l)$ 
    and  $\text{enc-ro-core-kb}$ :  $\bigwedge l. l < dd \implies xs\ !\ \text{ro-kb } l = \text{kbit } \text{key } l$ 

```

```

and enc-ro-core-root: seg ro-root W xs = digest-repr root
and enc-ro-core-vd: seg ro-vd W xs = digest-repr vd
and enc-ro-core-tag: xs ! ro-lp = leaf-tag-val leaf
proof –
  let ?f =  $\lambda l. \text{digest-repr } (\text{acc-list } (h\text{-leaf leaf}) \text{ key sibs } ! l)$ 
  let ?g =  $\lambda l. \text{digest-repr } (\text{sibs } ! l)$ 
  let ?ACC =  $\text{concat } (\text{map } ?f [0 \dots < \text{Suc } dd])$ 
  let ?SIB =  $\text{concat } (\text{map } ?g [0 \dots < dd])$ 
  let ?KB =  $\text{map } (\text{kbit key}) [0 \dots < dd] :: 'f \text{ list}$ 
  have xs-eq: xs = leaf-repr leaf @ ?ACC @ ?SIB @ ?KB
     $\text{@ digest-repr root @ digest-repr vd}$ 
    by (simp add: xs-def enc-ro-core-def)
  have lACC: length ?ACC = Suc dd * W
    by (intro concat-map-uniform-length simp)
  have lSIB: length ?SIB = dd * W
    by (intro concat-map-uniform-length simp)

  show seg ro-lp L xs = leaf-repr leaf
    unfolding xs-eq ro-lp-def by (simp add: seg-def)

  show seg (ro-acc l) W xs = ?f l if l: l ≤ dd for l
  proof –
    have split: ?ACC = concat (map ?f [0 ..< l]) @ ?f l @ concat (map ?f [Suc l ..< Suc dd])
      by (rule concat-map-upt-split) (simp add: le-imp-less-Suc[OF l])
    have xs2: xs = (leaf-repr leaf @ concat (map ?f [0 ..< l])) @ ?f l
       $\text{@ (concat (map ?f [Suc l ..< Suc dd]) @ ?SIB @ ?KB}$ 
       $\text{@ digest-repr root @ digest-repr vd)}$ 
    unfolding xs-eq split by simp
    have lblk: length (concat (map ?f [0 ..< l])) = l * W
      by (intro concat-map-uniform-length simp)
    have lpre: length (leaf-repr leaf @ concat (map ?f [0 ..< l])) = ro-acc l
      by (simp add: lbld ro-acc-def mult.commute)
    show ?thesis unfolding xs2 by (rule seg-at[OF lpre]) simp
  qed

  show seg (ro-sib l) W xs = ?g l if l: l < dd for l
  proof –
    have split: ?SIB = concat (map ?g [0 ..< l]) @ ?g l @ concat (map ?g [Suc l ..< dd])
      by (rule concat-map-upt-split) (rule l)
    have xs2: xs = (leaf-repr leaf @ ?ACC @ concat (map ?g [0 ..< l])) @ ?g l
       $\text{@ (concat (map ?g [Suc l ..< dd]) @ ?KB}$ 
       $\text{@ digest-repr root @ digest-repr vd)}$ 
    unfolding xs-eq split by simp
    have lblk: length (concat (map ?g [0 ..< l])) = l * W
      by (intro concat-map-uniform-length simp)
    have lpre: length (leaf-repr leaf @ ?ACC @ concat (map ?g [0 ..< l])) = ro-sib l
      unfolding ro-sib-def
      by (simp only: length-append lACC lbld leaf-len) (simp add: algebra-simps)
    show ?thesis unfolding xs2 by (rule seg-at[OF lpre]) simp
  qed

  show xs ! ro-kb l = kbit key l if l: l < dd for l
  proof –

```

```

have xs2: xs = (leaf-repr leaf @ ?ACC @ ?SIB) @ ?KB
               @ (digest-repr root @ digest-repr vd)
  unfolding xs-eq by simp
have lpre: length (leaf-repr leaf @ ?ACC @ ?SIB) = L + W * (2 * dd + 1)
  by (simp only: length-append LACC lSIB leaf-len) (simp add: algebra-simps)
have xs ! (L + W * (2 * dd + 1) + l) = ?KB ! l
  unfolding xs2 by (rule nth-at[OF lpre]) (simp add: l)
then show ?thesis using l by (simp add: ro-kb-def)
qed

show seg ro-root W xs = digest-repr root
proof -
  have xs2: xs = (leaf-repr leaf @ ?ACC @ ?SIB @ ?KB) @ digest-repr root
               @ digest-repr vd
  unfolding xs-eq by simp
have lpre: length (leaf-repr leaf @ ?ACC @ ?SIB @ ?KB) = ro-root
  unfolding ro-root-def
  by (simp only: length-append LACC lSIB leaf-len length-map length-upt)
  (simp add: algebra-simps)
show ?thesis unfolding xs2 by (rule seg-at[OF lpre]) simp
qed

show seg ro-vd W xs = digest-repr vd
proof -
  have xs2: xs = (leaf-repr leaf @ ?ACC @ ?SIB @ ?KB @ digest-repr root)
               @ digest-repr vd @ []
  unfolding xs-eq by simp
have lpre: length (leaf-repr leaf @ ?ACC @ ?SIB @ ?KB @ digest-repr root) = ro-vd
  unfolding ro-vd-def ro-root-def
  by (simp only: length-append LACC lSIB leaf-len length-map length-upt
               digest-len)
  (simp add: algebra-simps)
show ?thesis unfolding xs2 by (rule seg-at[OF lpre]) simp
qed

show xs ! ro-lp = leaf-tag-val leaf
  using L-pos leaf-tag[of leaf]
  unfolding xs-eq ro-lp-def
  by (simp add: nth-append)
qed

lemma enc-ro-segs:
  fixes leaf :: 'v leaf-state and key :: nat and sibs :: 'd list and root vd :: 'd
  assumes k: k ≠ KUpdate
  defines xs ≡ enc-ro k leaf key sibs root vd
  shows enc-ro-lp: seg ro-lp L xs = leaf-repr leaf
    and enc-ro-acc:
      ∧ l. l ≤ dd ⇒ seg (ro-acc l) W xs
                    = digest-repr (acc-list (h-leaf leaf) key sibs ! l)
    and enc-ro-sib:
      ∧ l. l < dd ⇒ seg (ro-sib l) W xs = digest-repr (sibs ! l)
    and enc-ro-kb: ∧ l. l < dd ⇒ xs ! ro-kb l = kbit key l
    and enc-ro-root: seg ro-root W xs = digest-repr root

```

and *enc-ro-vd*: $\text{seg ro-vd } W \text{ xs} = \text{digest-repr vd}$
and *enc-ro-tag*: $\text{xs} ! \text{ro-lp} = \text{leaf-tag-val leaf}$
proof –
let *?core* = *enc-ro-core leaf key sibs root vd*
have *lcore*: $\text{length } ?\text{core} = \text{ro-width}$ **by** (*rule length-enc-ro-core*)
have *xs-pad*: $\text{xs} = \text{pad-to (ivlen } k) ?\text{core}$ **by** (*simp add: xs-def enc-ro-def*)
have *segP*: $a + w \leq \text{ro-width} \implies \text{seg } a \text{ w xs} = \text{seg } a \text{ w } ?\text{core}$ **for** *a w*
unfolding *xs-pad* **by** (*rule seg-pad-to*) (*simp add: lcore*)
have *nthP*: $i < \text{ro-width} \implies \text{xs} ! i = ?\text{core} ! i$ **for** *i*
unfolding *xs-pad* **by** (*rule pad-to-nth*) (*simp add: lcore*)
show *seg ro-lp L xs* = *leaf-repr leaf*
using *segP[OF ro-seg-bounds(1)] enc-ro-core-lp* **by** *simp*
show *seg (ro-acc l) W xs* = *digest-repr (acc-list (h-leaf leaf) key sibs ! l)*
if *l*: $l \leq \text{dd}$ **for** *l*
using *segP[OF ro-seg-bounds(2)][OF l] enc-ro-core-acc[OF l]* **by** *simp*
show *seg (ro-sib l) W xs* = *digest-repr (sibs ! l)* **if** *l*: $l < \text{dd}$ **for** *l*
using *segP[OF ro-seg-bounds(3)][OF l] enc-ro-core-sib[OF l]* **by** *simp*
show $\text{xs} ! \text{ro-kb } l = \text{kbit key } l$ **if** *l*: $l < \text{dd}$ **for** *l*
using *nthP[OF ro-seg-bounds(4)][OF l] enc-ro-core-kb[OF l]* **by** *simp*
show *seg ro-root W xs* = *digest-repr root*
using *segP[OF ro-seg-bounds(5)] enc-ro-core-root* **by** *simp*
show *seg ro-vd W xs* = *digest-repr vd*
using *segP[OF ro-seg-bounds(6)] enc-ro-core-vd* **by** *simp*
have *lp-lt*: $\text{ro-lp} < \text{ro-width}$
using *L-pos ro-seg-bounds(1)* **by** (*simp add: ro-lp-def*)
show $\text{xs} ! \text{ro-lp} = \text{leaf-tag-val leaf}$
using *nthP[OF lp-lt] enc-ro-core-tag* **by** *simp*
qed

21.3 The read-only constraint conjunction on the honest encoding

On *enc-ro* the hash-binding, chain and booleanity clauses hold by construction (the encoder computes the honest accumulator chain), so *cons-ro* collapses to: path equality, value-digest binding, and (for NonMembership) the leaf-tag domain.

lemma *cons-ro-enc*:

assumes *k*: $k \neq \text{KUpdate}$ **and** *sl*: $\text{length sibs} = \text{dd}$
shows *cons-ro k (enc-ro k leaf key sibs root vd)* $\longleftrightarrow$
 $(\text{path-root (h-leaf leaf) key sibs} = \text{root} \wedge \text{vd} = \text{h-leaf leaf} \wedge$
 $(k = \text{KNonMembership} \longrightarrow \text{leaf} = \text{Empty} \vee \text{leaf} = \text{Tombstone}))$

proof –

let *?xs* = *enc-ro k leaf key sibs root vd*
let *?acc* = *acc-list (h-leaf leaf) key sibs*
have *dig-iff*: $\text{digest-repr } x = \text{digest-repr } y \longleftrightarrow x = y$ **for** *x y*
using *digest-inj* **by** *auto*

have *c1*: $\text{hl (seg ro-lp L ?xs)} = \text{seg (ro-acc 0) W ?xs}$
using *enc-ro-lp[OF k] enc-ro-acc[OF k, of 0]*
by (*simp add: hl-correct*)

have *c2*: $\text{hn (ro-mxl ?xs l) (ro-mxr ?xs l)} = \text{seg (ro-acc (Suc l)) W ?xs}$
if *l*: $l < \text{dd}$ **for** *l*

proof –

```

have accl: seg (ro-acc l) W ?xs = digest-repr (?acc ! l)
  using enc-ro-acc[OF k] l by simp
have accSl: seg (ro-acc (Suc l)) W ?xs = digest-repr (?acc ! Suc l)
  using enc-ro-acc[OF k] l by (simp add: Suc-le-eq)
have sibl: seg (ro-sib l) W ?xs = digest-repr (sibs ! l)
  using enc-ro-sib[OF k] l by simp
have kbl: ?xs ! ro-kb l = kbit key l
  using enc-ro-kb[OF k] l by simp
have step: ?acc ! Suc l =
  (if (key div 2 ^ l) mod 2 = 0
    then h-node (?acc ! l) (sibs ! l)
    else h-node (sibs ! l) (?acc ! l))
  by (rule acc-list-step) (simp add: sl l)
show ?thesis
proof (cases (key div 2 ^ l) mod 2 = 0)
case True
  have sel: ?xs ! ro-kb l = 0
    using True by (simp add: kbl kbit-def)
  have mxl: ro-mxl ?xs l = digest-repr (?acc ! l)
    by (simp add: ro-mxl-def sel accl sibl mux-zero)
  have mxr: ro-mxr ?xs l = digest-repr (sibs ! l)
    by (simp add: ro-mxr-def sel accl sibl mux-zero)
  have hn (ro-mxl ?xs l) (ro-mxr ?xs l)
    = digest-repr (h-node (?acc ! l) (sibs ! l))
    by (simp add: mxl mxr hn-correct)
  with step True accSl show ?thesis by simp
next
case False
  have lt2: (key div 2 ^ l) mod 2 < 2 by simp
  with False have m1: (key div 2 ^ l) mod 2 = 1 by arith
  have sel: ?xs ! ro-kb l = 1
    by (simp add: kbl kbit-def m1)
  have mxl: ro-mxl ?xs l = digest-repr (sibs ! l)
    by (simp add: ro-mxl-def sel accl sibl mux-one)
  have mxr: ro-mxr ?xs l = digest-repr (?acc ! l)
    by (simp add: ro-mxr-def sel accl sibl mux-one)
  have hn (ro-mxl ?xs l) (ro-mxr ?xs l)
    = digest-repr (h-node (sibs ! l) (?acc ! l))
    by (simp add: mxl mxr hn-correct)
  with step False accSl show ?thesis by simp
qed
qed

have c5: ?xs ! ro-kb l * (?xs ! ro-kb l - 1) = 0 if l: l < dd for l
  unfolding enc-ro-kb[OF k l] by (rule kbit-bool)

have accdd: ?acc ! dd = path-root (h-leaf leaf) key sibs
proof -
  have ?acc ! dd = last ?acc
    by (simp add: last-conv-nth sl)
  then show ?thesis by (simp add: acc-list-last)
qed

```

```

have c3: seg (ro-acc dd) W ?xs = seg ro-root W ?xs  $\longleftrightarrow$ 
  path-root (h-leaf leaf) key sibs = root
using enc-ro-acc[OF k, of dd] enc-ro-root[OF k]
by (simp add: dig-iff accdd)

have c4: seg (ro-acc 0) W ?xs = seg ro-vd W ?xs  $\longleftrightarrow$  vd = h-leaf leaf
using enc-ro-acc[OF k, of 0] enc-ro-vd[OF k]
by (auto simp add: dig-iff)

have c6: ?xs ! ro-lp * (?xs ! ro-lp - 2) = 0
   $\longleftrightarrow$  (leaf = Empty  $\vee$  leaf = Tombstone)
unfolding enc-ro-tag[OF k]
by (simp add: leaf-tag-val-zero-or-two)

show ?thesis
unfolding cons-ro-def
using c1 c2 c3 c4 c5 c6 by blast
qed

```

21.4 Segment values of the Update encoding

```

lemma length-enc-up-core:
  length (enc-up-core old new key sibs root root' vd) = up-width
proof -
  have acc1: length (concat (map ( $\lambda$ l. digest-repr (acc-list (h-leaf old) key sibs ! l))
    [0 ..< Suc dd])) = Suc dd * W
  by (intro concat-map-uniform-length) simp
  have acc2: length (concat (map ( $\lambda$ l. digest-repr (acc-list (h-leaf new) key sibs ! l))
    [0 ..< Suc dd])) = Suc dd * W
  by (intro concat-map-uniform-length) simp
  have sib: length (concat (map ( $\lambda$ l. digest-repr (sibs ! l)) [0 ..< dd])) = dd * W
  by (intro concat-map-uniform-length) simp
  show ?thesis
  unfolding enc-up-core-def up-width-def up-vd-def up-root2-def up-root-def
  by (simp only: length-append acc1 acc2 sib length-map length-upt
    digest-len leaf-len)
  (simp add: algebra-simps)
qed

```

```

lemma length-enc-up:
  length (enc-up old new key sibs root root' vd) = ivlen KUpdate
proof -
  have length (enc-up-core old new key sibs root root' vd)  $\leq$  ivlen KUpdate
  using in-width-le-ivlen[of KUpdate] by (simp add: length-enc-up-core)
  then show ?thesis by (simp add: enc-up-def length-pad-to)
qed

```

```

lemma enc-up-segs:
  fixes old new :: 'v leaf-state and key :: nat and sibs :: 'd list
  and root root' vd :: 'd
  defines xs  $\equiv$  enc-up old new key sibs root root' vd
  shows enc-up-lp: seg up-lp L xs = leaf-repr old
  and enc-up-lp2: seg up-lp2 L xs = leaf-repr new

```

```

and enc-up-acc:
   $\bigwedge l. l \leq dd \implies \text{seg } (\text{up-acc } l) \ W \ xs$ 
  = digest-repr (acc-list (h-leaf old) key sibs ! l)
and enc-up-acc2:
   $\bigwedge l. l \leq dd \implies \text{seg } (\text{up-acc2 } l) \ W \ xs$ 
  = digest-repr (acc-list (h-leaf new) key sibs ! l)
and enc-up-sib:
   $\bigwedge l. l < dd \implies \text{seg } (\text{up-sib } l) \ W \ xs = \text{digest-repr } (sibs ! l)$ 
and enc-up-kb:  $\bigwedge l. l < dd \implies xs ! \text{up-kb } l = \text{kbit key } l$ 
and enc-up-root:  $\text{seg up-root } W \ xs = \text{digest-repr root}$ 
and enc-up-root2:  $\text{seg up-root2 } W \ xs = \text{digest-repr root'}$ 
and enc-up-vd:  $\text{seg up-vd } W \ xs = \text{digest-repr vd}$ 
proof -
  let ?f1 =  $\lambda l. \text{digest-repr } (\text{acc-list } (h\text{-leaf old}) \text{ key sibs ! } l)$ 
  let ?f2 =  $\lambda l. \text{digest-repr } (\text{acc-list } (h\text{-leaf new}) \text{ key sibs ! } l)$ 
  let ?g =  $\lambda l. \text{digest-repr } (sibs ! l)$ 
  let ?ACC1 = concat (map ?f1 [0 ..< Suc dd])
  let ?ACC2 = concat (map ?f2 [0 ..< Suc dd])
  let ?SIB = concat (map ?g [0 ..< dd])
  let ?KB = map (kbit key) [0 ..< dd] :: 'f list
  let ?core = enc-up-core old new key sibs root root' vd
  have core-eq: ?core = leaf-repr old @ leaf-repr new @ ?ACC1 @ ?ACC2 @ ?SIB @ ?KB
    @ digest-repr root @ digest-repr root' @ digest-repr vd
    by (simp add: enc-up-core-def)
  have lACC1: length ?ACC1 = Suc dd * W
    by (intro concat-map-uniform-length) simp
  have lACC2: length ?ACC2 = Suc dd * W
    by (intro concat-map-uniform-length) simp
  have lSIB: length ?SIB = dd * W
    by (intro concat-map-uniform-length) simp
  have lcore: length ?core = up-width by (rule length-enc-up-core)
  have xs-pad: xs = pad-to (ivlen KUpdate) ?core by (simp add: xs-def enc-up-def)
  have segP:  $a + w \leq \text{up-width} \implies \text{seg } a \ w \ xs = \text{seg } a \ w \ ?core$  for  $a \ w$ 
    unfolding xs-pad by (rule seg-pad-to) (simp add: lcore)
  have nthP:  $i < \text{up-width} \implies xs ! i = ?core ! i$  for  $i$ 
    unfolding xs-pad by (rule pad-to-nth) (simp add: lcore)

show seg up-lp L xs = leaf-repr old
  using segP[OF up-seg-bounds(1)]
  unfolding core-eq up-lp-def by (simp add: seg-def)

show seg up-lp2 L xs = leaf-repr new
proof -
  have c2: ?core = leaf-repr old @ leaf-repr new
    @ (?ACC1 @ ?ACC2 @ ?SIB @ ?KB @ digest-repr root
      @ digest-repr root' @ digest-repr vd)
    unfolding core-eq by simp
  have seg up-lp2 L ?core = leaf-repr new
    unfolding c2 by (rule seg-at[of leaf-repr old]) (simp-all add: up-lp2-def)
  then show ?thesis using segP[OF up-seg-bounds(2)] by simp
qed

show seg (up-acc l) W xs = ?f1 l if  $l: l \leq dd$  for  $l$ 

```

proof –

have *split*: $?ACC1 = \text{concat} (\text{map } ?f1 [0 \dots l]) @ ?f1 l$
 $\quad @ \text{concat} (\text{map } ?f1 [Suc\ l \dots Suc\ dd])$
by (*rule concat-map-upt-split*) (*simp add: le-imp-less-Suc[OF l]*)
have *c2*: $?core = (\text{leaf-repr old} @ \text{leaf-repr new} @ \text{concat} (\text{map } ?f1 [0 \dots l]))$
 $\quad @ ?f1 l$
 $\quad @ (\text{concat} (\text{map } ?f1 [Suc\ l \dots Suc\ dd]) @ ?ACC2 @ ?SIB @ ?KB$
 $\quad @ \text{digest-repr root} @ \text{digest-repr root}' @ \text{digest-repr vd})$
unfolding *core-eq split* **by** *simp*
have *lblk*: $\text{length} (\text{concat} (\text{map } ?f1 [0 \dots l])) = l * W$
by (*intro concat-map-uniform-length*) *simp*
have *lpre*: $\text{length} (\text{leaf-repr old} @ \text{leaf-repr new} @ \text{concat} (\text{map } ?f1 [0 \dots l]))$
 $\quad = \text{up-acc } l$
by (*simp add: lbk up-acc-def mult.commute*)
have *seg* (*up-acc l*) *W* $?core = ?f1 l$
unfolding *c2* **by** (*rule seg-at[OF lpre]*) *simp*
then show *?thesis* **using** *segP[OF up-seg-bounds(3)[OF l]]* **by** *simp*
qed

show *seg* (*up-acc2 l*) *W* $xs = ?f2\ l$ **if** *l*: $l \leq dd$ **for** *l*

proof –

have *split*: $?ACC2 = \text{concat} (\text{map } ?f2 [0 \dots l]) @ ?f2 l$
 $\quad @ \text{concat} (\text{map } ?f2 [Suc\ l \dots Suc\ dd])$
by (*rule concat-map-upt-split*) (*simp add: le-imp-less-Suc[OF l]*)
have *c2*: $?core = (\text{leaf-repr old} @ \text{leaf-repr new} @ ?ACC1 @ \text{concat} (\text{map } ?f2 [0 \dots l]))$
 $\quad @ ?f2 l$
 $\quad @ (\text{concat} (\text{map } ?f2 [Suc\ l \dots Suc\ dd]) @ ?SIB @ ?KB$
 $\quad @ \text{digest-repr root} @ \text{digest-repr root}' @ \text{digest-repr vd})$
unfolding *core-eq split* **by** *simp*
have *lblk*: $\text{length} (\text{concat} (\text{map } ?f2 [0 \dots l])) = l * W$
by (*intro concat-map-uniform-length*) *simp*
have *lpre*: $\text{length} (\text{leaf-repr old} @ \text{leaf-repr new} @ ?ACC1 @ \text{concat} (\text{map } ?f2 [0 \dots l]))$
 $\quad = \text{up-acc2 } l$
unfolding *up-acc2-def*
by (*simp only: length-append lACC1 lbk leaf-len*) (*simp add: algebra-simps*)
have *seg* (*up-acc2 l*) *W* $?core = ?f2 l$
unfolding *c2* **by** (*rule seg-at[OF lpre]*) *simp*
then show *?thesis* **using** *segP[OF up-seg-bounds(4)[OF l]]* **by** *simp*
qed

show *seg* (*up-sib l*) *W* $xs = ?g\ l$ **if** *l*: $l < dd$ **for** *l*

proof –

have *split*: $?SIB = \text{concat} (\text{map } ?g [0 \dots l]) @ ?g\ l @ \text{concat} (\text{map } ?g [Suc\ l \dots dd])$
by (*rule concat-map-upt-split*) (*rule l*)
have *c2*: $?core = (\text{leaf-repr old} @ \text{leaf-repr new} @ ?ACC1 @ ?ACC2$
 $\quad @ \text{concat} (\text{map } ?g [0 \dots l]))$
 $\quad @ ?g\ l$
 $\quad @ (\text{concat} (\text{map } ?g [Suc\ l \dots dd]) @ ?KB$
 $\quad @ \text{digest-repr root} @ \text{digest-repr root}' @ \text{digest-repr vd})$
unfolding *core-eq split* **by** *simp*
have *lblk*: $\text{length} (\text{concat} (\text{map } ?g [0 \dots l])) = l * W$
by (*intro concat-map-uniform-length*) *simp*
have *lpre*: $\text{length} (\text{leaf-repr old} @ \text{leaf-repr new} @ ?ACC1 @ ?ACC2$

```

      @ concat (map ?g [0 ..< l]) = up-sib l
    unfolding up-sib-def
  by (simp only: length-append LACC1 LACC2 blk leaf-len)
    (simp add: algebra-simps)
  have seg (up-sib l) W ?core = ?g l
    unfolding c2 by (rule seg-at[OF lpre]) simp
  then show ?thesis using segP[OF up-seg-bounds(5)[OF l]] by simp
qed

show xs ! up-kb l = kbit key l if l: l < dd for l
proof -
  have c2: ?core = (leaf-repr old @ leaf-repr new @ ?ACC1 @ ?ACC2 @ ?SIB) @ ?KB
    @ (digest-repr root @ digest-repr root' @ digest-repr vd)
    unfolding core-eq by simp
  have lpre: length (leaf-repr old @ leaf-repr new @ ?ACC1 @ ?ACC2 @ ?SIB)
    = 2 * L + W * (3 * dd + 2)
    by (simp only: length-append LACC1 LACC2 lSIB leaf-len)
      (simp add: algebra-simps)
  have ?core ! (2 * L + W * (3 * dd + 2) + l) = ?KB ! l
    unfolding c2 by (rule nth-at[OF lpre]) (simp add: l)
  then have ?core ! up-kb l = kbit key l
    using l by (simp add: up-kb-def)
  then show ?thesis using nthP[OF up-seg-bounds(6)[OF l]] by simp
qed

show seg up-root W xs = digest-repr root
proof -
  have c2: ?core = (leaf-repr old @ leaf-repr new @ ?ACC1 @ ?ACC2 @ ?SIB @ ?KB)
    @ digest-repr root @ (digest-repr root' @ digest-repr vd)
    unfolding core-eq by simp
  have lpre: length (leaf-repr old @ leaf-repr new @ ?ACC1 @ ?ACC2 @ ?SIB @ ?KB)
    = up-root
    unfolding up-root-def
    by (simp only: length-append LACC1 LACC2 lSIB leaf-len length-map length-upt)
      (simp add: algebra-simps)
  have seg up-root W ?core = digest-repr root
    unfolding c2 by (rule seg-at[OF lpre]) simp
  then show ?thesis using segP[OF up-seg-bounds(7)] by simp
qed

show seg up-root2 W xs = digest-repr root'
proof -
  have c2: ?core = (leaf-repr old @ leaf-repr new @ ?ACC1 @ ?ACC2 @ ?SIB @ ?KB
    @ digest-repr root)
    @ digest-repr root' @ digest-repr vd
    unfolding core-eq by simp
  have lpre: length (leaf-repr old @ leaf-repr new @ ?ACC1 @ ?ACC2 @ ?SIB @ ?KB
    @ digest-repr root) = up-root2
    unfolding up-root2-def up-root-def
    by (simp only: length-append LACC1 LACC2 lSIB leaf-len length-map length-upt
      digest-len)
      (simp add: algebra-simps)
  have seg up-root2 W ?core = digest-repr root'

```

```

    unfolding c2 by (rule seg-at[OF lpre]) simp
  then show ?thesis using segP[OF up-seg-bounds(8)] by simp
qed

show seg up-vd W xs = digest-repr vd
proof -
  have c2: ?core = (leaf-repr old @ leaf-repr new @ ?ACC1 @ ?ACC2 @ ?SIB @ ?KB
    @ digest-repr root @ digest-repr root')
    @ digest-repr vd @ []
  unfolding core-eq by simp
  have lpre: length (leaf-repr old @ leaf-repr new @ ?ACC1 @ ?ACC2 @ ?SIB @ ?KB
    @ digest-repr root @ digest-repr root') = up-vd
  unfolding up-vd-def up-root2-def up-root-def
  by (simp only: length-append lACC1 lACC2 lSIB leaf-len length-map length-upt
    digest-len)
    (simp add: algebra-simps)
  have seg up-vd W ?core = digest-repr vd
  unfolding c2 by (rule seg-at[OF lpre]) simp
  then show ?thesis using segP[OF up-seg-bounds(9)] by simp
qed
qed

```

21.5 The Update constraint conjunction on the honest encoding

One compression-chain step holds by construction (either chain).

lemma *up-chain-holds*:

```

  assumes accF:  $\bigwedge j. j \leq dd \implies \text{seg } (\text{accoff } j) \ W \ xs$ 
    = digest-repr (acc-list a0 key sibs ! j)
  and sibF:  $\text{seg } (\text{up-sib } l) \ W \ xs = \text{digest-repr } (sibs ! l)$ 
  and kbF:  $xs ! \text{up-kb } l = \text{kbit key } l$ 
  and sl:  $\text{length sibs} = dd$  and l:  $l < dd$ 
  shows hn (up-mxl xs accoff l) (up-mxr xs accoff l) = seg (accoff (Suc l)) W xs

```

proof –

```

  let ?acc = acc-list a0 key sibs
  have accl:  $\text{seg } (\text{accoff } l) \ W \ xs = \text{digest-repr } (?acc ! l)$ 
  using accF l by simp
  have accSl:  $\text{seg } (\text{accoff } (Suc l)) \ W \ xs = \text{digest-repr } (?acc ! Suc l)$ 
  using accF l by (simp add: Suc-le-eq)
  have step:  $?acc ! Suc l =$ 
    (if (key div 2 ^ l) mod 2 = 0
      then h-node (?acc ! l) (sibs ! l)
      else h-node (sibs ! l) (?acc ! l))
  by (rule acc-list-step) (simp add: sl l)
  show ?thesis
  proof (cases (key div 2 ^ l) mod 2 = 0)
  case True
  have sel:  $xs ! \text{up-kb } l = 0$ 
  using True by (simp add: kbF kbit-def)
  have mxl:  $\text{up-mxl } xs \ \text{accoff } l = \text{digest-repr } (?acc ! l)$ 
  by (simp add: up-mxl-def sel accl sibF mux-zero)
  have mxr:  $\text{up-mxr } xs \ \text{accoff } l = \text{digest-repr } (sibs ! l)$ 
  by (simp add: up-mxr-def sel accl sibF mux-zero)

```

```

  have hn (up-mxl xs accoff l) (up-mxr xs accoff l)
    = digest-repr (h-node (?acc ! l) (sibs ! l))
    by (simp add: mxl mxr hn-correct)
  with step True accSl show ?thesis by simp
next
case False
have lt2: (key div 2 ^ l) mod 2 < 2 by simp
with False have m1: (key div 2 ^ l) mod 2 = 1 by arith
have sel: xs ! up-kb l = 1
  by (simp add: kbF kbit-def m1)
have mxl: up-mxl xs accoff l = digest-repr (sibs ! l)
  by (simp add: up-mxl-def sel accl sibF mux-one)
have mxr: up-mxr xs accoff l = digest-repr (?acc ! l)
  by (simp add: up-mxr-def sel accl sibF mux-one)
have hn (up-mxl xs accoff l) (up-mxr xs accoff l)
  = digest-repr (h-node (sibs ! l) (?acc ! l))
  by (simp add: mxl mxr hn-correct)
with step False accSl show ?thesis by simp
qed
qed

lemma cons-up-enc:
  assumes sl: length sibs = dd
  shows cons-up (enc-up old new key sibs root root' vd)  $\longleftrightarrow$ 
    (path-root (h-leaf old) key sibs = root  $\wedge$ 
     path-root (h-leaf new) key sibs = root'  $\wedge$  vd = h-leaf new)
proof -
  let ?xs = enc-up old new key sibs root root' vd
  let ?acc1 = acc-list (h-leaf old) key sibs
  let ?acc2 = acc-list (h-leaf new) key sibs
  have dig-iff: digest-repr x = digest-repr y  $\longleftrightarrow$  x = y for x y
    using digest-inj by auto

  have c1: hl (seg up-lp L ?xs) = seg (up-acc 0) W ?xs
    using enc-up-lp enc-up-acc[of 0] by (simp add: hl-correct)
  have c1': hl (seg up-lp2 L ?xs) = seg (up-acc2 0) W ?xs
    using enc-up-lp2 enc-up-acc2[of 0] by (simp add: hl-correct)

  have c2: hn (up-mxl ?xs up-acc l) (up-mxr ?xs up-acc l)
    = seg (up-acc (Suc l)) W ?xs if l: l < dd for l
    by (rule up-chain-holds[OF enc-up-acc enc-up-sib[OF l] enc-up-kb[OF l] sl l])
  have c2': hn (up-mxl ?xs up-acc2 l) (up-mxr ?xs up-acc2 l)
    = seg (up-acc2 (Suc l)) W ?xs if l: l < dd for l
    by (rule up-chain-holds[OF enc-up-acc2 enc-up-sib[OF l] enc-up-kb[OF l] sl l])

  have c8: ?xs ! up-kb l * (?xs ! up-kb l - 1) = 0 if l: l < dd for l
    unfolding enc-up-kb[OF l] by (rule kbit-bool)

  have accdd1: ?acc1 ! dd = path-root (h-leaf old) key sibs
  proof -
    have ?acc1 ! dd = last ?acc1 by (simp add: last-conv-nth sl)
    then show ?thesis by (simp add: acc-list-last)
  qed

```

```

have accdd2: ?acc2 ! dd = path-root (h-leaf new) key sibs
proof -
  have ?acc2 ! dd = last ?acc2 by (simp add: last-conv-nth sl)
  then show ?thesis by (simp add: acc-list-last)
qed

have c5: seg (up-acc dd) W ?xs = seg up-root W ?xs  $\longleftrightarrow$ 
  path-root (h-leaf old) key sibs = root
  using enc-up-acc[of dd] enc-up-root by (simp add: dig-iff accdd1)
have c6: seg (up-acc2 dd) W ?xs = seg up-root2 W ?xs  $\longleftrightarrow$ 
  path-root (h-leaf new) key sibs = root'
  using enc-up-acc2[of dd] enc-up-root2 by (simp add: dig-iff accdd2)
have c7: seg (up-acc2 0) W ?xs = seg up-vd W ?xs  $\longleftrightarrow$  vd = h-leaf new
  using enc-up-acc2[of 0] enc-up-vd by (auto simp add: dig-iff)

show ?thesis
  unfolding cons-up-def
  using c1 c1' c2 c2' c5 c6 c7 c8 by blast
qed

end

```

22 Theorem A: the verifier model and the compilation soundness

The complete verifier = the facade public-input checks (Rust: *verify-sync-op*'s native guards) plus circuit acceptance on the honest witness encoding. The value-digest canonicity (*vd-ok*) is derived from CANONICAL/public values only, never the private witness leaf (the fix for a false non-membership acceptance), and the two facade echoes mirror the literal code.

Theorem A then holds DETERMINISTICALLY (no collision resistance):

- soundness: an accepted (vd, w) yields *smt-valid* for the CANONICAL witness - identical to w except that Membership replaces the witness leaf by *Occupied v*. (The witness leaf reaches the circuit only through its *h-leaf* image, so the semantic clauses transport along *h-leaf leaf* = *vd* = *h-leaf (Occupied v)*; pinning the witness leaf ITSELF is exactly an *h-leaf* injectivity question, which lemma (b) reduces to Poseidon2 sponge collisions - system-level territory, property boundary.)
- completeness: a semantically valid witness is accepted with the canonical value digest.
- packaging: acceptance for SOME (vd, w) iff *smt-valid* for SOME w.

```

context compiler-model
begin

```

```

definition vd-ok :: 'v smt-op  $\Rightarrow$  'd  $\Rightarrow$  bool where
  vd-ok op vd = (case op of
    | Membership k v  $\Rightarrow$  vd = h-leaf (Occupied v)
    | NonMembership k  $\Rightarrow$  vd = h-leaf Empty  $\vee$  vd = h-leaf Tombstone
    | Update k old new  $\Rightarrow$  vd = h-leaf new)

```

definition *pub-ok* :: 'v smt-op $\Rightarrow$ 'd $\Rightarrow$ 'd $\Rightarrow$ 'd $\Rightarrow$ bool **where**
pub-ok op root root' vd $\longleftrightarrow$
 (kind-of op $\neq$ KUpdate $\longrightarrow$ root' = root) $\wedge$
 op-key op < 2 $\wedge$ dd $\wedge$ vd-ok op vd

definition *echo-ok* :: 'v smt-op $\Rightarrow$ ('v leaf-state $\times$ 'd list) $\Rightarrow$ bool **where**
echo-ok op w = (case op of
 Membership k v $\Rightarrow$ True
 | NonMembership k $\Rightarrow$ fst w = Empty $\vee$ fst w = Tombstone
 | Update k old new $\Rightarrow$ fst w = old)

definition *verifier-accept* :: 'v smt-op $\Rightarrow$ 'd $\Rightarrow$ 'd $\Rightarrow$ 'd $\Rightarrow$ ('v leaf-state $\times$ 'd list) $\Rightarrow$ bool **where**
verifier-accept op root root' vd w $\longleftrightarrow$
pub-ok op root root' vd $\wedge$ length (snd w) = dd $\wedge$ *echo-ok* op w $\wedge$
circuit-accept (compile (kind-of op)) (encode-witness op root root' vd w)

The canonical witness: Membership pins the leaf to the public payload.

definition *canon-witness* :: 'v smt-op $\Rightarrow$ ('v leaf-state $\times$ 'd list) $\Rightarrow$ ('v leaf-state $\times$ 'd list) **where**
canon-witness op w = (case op of
 Membership k v $\Rightarrow$ (Occupied v, snd w)
 | - $\Rightarrow$ w)

The canonical value digest for a semantically valid witness.

definition *wit-vd* :: 'v smt-op $\Rightarrow$ ('v leaf-state $\times$ 'd list) $\Rightarrow$ 'd **where**
wit-vd op w = (case op of
 Membership k v $\Rightarrow$ h-leaf (Occupied v)
 | NonMembership k $\Rightarrow$ h-leaf (fst w)
 | Update k old new $\Rightarrow$ h-leaf new)

theorem *theorem-A-soundness*:

assumes acc: *verifier-accept* op root root' vd w
shows smt-valid op root root' (canon-witness op w)

proof (cases op)

case (Membership k v)

obtain leaf sibs **where** w: w = (leaf, sibs) **by** (cases w)

from acc **have** pub: *pub-ok* op root root' vd

and lsib: length sibs = dd

and circ: *circuit-accept* (compile KMembership)

(enc-ro KMembership leaf k sibs root vd)

by (auto simp: *verifier-accept-def* Membership w *encode-witness-def*)

have cons-ro KMembership (enc-ro KMembership leaf k sibs root vd)

using A-i-ro[of KMembership] circ **by** (simp add: length-enc-ro)

then have path: path-root (h-leaf leaf) k sibs = root

and vdl: vd = h-leaf leaf

using cons-ro-enc[OF - lsib] **by** auto

from pub **have** root-eq: root' = root **and** key-rng: k < 2 $\wedge$ dd

and vdc: vd = h-leaf (Occupied v)

by (auto simp: *pub-ok-def* *vd-ok-def* Membership)

have path-root (h-leaf (Occupied v)) k sibs = root

using path vdl vdc **by** simp

then have path-ok (Occupied v) k sibs root

```

    using lsib key-rng by (simp add: path-ok-def)
  then show ?thesis
    by (simp add: Membership w canon-witness-def root-eq)
next
case (NonMembership k)
obtain leaf sibs where w: w = (leaf, sibs) by (cases w)
from acc have pub: pub-ok op root root' vd
  and lsib: length sibs = dd
  and circ: circuit-accept (compile KNonMembership)
    (enc-ro KNonMembership leaf k sibs root vd)
  by (auto simp: verifier-accept-def NonMembership w encode-witness-def)
have cons-ro KNonMembership (enc-ro KNonMembership leaf k sibs root vd)
  using A-i-ro[of KNonMembership] circ by (simp add: length-enc-ro)
then have path: path-root (h-leaf leaf) k sibs = root
  and tags: leaf = Empty ∨ leaf = Tombstone
  using cons-ro-enc[OF - lsib] by auto
from pub have root-eq: root' = root and key-rng: k < 2 ^ dd
  by (auto simp: pub-ok-def NonMembership)
have path-ok leaf k sibs root
  using path lsib key-rng by (simp add: path-ok-def)
then show ?thesis
  by (simp add: NonMembership w canon-witness-def root-eq tags)
next
case (Update k old new)
obtain leaf sibs where w: w = (leaf, sibs) by (cases w)
from acc have pub: pub-ok op root root' vd
  and lsib: length sibs = dd
  and echo: leaf = old
  and circ: circuit-accept (compile KUpdate)
    (enc-up old new k sibs root root' vd)
  by (auto simp: verifier-accept-def Update w encode-witness-def echo-ok-def)
have cons-up (enc-up old new k sibs root root' vd)
  using A-i-up circ by (simp add: length-enc-up)
then have path1: path-root (h-leaf old) k sibs = root
  and path2: path-root (h-leaf new) k sibs = root'
  using cons-up-enc[OF lsib] by auto
from pub have key-rng: k < 2 ^ dd by (simp add: pub-ok-def Update)
have path-ok old k sibs root and path-ok new k sibs root'
  using path1 path2 lsib key-rng by (simp-all add: path-ok-def)
then show ?thesis
  by (simp add: Update w canon-witness-def echo)
qed

```

theorem *theorem-A-completeness:*

assumes *valid: smt-valid op root root' w*

shows *verifier-accept op root root' (wit-vd op w) w*

proof (*cases op*)

case (*Membership k v*)

obtain *leaf sibs* where *w: w = (leaf, sibs)* by (*cases w*)

from *valid* have *root-eq: root' = root* and *leaf-eq: leaf = Occupied v*

and *pok: path-ok leaf k sibs root*

by (*auto simp: Membership w*)

from *pok* have *lsib: length sibs = dd* and *key-rng: k < 2 ^ dd*

```

    and path: path-root (h-leaf leaf) k sibs = root
    by (auto simp: path-ok-def)
  have vd: wit-vd op w = h-leaf (Occupied v)
    by (simp add: wit-vd-def Membership)
  have cons-ro KMembership (enc-ro KMembership leaf k sibs root (wit-vd op w))
    using cons-ro-enc[OF - lsib] path leaf-eq vd by auto
  then have circuit-accept (compile KMembership)
    (enc-ro KMembership leaf k sibs root (wit-vd op w))
    using A-i-ro[of KMembership] by (simp add: length-enc-ro)
  then show ?thesis
    by (simp add: verifier-accept-def pub-ok-def vd-ok-def echo-ok-def
        encode-witness-def wit-vd-def Membership w root-eq key-rng lsib)
next
case (NonMembership k)
obtain leaf sibs where w: w = (leaf, sibs) by (cases w)
from valid have root-eq: root' = root and tags: leaf = Empty  $\vee$  leaf = Tombstone
  and pok: path-ok leaf k sibs root
  by (auto simp: NonMembership w)
from pok have lsib: length sibs = dd and key-rng: k < 2 ^ dd
  and path: path-root (h-leaf leaf) k sibs = root
  by (auto simp: path-ok-def)
have vd: wit-vd op w = h-leaf leaf
  by (simp add: wit-vd-def NonMembership w)
have cons-ro KNonMembership (enc-ro KNonMembership leaf k sibs root (wit-vd op w))
  using cons-ro-enc[OF - lsib] path tags vd by auto
then have circuit-accept (compile KNonMembership)
  (enc-ro KNonMembership leaf k sibs root (wit-vd op w))
  using A-i-ro[of KNonMembership] by (simp add: length-enc-ro)
then show ?thesis
  using tags
  by (auto simp: verifier-accept-def pub-ok-def vd-ok-def echo-ok-def
      encode-witness-def wit-vd-def NonMembership w root-eq key-rng lsib)
next
case (Update k old new)
obtain leaf sibs where w: w = (leaf, sibs) by (cases w)
from valid have leaf-eq: leaf = old
  and pok1: path-ok old k sibs root and pok2: path-ok new k sibs root'
  by (auto simp: Update w)
from pok1 have lsib: length sibs = dd and key-rng: k < 2 ^ dd
  and path1: path-root (h-leaf old) k sibs = root
  by (auto simp: path-ok-def)
from pok2 have path2: path-root (h-leaf new) k sibs = root'
  by (auto simp: path-ok-def)
have vd: wit-vd op w = h-leaf new
  by (simp add: wit-vd-def Update)
have cons-up (enc-up old new k sibs root root' (wit-vd op w))
  using cons-up-enc[OF lsib] path1 path2 vd by auto
then have circuit-accept (compile KUpdate)
  (enc-up old new k sibs root root' (wit-vd op w))
  using A-i-up by (simp add: length-enc-up)
then show ?thesis
  by (simp add: verifier-accept-def pub-ok-def vd-ok-def echo-ok-def
      encode-witness-def wit-vd-def Update w key-rng lsib leaf-eq)

```

qed

Theorem A, packaged: the verifier accepts SOME (value digest, witness) for (op, root, root') iff the operation is semantically valid between those roots for SOME witness. This existential biconditional is the exact form the composition (Theorem C) consumes.

theorem *theorem-A-compilation-soundness*:

$(\exists vd\ w. \text{verifier-accept op root root'}\ vd\ w)$
 $\longleftrightarrow (\exists w. \text{smt-valid op root root'}\ w)$

proof

assume $\exists vd\ w. \text{verifier-accept op root root'}\ vd\ w$
then obtain $vd\ w$ **where** $\text{verifier-accept op root root'}\ vd\ w$ **by** *blast*
then have $\text{smt-valid op root root'}\ (canon-witness\ op\ w)$
by (*rule theorem-A-soundness*)
then show $\exists w. \text{smt-valid op root root'}\ w$ **by** *blast*

next

assume $\exists w. \text{smt-valid op root root'}\ w$
then obtain w **where** $\text{smt-valid op root root'}\ w$ **by** *blast*
then have $\text{verifier-accept op root root'}\ (wit-vd\ op\ w)\ w$
by (*rule theorem-A-completeness*)
then show $\exists vd\ w. \text{verifier-accept op root root'}\ vd\ w$ **by** *blast*

qed

end

end

theory *Composition*

imports *Compiler-Correctness GKR-Protocol.GKR-Assembly*

begin

23 The composition locale (v0.1 pipeline)

The compiler model and the GKR bridge locale over ONE challenge field: the compiler's wire field is the (finite) field the sumcheck challenges are drawn from. (Production splits base and challenge fields; the model identifies them - the extension-field split is an optimisation seam, not a semantic one.)

locale *statesync-gkr-v01* =

compiler-model params h-leaf h-node W L digest-repr leaf-repr hl hn stack +
gkr-layer-sumcheck vars deg eval inst

for *params* :: *smt-params*

and *h-leaf* :: $'v\ \text{leaf-state} \Rightarrow 'd$
and *h-node* :: $'d \Rightarrow 'd \Rightarrow 'd$
and *W L* :: *nat*
and *digest-repr* :: $'d \Rightarrow 'a::\{\text{finite}, \text{field}\}\ \text{list}$
and *leaf-repr* :: $'v\ \text{leaf-state} \Rightarrow 'a\ \text{list}$
and *hl* :: $'a\ \text{list} \Rightarrow 'a\ \text{list}$
and *hn* :: $'a\ \text{list} \Rightarrow 'a\ \text{list} \Rightarrow 'a\ \text{list}$
and *stack* :: *op-kind* $\Rightarrow 'a\ \text{layer list}$
and *vars* :: $'p::\text{comm-monoid-add} \Rightarrow \text{nat set}$
and *deg* :: $'p \Rightarrow \text{nat}$

```

and eval :: 'p ⇒ (nat, 'a) subst ⇒ 'a
and inst :: 'p ⇒ (nat, 'a) subst ⇒ 'p
begin

```

The compiled circuit's layer list and its value chain on the honest witness encoding.

```

lemma compile-layers:
  circ-layers (compile k) = mk-res-layer (descs k) # stack k
by (simp add: compile-def)

```

```

lemma compile-output-width:
  layer-width-bits (mk-res-layer (descs k)) = length (descs k)
by (simp add: mk-res-layer-def)

```

24 Theorem C

theorem theorem-C-composition:

```

fixes P :: 'a layer ⇒ nat ⇒ ('a list × 'a) list ⇒ 'a list ⇒ 'p
and A :: nat ⇒ 'a list ⇒ ('p, 'a, 'a, nat, 's) prover × 's × 'a × 'a × 'a
and w :: 'v leaf-state × 'd list
and root root' vd :: 'd

```

— explicit: the import chain brings HOL-Analysis' *root* constant into scope, which would otherwise capture the variable

```

and dbnd :: nat ⇒ nat

```

— width-dependent representative degree bound: dischargeable with $\lambda s. 2 * s$ in the AFP total-degree mpoly instance

```

assumes bels: bels = tl (circuit-values (mk-res-layer (descs (kind-of op)) # stack (kind-of op))
  (encode-witness op root root' vd w))
and wf: gkr-chain-wf (s0 # ss) (mk-res-layer (descs (kind-of op)) # stack (kind-of op)) bels
and repr: ⋀ Ly s claims b. layer-poly-repr (P Ly s claims b) Ly s claims b (dbnd s)
and pub: pub-ok op root root' vd
and wlen: length (snd w) = depth params
and echo: echo-ok op w
and nosemt: ¬ (∃ w'. smt-valid op root root' w')

```

shows measure-pmf.prob

```

  (pmf-of-set (tuples (UNIV :: 'a set)
    (length (descs (kind-of op)) + chain-rlen (s0 # ss))))
  {rs. gkr-chain-bad P (λj pfx. A j (take (length (descs (kind-of op))) rs @ pfx))
    (s0 # ss) (mk-res-layer (descs (kind-of op)) # stack (kind-of op)) bels
    [(take (length (descs (kind-of op))) rs, 1)]
    (mle (length (descs (kind-of op))) (λ-. 0)
      (take (length (descs (kind-of op))) rs)
      - const-mle (mk-res-layer (descs (kind-of op)))
        (take (length (descs (kind-of op))) rs)
      (drop (length (descs (kind-of op))) rs))}
  ≤ (real (length (descs (kind-of op)))
    + (∑ s←s0 # ss. real (2 * s) * real (dbnd s) + 1)) / real CARD('a)

```

proof —

```

let ?k = kind-of op
let ?L0 = mk-res-layer (descs ?k) :: 'a layer

```

— type pinned: *layer-width-bits* ?L0 occurrences do not determine the layer's wire field, so an unannotated abbreviation would give each standalone occurrence a fresh type variable (the type-binding pathology this corpus guards against)

let $?enc = \text{encode-witness } op \text{ root root' } vd \ w$
let $?cvs = \text{circuit-values } (?L0 \ \# \ \text{stack } ?k) \ ?enc$

— Theorem A, contrapositive: semantic invalidity kills circuit acceptance

have $\text{not-accept}: \neg \text{circuit-accept } (\text{compile } ?k) \ ?enc$

proof

assume $\text{circuit-accept } (\text{compile } ?k) \ ?enc$
then have $\text{verifier-accept } op \text{ root root' } vd \ w$
using $\text{pub when echo by } (\text{simp add: verifier-accept-def})$
then have $\text{smt-valid } op \text{ root root' } (\text{canon-witness } op \ w)$
by $(\text{rule theorem-A-soundness})$
with $\text{nosem show False by blast}$

qed

— the below vector of the residual layer, and the true output table

obtain $b0 \ bs$ **where** $\text{bels-cons}: \text{bels} = b0 \ \# \ bs$

using $\text{wf by } (\text{cases bels}) \ \text{auto}$

have $\text{tl-cvs}: \text{tl } ?cvs = \text{circuit-values } (\text{stack } ?k) \ ?enc$

by $(\text{simp add: Let-def})$

have $\text{hd-cvs}: \text{hd } ?cvs = \text{layer-eval } ?L0 \ (\text{hd } (\text{circuit-values } (\text{stack } ?k) \ ?enc))$

by $(\text{simp add: Let-def})$

have $\text{b0-hd}: b0 = \text{hd } (\text{circuit-values } (\text{stack } ?k) \ ?enc)$

proof —

have $b0 \ \# \ bs = \text{bels}$ **by** $(\text{rule bels-cons[symmetric]})$
also have $\dots = \text{tl } ?cvs$ **by** (rule bels)
also have $\dots = \text{circuit-values } (\text{stack } ?k) \ ?enc$ **by** (rule tl-cvs)
finally have $\text{hd } (b0 \ \# \ bs) = \text{hd } (\text{circuit-values } (\text{stack } ?k) \ ?enc)$
by (rule arg-cong)
then show $?thesis$ **by** simp

qed

have $\text{out-eq}: \text{circuit-output } (\text{compile } ?k) \ ?enc = \text{layer-eval } ?L0 \ b0$

proof —

have $\text{circuit-output } (\text{compile } ?k) \ ?enc = \text{hd } ?cvs$
unfolding $\text{circuit-output-def compile-layers}$ **by** (rule refl)
also have $\dots = \text{layer-eval } ?L0 \ (\text{hd } (\text{circuit-values } (\text{stack } ?k) \ ?enc))$
by (rule hd-cvs)
also have $\dots = \text{layer-eval } ?L0 \ b0$
by $(\text{simp add: b0-hd[symmetric]})$
finally show $?thesis$.

qed

— the all-zero claimed table differs from the true output somewhere

have $\text{outs-ne}: \exists \text{idx} < 2 \wedge \text{layer-width-bits } ?L0. (\lambda \cdot. 0 :: 'a) \ \text{idx} \neq \text{layer-eval } ?L0 \ b0 \ ! \ \text{idx}$

proof —

from not-accept **obtain** x **where** $x\text{-in}: x \in \text{set } (\text{circuit-output } (\text{compile } ?k) \ ?enc)$
and $x\text{-ne}: x \neq 0$
by $(\text{auto simp add: circuit-accept-def})$
from $x\text{-in}$ **obtain** idx **where** $\text{idx-lt}: \text{idx} < \text{length } (\text{circuit-output } (\text{compile } ?k) \ ?enc)$
and $\text{idx-eq}: \text{circuit-output } (\text{compile } ?k) \ ?enc \ ! \ \text{idx} = x$
by $(\text{auto simp add: in-set-conv-nth})$
have $\text{length } (\text{circuit-output } (\text{compile } ?k) \ ?enc) = 2 \wedge \text{layer-width-bits } ?L0$
by $(\text{simp add: out-eq layer-width-def})$
then show $?thesis$

```

    using idx-lt idx-eq x-ne by (auto simp add: out-eq)
qed

have measure-pmf.prob
  (pmf-of-set (tuples (UNIV :: 'a set)
    (layer-width-bits ?L0 + chain-rlen (s0 # ss))))
  {rs. gkr-chain-bad P (λj pfx. A j (take (layer-width-bits ?L0) rs @ pfx))
    (s0 # ss) (?L0 # stack ?k) (b0 # bs)
    [(take (layer-width-bits ?L0) rs, 1)]
    (mle (layer-width-bits ?L0) (λ-. 0) (take (layer-width-bits ?L0) rs)
      - const-mle ?L0 (take (layer-width-bits ?L0) rs))
    (drop (layer-width-bits ?L0) rs)}
  ≤ (real (layer-width-bits ?L0)
    + (∑ s←s0 # ss. real (2 * s) * real (dbnd s) + 1)) / real CARD('a)
  using wf bels-cons
  by (intro gkr-assembly-soundness[OF - repr outs-ne]) simp
then show ?thesis
  by (simp add: bels-cons compile-output-width)
qed

```

The design-bound specialisation (reduce.rs $LAYER-ROUND-DEGREE = 4$): the composed pipeline's soundness error is $(|residuals| + \text{sum over layers of } (8s_i + 1)) / |a|$.

corollary *theorem-C-composition-deg4*:

```

fixes P :: 'a layer ⇒ nat ⇒ ('a list × 'a) list ⇒ 'a list ⇒ 'p
and A :: nat ⇒ 'a list ⇒ ('p, 'a, 'a, nat, 's) prover × 's × 'a × 'a × 'a
and w :: 'v leaf-state × 'd list
and root root' vd :: 'd
assumes bels = tl (circuit-values (mk-res-layer (descs (kind-of op)) # stack (kind-of op))
  (encode-witness op root root' vd w))
and gkr-chain-wf (s0 # ss) (mk-res-layer (descs (kind-of op)) # stack (kind-of op)) bels
and ∧ Ly s claims b. layer-poly-repr (P Ly s claims b) Ly s claims b 4
and pub-ok op root root' vd
and length (snd w) = depth params
and echo-ok op w
and ¬ (∃ w'. smt-valid op root root' w')

```

shows *measure-pmf.prob*

```

  (pmf-of-set (tuples (UNIV :: 'a set)
    (length (descs (kind-of op)) + chain-rlen (s0 # ss))))
  {rs. gkr-chain-bad P (λj pfx. A j (take (length (descs (kind-of op))) rs @ pfx))
    (s0 # ss) (mk-res-layer (descs (kind-of op)) # stack (kind-of op)) bels
    [(take (length (descs (kind-of op))) rs, 1)]
    (mle (length (descs (kind-of op))) (λ-. 0)
      (take (length (descs (kind-of op))) rs)
      - const-mle (mk-res-layer (descs (kind-of op)))
        (take (length (descs (kind-of op))) rs))
    (drop (length (descs (kind-of op))) rs)}
  ≤ (real (length (descs (kind-of op)))
    + (∑ s←s0 # ss. real (2 * s) * 4 + 1)) / real CARD('a)

```

proof –

```

  have r: ∧ Ly s claims b. layer-poly-repr (P Ly s claims b) Ly s claims b ((λ-. 4) s)
  using assms(3) by simp
  show ?thesis
  using theorem-C-composition[OF assms(1,2) r assms(4–7)] by simp

```

qed

end

end

theory *KoalaBear-Field*

imports

HOL-Computational-Algebra.Primes

HOL-Library.Code-Target-Numeral

HOL-Library.Cardinality

begin

— *Code-Target-Numeral* maps nat/int arithmetic to native ML integers for the primality sweep evaluation below; the plain HOL code equations compute *mod* by REPEATED SUBTRACTION, which makes the 46k-divisor sweep over a 31-bit prime computationally infeasible (observed).

25 The KoalaBear prime

definition *kb-p* :: *int* **where**

kb-p = 2130706433 − 2³¹ − 2²⁴ + 1

lemma *kb-p-val*: *kb-p* = 2³¹ − 2²⁴ + 1

by (*simp add: kb-p-def*)

lemma *kb-p-pos*: 0 < *kb-p*

by (*simp add: kb-p-def*)

lemma *kb-p-gt1*: 1 < *kb-p*

by (*simp add: kb-p-def*)

Primality by sqrt-bounded trial division: any composite $p = a * b$ with $1 < a \leq b$ has $a * a \leq p$, so checking divisors up to $\lfloor \sqrt{p} \rfloor = 46159$ suffices. The finite divisor sweep is discharged by evaluation.

lemma *nat-prime-sqrt-criterion*:

fixes *p* :: *nat*

assumes *gt*: 1 < *p*

and *nd*: $\bigwedge n. 2 \leq n \implies n * n \leq p \implies \neg n \text{ dvd } p$

shows *prime p*

proof (*rule ccontr*)

assume $\neg \text{prime } p$

then obtain *a* **where** *a-dvd*: *a* dvd *p* **and** *a-ne1*: *a* ≠ 1 **and** *a-nep*: *a* ≠ *p*

using *gt* **by** (*auto simp add: prime-nat-iff*)

from *a-dvd* **obtain** *b* **where** *p-ab*: *p* = *a* * *b* ..

have *a-pos*: 0 < *a*

proof (*rule ccontr*)

assume $\neg 0 < a$

then have *a* = 0 **by** *simp*

with *p-ab* *gt* **show** *False* **by** *simp*

qed

have *b-pos*: 0 < *b*

proof (*rule ccontr*)

```

    assume  $\neg 0 < b$ 
    then have  $b = 0$  by simp
    with  $p\text{-}ab$  gt show False by simp
qed
have  $a2: 2 \leq a$  using  $a\text{-}pos$   $a\text{-}ne1$  by simp
have  $b\text{-}ne1: b \neq 1$  using  $p\text{-}ab$   $a\text{-}nep$  by auto
have  $b2: 2 \leq b$  using  $b\text{-}pos$   $b\text{-}ne1$  by simp
show False
proof (cases  $a \leq b$ )
  case True
    then have  $a * a \leq p$  using  $p\text{-}ab$   $a\text{-}pos$  by (simp add: mult-le-mono2)
    with  $nd[OF a2 this]$   $a\text{-}dvd$  show False by simp
  next
    case False
    then have  $b * b \leq p$  using  $p\text{-}ab$   $b\text{-}pos$ 
      by (simp add: mult.commute mult-le-mono2)
    moreover have  $b \text{ dvd } p$  using  $p\text{-}ab$  by auto
    ultimately show False using  $nd[OF b2]$  by simp
qed
qed

lemma  $kb\text{-}p\text{-}prime\text{-}nat$ : prime (2130706433 :: nat)
proof (rule nat-prime-sqrt-criterion)
  show  $1 < (2130706433 :: nat)$  by simp
next
  fix  $n :: nat$  assume  $n2: 2 \leq n$  and  $nsq: n * n \leq 2130706433$ 
  have  $nb: n \leq 46159$ 
  proof (rule ccontr)
    assume  $\neg n \leq 46159$ 
    then have  $46160 \leq n$  by simp
    then have  $(46160 :: nat) * 46160 \leq n * n$ 
      by (intro mult-le-mono) simp-all
    with  $nsq$  show False by simp
  qed
  have sweep:  $\forall m \in \{2..46159 :: nat\}. \neg m \text{ dvd } 2130706433$  by eval
  have  $n \in \{2..46159 :: nat\}$  using  $n2$   $nb$  by simp
  then show  $\neg n \text{ dvd } 2130706433$ 
    by (rule bspec[OF sweep])
    — explicit bspec: feeding the 46k-element ball fact to simp/auto invites an interval-expansion
term blow-up
qed

lemma  $kb\text{-}p\text{-}prime$ : prime  $kb\text{-}p$ 
  — via the nat-projection form, fully by resolution: handing a prime (numeral ...) goal to simp
  invites a divergent search (observed), and the int n-pattern transfer rule never fires on a numeral goal
  (the embedding normalises away)
  unfolding  $kb\text{-}p\text{-}def$  prime-int-nat-transfer nat-numeral
  by (intro conjI  $kb\text{-}p\text{-}prime\text{-}nat$ ) simp

```

26 The field type

```

typedef koala-bear = {0.. $kb\text{-}p$ }

```

```

morphisms kb-val KB
using kb-p-pos by (intro exI[of - 0]) simp

setup-lifting type-definition-koala-bear

instantiation koala-bear :: comm-ring-1
begin

lift-definition zero-koala-bear :: koala-bear is 0
using kb-p-pos by simp

lift-definition one-koala-bear :: koala-bear is 1
using kb-p-gt1 by simp

lift-definition plus-koala-bear :: koala-bear  $\Rightarrow$  koala-bear  $\Rightarrow$  koala-bear is
 $\lambda x y. (x + y) \bmod kb-p$ 
using kb-p-pos by simp

lift-definition uminus-koala-bear :: koala-bear  $\Rightarrow$  koala-bear is
 $\lambda x. (- x) \bmod kb-p$ 
using kb-p-pos by simp

lift-definition minus-koala-bear :: koala-bear  $\Rightarrow$  koala-bear  $\Rightarrow$  koala-bear is
 $\lambda x y. (x - y) \bmod kb-p$ 
using kb-p-pos by simp

lift-definition times-koala-bear :: koala-bear  $\Rightarrow$  koala-bear  $\Rightarrow$  koala-bear is
 $\lambda x y. (x * y) \bmod kb-p$ 
using kb-p-pos by simp

instance
proof
  — every goal is discharged by a DETERMINISTIC minimal rewrite sequence: the blanket
  mod-simps + algebra-simps combination diverges on these goals (observed: multi-GB simp blow-up),
  the environment-sensitive-simp trap this corpus guards against
  fix a b c :: koala-bear
  show  $a + b + c = a + (b + c)$ 
    by transfer (simp only: mod-add-left-eq mod-add-right-eq add.assoc)
  show  $a + b = b + a$ 
    by transfer (simp only: add.commute)
  show  $0 + a = a$ 
    by transfer (simp add: mod-pos-pos-trivial)
  show  $- a + a = 0$ 
    by (transfer, simp only: mod-add-left-eq) simp
  show  $a - b = a + - b$ 
    by (transfer, simp only: mod-add-right-eq) simp
  show  $a * b * c = a * (b * c)$ 
    by transfer (simp only: mod-mult-left-eq mod-mult-right-eq mult.assoc)
  show  $a * b = b * a$ 
    by transfer (simp only: mult.commute)
  show  $1 * a = a$ 
    by transfer (simp add: mod-pos-pos-trivial)
  show  $(a + b) * c = a * c + b * c$ 

```

```

    by transfer
      (simp only: mod-mult-left-eq mod-add-left-eq mod-add-right-eq
        distrib-right)
  show (0 :: koala-bear) ≠ 1
    by transfer (simp add: kb-p-def)
qed

end

lemma UNIV-koala-bear: (UNIV :: koala-bear set) = KB ‘ {0.. $kb-p$ }
proof
  show (UNIV :: koala-bear set) ⊆ KB ‘ {0.. $kb-p$ }
  proof
    fix z :: koala-bear
    have z = KB (kb-val z) by (simp add: kb-val-inverse)
    moreover have kb-val z ∈ {0.. $kb-p$ } by (rule kb-val)
    ultimately show z ∈ KB ‘ {0.. $kb-p$ } by blast
  qed
qed simp

instance koala-bear :: finite
proof
  show finite (UNIV :: koala-bear set)
    unfolding UNIV-koala-bear by simp
qed

lemma CARD-koala-bear: CARD(koala-bear) = 2130706433
proof -
  have inj: inj-on KB {0.. $kb-p$ }
  proof (rule inj-onI)
    fix a b assume a: a ∈ {0.. $kb-p$ } and b: b ∈ {0.. $kb-p$ }
    and eq: KB a = KB b
    have kb-val (KB a) = kb-val (KB b) using eq by simp
    with a b show a = b by (simp add: KB-inverse)
  qed
  have CARD(koala-bear) = card (KB ‘ {0.. $kb-p$ })
    by (simp add: UNIV-koala-bear)
  also have ... = card {0.. $kb-p$ }
    by (rule card-image[OF inj])
  finally show ?thesis by (simp add: kb-p-def)
qed

```

27 Field structure (finite integral domain argument)

```

lemma kb-not-dvd:
  assumes x: x ∈ {0.. $kb-p$ } and ne: x ≠ 0
  shows ¬ kb-p dvd x
proof
  assume dvd: kb-p dvd x
  have |kb-p| ≤ |x|
    by (rule dvd-imp-le-int[OF ne dvd])
  with x kb-p-pos show False by auto

```

qed

lemma *kb-mult-inj-on*:

assumes $x: x \in \{0..<kb-p\}$ **and** $ne: x \neq 0$

shows *inj-on* $(\lambda y. (x * y) \bmod kb-p)$ $\{0..<kb-p\}$

proof (*rule inj-onI*)

fix $y1\ y2$

assume $y1: y1 \in \{0..<kb-p\}$ **and** $y2: y2 \in \{0..<kb-p\}$

and $eq: (x * y1) \bmod kb-p = (x * y2) \bmod kb-p$

have $kb-p \text{ dvd } (x * y1 - x * y2)$

using eq **by** (*simp add: mod-eq-dvd-iff*)

then have $kb-p \text{ dvd } x * (y1 - y2)$

by (*simp add: algebra-simps*)

then have $kb-p \text{ dvd } x \vee kb-p \text{ dvd } (y1 - y2)$

using $kb-p\text{-prime}$ **by** (*simp add: prime-dvd-mult-iff*)

then have $dvd\text{-diff}: kb-p \text{ dvd } (y1 - y2)$

using $kb\text{-not-dvd}[OF\ x\ ne]$ **by** *blast*

have $y1 - y2 = 0$

proof (*rule ccontr*)

assume $ne\text{-diff}: y1 - y2 \neq 0$

have $|kb-p| \leq |y1 - y2|$

by (*rule dvd-imp-le-int[OF ne-diff dvd-diff]*)

moreover have $|y1 - y2| < kb-p$ **using** $y1\ y2$ **by** *auto*

ultimately show *False* **using** $kb-p\text{-pos}$ **by** *simp*

qed

then show $y1 = y2$ **by** *simp*

qed

lemma *kb-inverse-exists*:

assumes $x: x \in \{0..<kb-p\}$ **and** $ne: x \neq 0$

shows $\exists y \in \{0..<kb-p\}. (x * y) \bmod kb-p = 1$

proof —

have *surj*: $(\lambda y. (x * y) \bmod kb-p)$ ‘ $\{0..<kb-p\} = \{0..<kb-p\}$

proof (*rule endo-inj-surj*)

show *finite* $\{0..<kb-p\}$ **by** *simp*

show $(\lambda y. (x * y) \bmod kb-p)$ ‘ $\{0..<kb-p\} \subseteq \{0..<kb-p\}$

using $kb-p\text{-pos}$ **by** *auto*

show *inj-on* $(\lambda y. (x * y) \bmod kb-p)$ $\{0..<kb-p\}$

by (*rule kb-mult-inj-on[OF x ne]*)

qed

have $(1 :: int) \in (\lambda y. (x * y) \bmod kb-p)$ ‘ $\{0..<kb-p\}$

using *surj kb-p-gt1* **by** *simp*

then obtain y **where** $y: y \in \{0..<kb-p\}$ **and** $eq1: 1 = (x * y) \bmod kb-p$

by (*rule imageE*)

— the orientation is fixed by an explicit [*symmetric*]: *intro sym* REPEATS its rule, and *sym* is its own inverse, so it recurses until the ML stack dies ("Unable to increase stack") - and the resulting interrupt cascade then hides the real errors in neighbouring proofs

from $y\ eq1$ [*symmetric*] **show** *?thesis* **by** *blast*

qed

instantiation *koala-bear* :: *inverse*

— the class PARAMETERS (*inverse*, *divide*) are introduced here and the field AXIOMS are discharged by a separate *instance* below: the / notation is an abbreviation of class *inverse*, so it only

elaborates against a REGISTERED arity - inside an *instantiation* ... :: *field* block it fails with *No type arity koala-bear :: inverse*
begin

The inverse is a PLAIN definition through the representation (not a *lift-definition*: the lifting/code machinery diverges on the Hilbert-choice body - observed), with the field axioms proved at the representation level by injectivity of *kb-val*.

definition *inverse-koala-bear* :: *koala-bear* $\Rightarrow$ *koala-bear* **where**

inverse-koala-bear a =
 KB (if *kb-val* a = 0 then 0
 else (SOME y. y $\in$ {0..*kb-p*} $\wedge$ (*kb-val* a * y) mod *kb-p* = 1))

definition *divide-koala-bear* :: *koala-bear* $\Rightarrow$ *koala-bear* $\Rightarrow$ *koala-bear* **where**

divide-koala-bear x y = x * *inverse* y

instance ..

end

instance *koala-bear* :: *field*

proof

fix a b :: *koala-bear*

show a $\neq$ 0 $\implies$ *inverse* a * a = 1

proof -

assume ne: a $\neq$ 0

have vne: *kb-val* a $\neq$ 0

proof

assume *kb-val* a = 0

then have *kb-val* a = *kb-val* (0 :: *koala-bear*)

by (*simp add: zero-koala-bear.rep-eq*)

then have a = 0 **by** (*simp add: kb-val-inject*)

with ne **show** False ..

qed

define *invv* **where**

invv = (SOME y. y $\in$ {0..*kb-p*} $\wedge$ (*kb-val* a * y) mod *kb-p* = 1)

from *kb-inverse-exists*[OF *kb-val* vne] **obtain** y

where y $\in$ {0..*kb-p*} $\wedge$ (*kb-val* a * y) mod *kb-p* = 1 **by** *blast*

then have someP: *invv* $\in$ {0..*kb-p*} $\wedge$ (*kb-val* a * *invv*) mod *kb-p* = 1

unfolding *invv-def* **by** (*rule someI*)

have *inv-a*: *inverse* a = KB *invv*

unfolding *inverse-koala-bear-def invv-def*

by (*rule arg-cong[of - - KB]*) (*rule if-not-P[OF vne]*)

have *rep-inv*: *kb-val* (*inverse* a) = *invv*

unfolding *inv-a*

by (*rule KB-inverse*) (*rule conjunct1[OF someP]*)

have *kb-val* (*inverse* a * a) = (*invv* * *kb-val* a) mod *kb-p*

by (*simp add: times-koala-bear.rep-eq rep-inv*)

also have ... = (*kb-val* a * *invv*) mod *kb-p*

by (*simp only: mult.commute*)

also have ... = 1

by (*rule conjunct2[OF someP]*)

also have (1 :: *int*) = *kb-val* (1 :: *koala-bear*)

```

    by (simp add: one-koala-bear.rep-eq)
  finally show inverse a * a = 1
    by (simp add: kb-val-inject)
qed
show a / b = a * inverse b
  by (simp add: divide-koala-bear-def)
show inverse (0 :: koala-bear) = 0
proof -
  have inverse (0 :: koala-bear) = KB 0
    unfolding inverse-koala-bear-def
    by (rule arg-cong[of - - KB]) (simp add: zero-koala-bear.rep-eq)
  also have ... = (0 :: koala-bear)
  proof -
    have kb-val (KB 0) = kb-val (0 :: koala-bear)
      using kb-p-pos
    by (simp add: KB-inverse zero-koala-bear.rep-eq)
    then show ?thesis by (simp add: kb-val-inject)
  qed
  finally show ?thesis .
qed
qed

```

28 Concrete facts the instantiations consume

The tag values 0/1/2 are pairwise distinct (odd characteristic).

lemma *kb-two-val*: $kb\text{-val } (2 :: koala\text{-bear}) = 2$

— the rewrite must reach *rep-eq* BEFORE the numeral rules fold $1 + 1$ back into 2 , hence the *simp only* stage and the *unfolding* (not *simp only*) transfer of the folded form

```

proof -
  have two: (2 :: koala-bear) = 1 + 1 by simp
  have kb-val ((1 :: koala-bear) + 1) = (1 + 1) mod kb-p
    by (simp only: plus-koala-bear.rep-eq one-koala-bear.rep-eq)
  also have ... = 2
    by (simp add: kb-p-def)
  finally show ?thesis unfolding two .
qed

```

lemma *kb-two-ne-zero*: $(2 :: koala\text{-bear}) \neq 0$

```

proof
  assume (2 :: koala-bear) = 0
  then have kb-val (2 :: koala-bear) = kb-val (0 :: koala-bear) by simp
  then show False by (simp add: kb-two-val zero-koala-bear.rep-eq)
qed

```

lemma *kb-two-ne-one*: $(2 :: koala\text{-bear}) \neq 1$

```

proof
  assume (2 :: koala-bear) = 1
  then have kb-val (2 :: koala-bear) = kb-val (1 :: koala-bear) by simp
  then show False by (simp add: kb-two-val one-koala-bear.rep-eq)
qed

```

of-nat is the mod-p embedding: the ground fact for discharging the *of-nat*-injectivity-on-bound

premises of lemma (b) in this concrete field.

```

lemma kb-val-of-nat: kb-val (of-nat n) = int n mod kb-p
proof (induction n)
  case 0 then show ?case using kb-p-pos by (simp add: zero-koala-bear.rep-eq)
next
  case (Suc n)
  have kb-val (of-nat (Suc n)) = (1 + kb-val (of-nat n)) mod kb-p
    by (simp add: plus-koala-bear.rep-eq one-koala-bear.rep-eq kb-p-gt1)
  also have ... = (1 + int n mod kb-p) mod kb-p by (simp add: Suc.IH)
  also have ... = int (Suc n) mod kb-p
    by (simp only: mod-add-right-eq) simp
  finally show ?case .
qed

```

```

lemma kb-of-nat-inj-on-bound:
  assumes m: m < 2130706433 and n: n < 2130706433
  and eq: (of-nat m :: koala-bear) = of-nat n
  shows m = n
proof -
  have vm: kb-val (of-nat m :: koala-bear) = kb-val (of-nat n :: koala-bear)
    using eq by simp
  have im: int m < kb-p using m unfolding kb-p-def by simp
  have inn: int n < kb-p using n unfolding kb-p-def by simp
  have m-triv: int m mod kb-p = int m
    by (rule mod-pos-pos-trivial) (simp-all add: im)
  have n-triv: int n mod kb-p = int n
    by (rule mod-pos-pos-trivial) (simp-all add: inn)
  have int m = int n
    using vm unfolding kb-val-of-nat m-triv n-triv .
  then show ?thesis by simp
qed

end

```

```

theory Compiler-Instance
  imports Compiler-Correctness KoalaBear-Field
begin

```

```

declare One-nat-def [simp del]
  — One-nat-def (1 = Suc 0) is a default simp rule, and it rewrites the NUMERAL ARGUMENTS of
  the interpreted locale constants: a goal about compiler-model-base.ev-tag kb-params 1 silently becomes
  one about ... kb-params (Suc 0), after which none of the kb-base.*-def equations - whose left-hand
  sides carry the literal 1 that the interpretation supplied for W - can ever match, and every layout
  lemma fails with an unsolved goal that LOOKS identical to its own definition. It is re-enabled locally
  where Suc-shaped list/index arithmetic needs it

```

29 The KoalaBear toy hash pair and representations

```

fun kb-pay :: koala-bear leaf-state  $\Rightarrow$  koala-bear where
  kb-pay Empty = 0
  | kb-pay (Occupied v) = v

```

| $kb\text{-}pay\ Tombstone = 0$

definition $kb\text{-}h\text{-}leaf :: koala\text{-}bear\ leaf\text{-}state \Rightarrow koala\text{-}bear$ **where**
 $kb\text{-}h\text{-}leaf\ lf = leaf\text{-}tag\text{-}val\ lf + 3 * kb\text{-}pay\ lf$

definition $kb\text{-}h\text{-}node :: koala\text{-}bear \Rightarrow koala\text{-}bear \Rightarrow koala\text{-}bear$ **where**
 $kb\text{-}h\text{-}node\ a\ b = a + 2 * b$

definition $kb\text{-}digest\text{-}repr :: koala\text{-}bear \Rightarrow koala\text{-}bear\ list$ **where**
 $kb\text{-}digest\text{-}repr\ d = [d]$

definition $kb\text{-}leaf\text{-}repr :: koala\text{-}bear\ leaf\text{-}state \Rightarrow koala\text{-}bear\ list$ **where**
 $kb\text{-}leaf\text{-}repr\ lf = [leaf\text{-}tag\text{-}val\ lf, kb\text{-}pay\ lf]$

definition $kb\text{-}hl :: koala\text{-}bear\ list \Rightarrow koala\text{-}bear\ list$ **where**
 $kb\text{-}hl\ xs = [xs ! 0 + 3 * xs ! 1]$

definition $kb\text{-}hn :: koala\text{-}bear\ list \Rightarrow koala\text{-}bear\ list \Rightarrow koala\text{-}bear\ list$ **where**
 $kb\text{-}hn\ a\ b = [a ! 0 + 2 * b ! 0]$

definition $kb\text{-}params :: smt\text{-}params$ **where**
 $kb\text{-}params = (\mid depth = 1, leaf\text{-}max\text{-}fields = 4 \mid)$

interpretation $kbc\text{-}base$: *compiler-model-base*
 $kb\text{-}params\ kb\text{-}h\text{-}leaf\ kb\text{-}h\text{-}node\ 1\ 2\ kb\text{-}digest\text{-}repr\ kb\text{-}leaf\text{-}repr\ kb\text{-}hl\ kb\text{-}hn$
by *unfold-locales*
 $(auto\ simp\ add: kb\text{-}digest\text{-}repr\text{-}def\ kb\text{-}leaf\text{-}repr\text{-}def\ kb\text{-}hl\text{-}def\ kb\text{-}hn\text{-}def$
 $kb\text{-}h\text{-}leaf\text{-}def\ kb\text{-}h\text{-}node\text{-}def)$

lemma $kbc\text{-}dd\ [simp]: depth\ kb\text{-}params = 1$
by $(simp\ add: kb\text{-}params\text{-}def)$

30 Quadratic wire descriptors and their gate layers

A wire is a linear combination of below-wires plus a linear combination of products of below-wire pairs - the exact shape of the instance's stack components (affine hashes over key-bit muxes).

type-synonym $'f\ qwire = ('f \times nat)\ list \times ('f \times nat \times nat)\ list$

definition $qw\text{-}sem :: 'f::comm\text{-}ring\ 1\ list \Rightarrow 'f\ qwire \Rightarrow 'f$ **where**
 $qw\text{-}sem\ below\ w =$
 $(\sum (c, i) \leftarrow fst\ w. c * below\ !\ i)$
 $+ (\sum (c, i, j) \leftarrow snd\ w. c * (below\ !\ i * below\ !\ j))$

definition $qw\text{-}gates :: nat \Rightarrow 'f::comm\text{-}ring\ 1\ qwire \Rightarrow 'f\ gate\ list$ **where**
 $qw\text{-}gates\ z\ w =$
 $map\ (\lambda(c, i). (\mid g\text{-}kind = GLin, g\text{-}out = z, g\text{-}in1 = i, g\text{-}in2 = i, g\text{-}coeff = c \mid))\ (fst\ w)$
 $@\ map\ (\lambda(c, i, j). (\mid g\text{-}kind = GMul, g\text{-}out = z, g\text{-}in1 = i, g\text{-}in2 = j, g\text{-}coeff = c \mid))\ (snd\ w)$

definition $mk\text{-}q\text{-}layer :: 'f::comm\text{-}ring\ 1\ qwire\ list \Rightarrow 'f\ layer$ **where**
 $mk\text{-}q\text{-}layer\ ws =$

```

(| layer-width-bits = length ws,
   layer-gates = concat (map (λ(z, w). qw-gates z w) (List.enumerate 0 ws)),
   layer-consts = [] |)

```

lemma *qw-gates-out*: $g \in \text{set } (qw\text{-gates } z \ w) \implies g\text{-out } g = z$
by (auto simp add: qw-gates-def)

lemma *filter-qw-gates-same*:
 $\text{filter } (\lambda g. g\text{-out } g = z) (qw\text{-gates } z \ w) = qw\text{-gates } z \ w$
by (auto simp add: filter-id-conv dest: qw-gates-out)

lemma *filter-qw-gates-other*:
 $k \neq z \implies \text{filter } (\lambda g. g\text{-out } g = z) (qw\text{-gates } k \ w) = []$
by (auto simp add: filter-empty-conv dest: qw-gates-out)

lemma *filter-enumerate-qw-gates-miss*:
assumes $z < n \vee n + \text{length } ws \leq z$
shows $\text{filter } (\lambda g. g\text{-out } g = z)$
 $(\text{concat } (\text{map } (\lambda(k, w). qw\text{-gates } k \ w) (\text{List.enumerate } n \ ws))) = []$
using *assms*
proof (induction ws arbitrary: n)
case Nil **then show** ?case **by** simp
next
case (Cons w ws)
then have $z \neq n$ **and** $\text{rest}: z < \text{Suc } n \vee \text{Suc } n + \text{length } ws \leq z$ **by** auto
with Cons.IH[OF rest] **show** ?case
by (simp add: filter-qw-gates-other)
qed

lemma *filter-enumerate-qw-gates*:
assumes $n \leq z \wedge z < n + \text{length } ws$
shows $\text{filter } (\lambda g. g\text{-out } g = z)$
 $(\text{concat } (\text{map } (\lambda(k, w). qw\text{-gates } k \ w) (\text{List.enumerate } n \ ws)))$
 $= (qw\text{-gates } z \ (ws ! (z - n)) :: 'f::comm-ring-1 \text{ gate list})$
using *assms*
proof (induction ws arbitrary: n)
case Nil **then show** ?case **by** simp
next
case (Cons w ws)
show ?case
proof (cases n = z)
case True
have $\text{filter } (\lambda g. g\text{-out } g = z)$
 $(\text{concat } (\text{map } (\lambda(k, w). qw\text{-gates } k \ w) (\text{List.enumerate } (\text{Suc } n) \ ws))) = []$
using True **by** (intro filter-enumerate-qw-gates-miss) simp
with True **show** ?thesis
by (simp add: filter-qw-gates-same)
next
case False
with Cons.prem1 **have** $\text{le}: \text{Suc } n \leq z$ **and** $\text{lt}: z < \text{Suc } n + \text{length } ws$ **by** auto
have *ih*: $\text{filter } (\lambda g. g\text{-out } g = z)$
 $(\text{concat } (\text{map } (\lambda(k, w). qw\text{-gates } k \ w) (\text{List.enumerate } (\text{Suc } n) \ ws)))$
 $= (qw\text{-gates } z \ (ws ! (z - \text{Suc } n)) :: 'f \text{ gate list})$

by (rule Cons.IH[OF le lt])
 have idx: $(w \# ws) ! (z - n) = ws ! (z - \text{Suc } n)$
 using le by (simp add: Suc-diff-le nth-Cons' One-nat-def)
 — one of the few places that wants *One-nat-def* back: the goal pairs $z - (n + 1)$ against $z - \text{Suc } n$
 from False show ?thesis
 by (simp add: filter-qw-gates-other ih idx)
 qed
 qed

lemma *qw-gates-contrib-sum*:

sum-list (map ($\lambda g. \text{gate-contrib } g \text{ below}$) (*qw-gates* *z w*)) = *qw-sem below w*

by (simp add: *qw-gates-def qw-sem-def gate-contrib-def comp-def case-prod-unfold mult.assoc*)

— *case-prod-unfold*, not *case-prod-beta*: under *map* the pair pattern is ETA-CONTRACTED (*case-prod f*, no argument), and the *beta* rule's left-hand side is an APPLIED *case-prod f p*, so it never matches

lemma *mk-q-layer-width*: $\text{layer-width } (\text{mk-q-layer } ws) = 2 \wedge \text{length } ws$

by (simp add: *mk-q-layer-def layer-width-def*)

lemma *mk-q-layer-eval-in*:

assumes *z*: $z < \text{length } ws$

shows *layer-eval* (*mk-q-layer* *ws*) *below* ! $z = \text{qw-sem below } (ws ! z)$

proof —

have *zw*: $z < \text{layer-width } (\text{mk-q-layer } ws)$

unfolding *mk-q-layer-width* by (meson *z less-exp less-trans*)

have *flt*: $\text{filter } (\lambda g. g\text{-out } g = z) (\text{layer-gates } (\text{mk-q-layer } ws)) = \text{qw-gates } z (ws ! z)$

using *filter-enumerate-qw-gates[of 0 z ws]* *z*

by (simp add: *mk-q-layer-def*)

have *cst*: $\text{const-at } (\text{mk-q-layer } ws) \ z = 0$

by (simp add: *mk-q-layer-def const-at-def*)

show ?thesis

using *layer-eval-nth[OF zw]*

by (simp add: *flt cst qw-gates-contrib-sum*)

qed

lemma *mk-q-layer-eval-pad*:

fixes *below* :: '*f*::comm-ring-1 list

assumes *ge*: $\text{length } ws \leq z$

and *zw*: $z < \text{layer-width } (\text{mk-q-layer } ws :: 'f \text{ layer})$

shows *layer-eval* (*mk-q-layer* *ws*) *below* ! $z = 0$

proof —

have *flt*: $\text{filter } (\lambda g. g\text{-out } g = z) (\text{layer-gates } (\text{mk-q-layer } ws :: 'f \text{ layer})) = []$

using *filter-enumerate-qw-gates-miss[of z 0 ws]* *ge*

by (simp add: *mk-q-layer-def*)

have *cst*: $\text{const-at } (\text{mk-q-layer } ws :: 'f \text{ layer}) \ z = 0$

by (simp add: *mk-q-layer-def const-at-def*)

show ?thesis

by (simp add: *layer-eval-nth[OF zw] flt cst*)

qed

31 Small list-segment computation lemmas

lemma *seg-one*:

assumes $a < \text{length } xs$

shows $\text{seg } a \ 1 \ xs = [xs \ ! \ a]$

proof –

from *assms* **have** $\text{drop } a \ xs = xs \ ! \ a \ \# \ \text{drop } (\text{Suc } a) \ xs$

by (*simp add: Cons-nth-drop-Suc*)

then show *?thesis* **by** (*simp add: seg-def One-nat-def*)

qed

lemma *seg-two*:

assumes $\text{Suc } a < \text{length } xs$

shows $\text{seg } a \ 2 \ xs = [xs \ ! \ a, xs \ ! \ \text{Suc } a]$

proof –

from *assms* **have** $\text{drop } a \ xs = xs \ ! \ a \ \# \ \text{drop } (\text{Suc } a) \ xs$

by (*simp add: Cons-nth-drop-Suc*)

moreover from *assms* **have** $\text{drop } (\text{Suc } a) \ xs = xs \ ! \ \text{Suc } a \ \# \ \text{drop } (\text{Suc } (\text{Suc } a)) \ xs$

by (*simp add: Cons-nth-drop-Suc*)

ultimately have $\text{drop } a \ xs = xs \ ! \ a \ \# \ xs \ ! \ \text{Suc } a \ \# \ \text{drop } (\text{Suc } (\text{Suc } a)) \ xs$

by *simp*

then show *?thesis* **by** (*simp add: seg-def numeral-2-eq-2*)

qed

32 The concrete evaluation stacks (depth 1, one-lane digests)

Read-only layout at $W = 1, L = 2, dd = 1$: $lp = \text{wires } 0-1, acc-0 = 2, acc-1 = 3, sib-0 = 4, kb-0 = 5, \text{root} = 6, vd = 7; ro-width = 8$. Stack output slots (*ev-ro-core* order): $hl = 0, hn-0 = 1, acc-0 = 2, acc-1 = 3, kb-0 = 4, \text{root} = 5, vd = 6, \text{tag} = 7; ev-ro-len = 8$.

definition *kb-ro-stack-desc* :: *koala-bear qwire list where*

kb-ro-stack-desc =

[$((1, 0), (3, 1)), []$,
 $((1, 2), (2, 4)), [(1, 5, 2), (-1, 5, 4)]$,
 $((1, 2)), []$,
 $((1, 3)), []$,
 $((1, 5)), []$,
 $((1, 6)), []$,
 $((1, 7)), []$,
 $((1, 0)), []$]

Update layout at $W = 1, L = 2, dd = 1$: $lp = 0-1, lp2 = 2-3, acc-0 = 4, acc-1 = 5, acc2-0 = 6, acc2-1 = 7, sib-0 = 8, kb-0 = 9, \text{root} = 10, \text{root2} = 11, vd = 12; up-width = 13$. Stack output slots (*ev-up-core* order): $hl = 0, hl2 = 1, hn-0 = 2, hn2-0 = 3, acc-0 = 4, acc-1 = 5, acc2-0 = 6, acc2-1 = 7, kb-0 = 8, \text{root} = 9, \text{root2} = 10, vd = 11; ev-up-len = 12$.

definition *kb-up-stack-desc* :: *koala-bear qwire list where*

kb-up-stack-desc =

[$((1, 0), (3, 1)), []$,
 $((1, 2), (3, 3)), []$,
 $((1, 4), (2, 8)), [(1, 9, 4), (-1, 9, 8)]$,
 $((1, 6), (2, 8)), [(1, 9, 6), (-1, 9, 8)]$,
 $((1, 4)), []$,]

$((1, 5), []),$
 $((1, 6), []),$
 $((1, 7), []),$
 $((1, 9), []),$
 $((1, 10), []),$
 $((1, 11), []),$
 $((1, 12), [])$

definition *kb-stack* :: *op-kind* $\Rightarrow$ *koala-bear layer list* **where**

kb-stack *k* =
 (if *k* = *KUpdate* then [*mk-q-layer kb-up-stack-desc*]
 else [*mk-q-layer kb-ro-stack-desc*])

32.1 Layout arithmetic at the concrete parameters

lemma *kb-ro-offsets* [*simp*]:

kbc-base.ro-lp = 0 *kbc-base.ro-acc* *l* = 2 + *l* *kbc-base.ro-sib* *l* = 4 + *l*
kbc-base.ro-kb *l* = 5 + *l* *kbc-base.ro-root* = 6 *kbc-base.ro-vd* = 7
kbc-base.ro-width = 8
by (*simp-all* add: *kbc-base.ro-lp-def kbc-base.ro-acc-def kbc-base.ro-sib-def*
kbc-base.ro-kb-def kbc-base.ro-root-def kbc-base.ro-vd-def
kbc-base.ro-width-def)

lemma *kb-up-offsets* [*simp*]:

kbc-base.up-lp = 0 *kbc-base.up-lp2* = 2 *kbc-base.up-acc* *l* = 4 + *l*
kbc-base.up-acc2 *l* = 6 + *l* *kbc-base.up-sib* *l* = 8 + *l* *kbc-base.up-kb* *l* = 9 + *l*
kbc-base.up-root = 10 *kbc-base.up-root2* = 11 *kbc-base.up-vd* = 12
kbc-base.up-width = 13
by (*simp-all* add: *kbc-base.up-lp-def kbc-base.up-lp2-def kbc-base.up-acc-def*
kbc-base.up-acc2-def kbc-base.up-sib-def kbc-base.up-kb-def
kbc-base.up-root-def kbc-base.up-root2-def kbc-base.up-vd-def
kbc-base.up-width-def)

lemma *kb-ev-slots* [*simp*]:

kbc-base.ev-hl = 0 *kbc-base.ev-hn* *l* = 1 + *l* *kbc-base.ev-acc* *l* = 2 + *l*
kbc-base.ev-kb *l* = 4 + *l* *kbc-base.ev-root* = 5 *kbc-base.ev-vd* = 6
kbc-base.ev-tag = 7
by (*simp-all* add: *kbc-base.ev-hl-def kbc-base.ev-hn-def kbc-base.ev-acc-def*
kbc-base.ev-kb-def kbc-base.ev-root-def kbc-base.ev-vd-def
kbc-base.ev-tag-def)

lemma *kb-ev2-slots* [*simp*]:

kbc-base.ev2-hl = 0 *kbc-base.ev2-hl2* = 1 *kbc-base.ev2-hn* *l* = 2 + *l*
kbc-base.ev2-hn2 *l* = 3 + *l* *kbc-base.ev2-acc* *l* = 4 + *l*
kbc-base.ev2-acc2 *l* = 6 + *l* *kbc-base.ev2-kb* *l* = 8 + *l*
kbc-base.ev2-root = 9 *kbc-base.ev2-root2* = 10 *kbc-base.ev2-vd* = 11
by (*simp-all* add: *kbc-base.ev2-hl-def kbc-base.ev2-hl2-def kbc-base.ev2-hn-def*
kbc-base.ev2-hn2-def kbc-base.ev2-acc-def kbc-base.ev2-acc2-def
kbc-base.ev2-kb-def kbc-base.ev2-root-def kbc-base.ev2-root2-def
kbc-base.ev2-vd-def)

lemma *kb-ev-ro-len* [*simp*]: *kbc-base.ev-ro-len* = 8

by (*simp* add: *kbc-base.ev-ro-len-def*)

lemma *kb-ev-up-len* [*simp*]: *kbc-base.ev-up-len* = 12
by (*simp add: kbc-base.ev-up-len-def*)

lemma *kb-iwb-ro* [*simp*]: $k \neq KUpdate \implies kbc-base.iwb\ k = 8$
by (*cases k*) (*simp-all add: kbc-base.iwb-def*)

lemma *kb-iwb-up* [*simp*]: *kbc-base.iwb KUpdate* = 13
by (*simp add: kbc-base.iwb-def*)

32.2 The explicit specification vectors at the concrete layout

lemma *kb-ev-ro-core-explicit*:
assumes *len*: $(8 :: nat) \leq length\ xs$
shows *kbc-base.ev-ro-core xs* =
 $[xs!0 + 3 * xs!1,$
 $(xs!2 + xs!5 * (xs!4 - xs!2)) + 2 * (xs!4 + xs!5 * (xs!2 - xs!4)),$
 $xs!2, xs!3, xs!5, xs!6, xs!7, xs!0]$
using *len*
unfolding *kbc-base.ev-ro-core-def kbc-base.ro-mxl-def kbc-base.ro-mxr-def*
kb-ro-offsets kbc-dd
by (*simp add: kb-hl-def kb-hn-def mux-left-def mux-right-def*
seg-one seg-two upt-rec)
(*simp add: One-nat-def numeral-2-eq-2 numeral-3-eq-3*)

lemma *kb-ev-up-core-explicit*:
assumes *len*: $(13 :: nat) \leq length\ xs$
shows *kbc-base.ev-up-core xs* =
 $[xs!0 + 3 * xs!1,$
 $xs!2 + 3 * xs!3,$
 $(xs!4 + xs!9 * (xs!8 - xs!4)) + 2 * (xs!8 + xs!9 * (xs!4 - xs!8)),$
 $(xs!6 + xs!9 * (xs!8 - xs!6)) + 2 * (xs!8 + xs!9 * (xs!6 - xs!8)),$
 $xs!4, xs!5, xs!6, xs!7, xs!9, xs!10, xs!11, xs!12]$
using *len*
unfolding *kbc-base.ev-up-core-def kbc-base.up-mxl-def kbc-base.up-mxr-def*
kb-up-offsets kbc-dd
by (*simp add: kb-hl-def kb-hn-def mux-left-def mux-right-def*
seg-one seg-two upt-rec)
(*simp add: One-nat-def numeral-2-eq-2 numeral-3-eq-3*)

32.3 The stack layers compute the specification vectors

lemma *kb-ro-stack-eval*:
assumes *len*: $length\ xs = 2 \wedge 8$
shows *layer-eval (mk-q-layer kb-ro-stack-desc) xs* = *kbc-base.ev-ro xs*
proof (*rule nth-equalityI*)
have *len8*: $(8 :: nat) \leq length\ xs$ **using** *len* **by** *simp*
have *core*: *kbc-base.ev-ro-core xs* =
 $[xs!0 + 3 * xs!1,$
 $(xs!2 + xs!5 * (xs!4 - xs!2)) + 2 * (xs!4 + xs!5 * (xs!2 - xs!4)),$
 $xs!2, xs!3, xs!5, xs!6, xs!7, xs!0]$
by (*rule kb-ev-ro-core-explicit[OF len8]*)
have *lcore*: $length\ (kbc-base.ev-ro-core\ xs) = 8$ **by** (*simp add: core*)

```

show leq: length (layer-eval (mk-q-layer kb-ro-stack-desc) xs)
  = length (kbc-base.ev-ro xs)
  by (simp add: mk-q-layer-width layer-width-def mk-q-layer-def
      kb-ro-stack-desc-def kbc-base.ev-ro-def length-pad-to lcore)
fix z assume z: z < length (layer-eval (mk-q-layer kb-ro-stack-desc) xs)
then have z256: z <  $2^8$ 
  by (simp add: mk-q-layer-width layer-width-def mk-q-layer-def kb-ro-stack-desc-def)
show layer-eval (mk-q-layer kb-ro-stack-desc) xs ! z = kbc-base.ev-ro xs ! z
proof (cases z < 8)
  case True
  have lhs: layer-eval (mk-q-layer kb-ro-stack-desc) xs ! z
    = qw-sem xs (kb-ro-stack-desc ! z)
    by (rule mk-q-layer-eval-in) (simp add: kb-ro-stack-desc-def True One-nat-def [symmetric])
  have rhs: kbc-base.ev-ro xs ! z = kbc-base.ev-ro-core xs ! z
    unfolding kbc-base.ev-ro-def
    by (rule pad-to-nth) (simp add: lcore True)
  have z ∈ {0, 1, 2, 3, 4, 5, 6, 7} using True by auto
  then show ?thesis
    unfolding lhs rhs core
    by (elim insertE emptyE)
    (simp-all add: kb-ro-stack-desc-def qw-sem-def algebra-simps)
next
  case False
  have lhs: layer-eval (mk-q-layer kb-ro-stack-desc) xs ! z = 0
    by (rule mk-q-layer-eval-pad)
    (use False z in <simp-all add: kb-ro-stack-desc-def>)
  have kbc-base.ev-ro xs ! z = (kbc-base.ev-ro-core xs @ replicate (2^8 - 8) 0) ! z
    by (simp add: kbc-base.ev-ro-def pad-to-def lcore)
  also have ... = replicate (2^8 - 8) 0 ! (z - 8)
    using False by (simp add: nth-append lcore)
  also have ... = 0
    using z256 False by (intro nth-replicate) simp
  finally show ?thesis by (simp add: lhs)
qed
qed

lemma kb-up-stack-eval:
  assumes len: length xs =  $2^{13}$ 
  shows layer-eval (mk-q-layer kb-up-stack-desc) xs = kbc-base.ev-up xs
proof (rule nth-equalityI)
  have len13: ( $13 :: \text{nat}$ ) ≤ length xs using len by simp
  have core: kbc-base.ev-up-core xs =
    [ xs ! 0 + 3 * xs ! 1,
      xs ! 2 + 3 * xs ! 3,
      (xs ! 4 + xs ! 9 * (xs ! 8 - xs ! 4)) + 2 * (xs ! 8 + xs ! 9 * (xs ! 4 - xs ! 8)),
      (xs ! 6 + xs ! 9 * (xs ! 8 - xs ! 6)) + 2 * (xs ! 8 + xs ! 9 * (xs ! 6 - xs ! 8)),
      xs ! 4, xs ! 5, xs ! 6, xs ! 7, xs ! 9, xs ! 10, xs ! 11, xs ! 12 ]
    by (rule kb-ev-up-core-explicit[OF len13])
  have lcore: length (kbc-base.ev-up-core xs) = 12 by (simp add: core)
  show length (layer-eval (mk-q-layer kb-up-stack-desc) xs)
    = length (kbc-base.ev-up xs)
    by (simp add: mk-q-layer-width layer-width-def mk-q-layer-def
        kb-up-stack-desc-def kbc-base.ev-up-def length-pad-to lcore)

```

```

fix z assume z: z < length (layer-eval (mk-q-layer kb-up-stack-desc) xs)
then have z4096: z < 2 ^ 12
  by (simp add: mk-q-layer-width layer-width-def mk-q-layer-def kb-up-stack-desc-def)
show layer-eval (mk-q-layer kb-up-stack-desc) xs ! z = kbc-base.ev-up xs ! z
proof (cases z < 12)
  case True
  have lhs: layer-eval (mk-q-layer kb-up-stack-desc) xs ! z
    = qw-sem xs (kb-up-stack-desc ! z)
    by (rule mk-q-layer-eval-in) (simp add: kb-up-stack-desc-def True One-nat-def [symmetric])
  have rhs: kbc-base.ev-up xs ! z = kbc-base.ev-up-core xs ! z
    unfolding kbc-base.ev-up-def
    by (rule pad-to-nth) (simp add: lcore True)
  have z ∈ {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11} using True by (auto; presburger)
  then show ?thesis
    unfolding lhs rhs core
    by (elim insertE emptyE)
      (simp-all add: kb-up-stack-desc-def qw-sem-def algebra-simps)
next
  case False
  have lhs: layer-eval (mk-q-layer kb-up-stack-desc) xs ! z = 0
    by (rule mk-q-layer-eval-pad)
      (use False z in ⟨simp-all add: kb-up-stack-desc-def⟩)
  have kbc-base.ev-up xs ! z = (kbc-base.ev-up-core xs @ replicate (2 ^ 12 - 12) 0) ! z
    by (simp add: kbc-base.ev-up-def pad-to-def lcore)
  also have ... = replicate (2 ^ 12 - 12) 0 ! (z - 12)
    using False by (simp add: nth-append lcore)
  also have ... = 0
    using z4096 False by (intro nth-replicate) simp
  finally show ?thesis by (simp add: lhs)
qed
qed

```

33 The compiler model is inhabited over KoalaBear

lemma *compiler-model-kb*:

compiler-model kb-params kb-h-leaf kb-h-node 1 2
kb-digest-repr kb-leaf-repr kb-hl kb-hn kb-stack

— *unfold-locales* discharges the entire *compiler-model-base* part from the *kbc-base* interpretation above; exactly the two *stack-spec* obligations remain

proof (*unfold-locales*)

fix k :: op-kind and xs :: koala-bear list

assume k: k ≠ KUpdate and len: length xs = kbc-base.ivlen k

have length xs = 2 ^ 8

using len k by (simp add: kbc-base.ivlen-def)

then show foldr layer-eval (kb-stack k) xs = kbc-base.ev-ro xs

using k by (simp add: kb-stack-def kb-ro-stack-eval)

next

fix xs :: koala-bear list

assume len: length xs = kbc-base.ivlen KUpdate

have length xs = 2 ^ 13

using len by (simp add: kbc-base.ivlen-def)

then show foldr layer-eval (kb-stack KUpdate) xs = kbc-base.ev-up xs

by (simp add: kb-stack-def kb-up-stack-eval)
qed

interpretation kbc: compiler-model
 kb-params kb-h-leaf kb-h-node 1 2 kb-digest-repr kb-leaf-repr kb-hl kb-hn kb-stack
 by (rule compiler-model-kb)

34 Activation examples

A concrete valid Membership round-trips through the verifier model: completeness accepts it, soundness recovers the semantics.

definition ex-v :: koala-bear **where** ex-v = 5
definition ex-s :: koala-bear **where** ex-s = 7
definition ex-root :: koala-bear **where**
 ex-root = kb-h-node (kb-h-leaf (Occupied ex-v)) ex-s

lemma ex-smt-valid:
 kbc-base.smt-valid (Membership 0 ex-v) ex-root ex-root (Occupied ex-v, [ex-s])
proof –
 have kbc-base.path-ok (Occupied ex-v) 0 [ex-s] ex-root
 unfolding kbc-base.path-ok-def
 by (simp add: kbc-base.path-root.simps ex-root-def kb-params-def)
 then show ?thesis by (simp add: kbc-base.smt-valid.simps)
 qed

theorem instance-theorem-A-activation:
 kbc.verifier-accept (Membership 0 ex-v) ex-root ex-root
 (kb-h-leaf (Occupied ex-v)) (Occupied ex-v, [ex-s])
 using kbc.theorem-A-completeness[OF ex-smt-valid]
 by (simp add: kbc.wit-vd-def)

corollary instance-theorem-A-roundtrip:
 kbc-base.smt-valid (Membership 0 ex-v) ex-root ex-root (Occupied ex-v, [ex-s])
 using kbc.theorem-A-soundness[OF instance-theorem-A-activation]
 by (simp add: kbc.canon-witness-def)

A mismatched root pair is semantically unsatisfiable - the *nosem* premise of Theorem C, discharged concretely.

lemma ex-nosem:
 $\neg (\exists w. \text{kbc-base.smt-valid (Membership 0 ex-v) ex-root (ex-root + 1) w})$
proof
 assume $\exists w. \text{kbc-base.smt-valid (Membership 0 ex-v) ex-root (ex-root + 1) w}$
 then obtain lf sb **where** kbc-base.smt-valid (Membership 0 ex-v) ex-root (ex-root + 1) (lf, sb)
 by (metis prod.exhaust)
 then have $\text{ex-root} + 1 = \text{ex-root}$ **by** (simp add: kbc-base.smt-valid.simps)
 then show False **by** simp
 qed
 end

theory *Composition-Instance*

imports *Composition Compiler-Instance Leaf-Fold GKR-Protocol.Layer-Representative*
begin

declare *One-nat-def* [*simp del*]

— as in *Compiler-Instance*: $1 = \text{Suc } 0$ rewrites the numeral arguments of the interpreted locale constants and thereby breaks every *kbc-base.*-def* match

35 Lemma (b) over the production field

The leaf-hash structure locale is inhabited over KoalaBear: the *of-nat-injectivity-on-bound* premise is discharged by the concrete field fact that 31 is far below the characteristic, witnessing that the collision reduction of lemma (b) applies at the production parameters.

interpretation *kb-leaf-hash*: *leaf-hash-structure*

$\lambda \text{enc} :: \text{koala-bear list. leaf-fold enc } 31 \lambda p. p \ 31$

proof

fix $a \ b$ **assume** $a \leq (31 :: \text{nat})$ **and** $b \leq (31 :: \text{nat})$

and $(\text{of-nat } a :: \text{koala-bear}) = \text{of-nat } b$

then show $a = b$

by $(\text{intro kb-of-nat-inj-on-bound}) \text{ simp-all}$

qed *simp-all*

36 The composed locale is inhabited over KoalaBear + mpoly

lemma *gkr-layer-sumcheck-mpoly*:

$\text{gkr-layer-sumcheck } (\text{vars} :: 'a :: \{\text{finite, field}\} \text{ mpoly} \Rightarrow \text{nat set}) \text{ total-degree}$

$(\lambda p \sigma. \text{insertion } (\text{the } \circ \sigma) p) \text{ inst}$

by *unfold-locales (auto simp add: multi-variate-polynomial-lemmas)*

lemma *statesync-gkr-v01-kb*:

$\text{statesync-gkr-v01 kb-params kb-h-leaf kb-h-node } 1 \ 2$

$\text{kb-digest-repr kb-leaf-repr kb-hl kb-hn kb-stack}$

$(\text{vars} :: \text{koala-bear mpoly} \Rightarrow \text{nat set}) \text{ total-degree}$

$(\lambda p \sigma. \text{insertion } (\text{the } \circ \sigma) p) \text{ inst}$

by $(\text{rule statesync-gkr-v01.intro}[\text{OF compiler-model-kb gkr-layer-sumcheck-mpoly}])$

interpretation *kbs*: *statesync-gkr-v01*

$\text{kb-params kb-h-leaf kb-h-node } 1 \ 2 \text{ kb-digest-repr kb-leaf-repr kb-hl kb-hn kb-stack}$

$\text{vars} :: \text{koala-bear mpoly} \Rightarrow \text{nat set total-degree}$

$\lambda p \sigma. \text{insertion } (\text{the } \circ \sigma) p \text{ inst}$

by $(\text{rule statesync-gkr-v01-kb})$

37 The activation scenario

definition *exC-op* :: *koala-bear smt-op* **where**

$\text{exC-op} = \text{Update } 0 \text{ Empty (Occupied } 1)$

definition *exC-w* :: *koala-bear leaf-state* $\times$ *koala-bear list* **where**

$\text{exC-w} = (\text{Empty}, [0])$

definition $exC\text{-}vd :: koala\text{-}bear$ **where**
 $exC\text{-}vd = kb\text{-}h\text{-}leaf (Occupied\ 1)$

lemma $exC\text{-}kind [simp]: kind\text{-}of\ exC\text{-}op = KUpdate$
by ($simp\ add: exC\text{-}op\text{-}def$)

37.1 Semantic unsatisfiability (the nosem premise)

lemma $exC\text{-}nosem: \neg (\exists w'. kb\text{-}base.smt\text{-}valid\ exC\text{-}op\ 0\ 0\ w')$

proof

assume $\exists w'. kb\text{-}base.smt\text{-}valid\ exC\text{-}op\ 0\ 0\ w'$

then obtain $lf\ sb$ **where** $v: kb\text{-}base.smt\text{-}valid (Update\ 0\ Empty\ (Occupied\ 1))\ 0\ 0\ (lf, sb)$

unfolding $exC\text{-}op\text{-}def$ **by** ($metis\ prod.exhaust$)

then have $pok1: kb\text{-}base.path\text{-}ok\ Empty\ 0\ sb\ 0$

and $pok2: kb\text{-}base.path\text{-}ok\ (Occupied\ 1)\ 0\ sb\ 0$

by ($simp\text{-}all\ add: kb\text{-}base.smt\text{-}valid.simps$)

from $pok1[unfolded\ kb\text{-}base.path\text{-}ok\text{-}def]$ **have** $length\ sb = 1$

by ($simp\ add: kb\text{-}params\text{-}def$)

then obtain s **where** $sb: sb = [s]$

by ($cases\ sb$) *auto*

from $pok1\ sb$ **have** $kb\text{-}h\text{-}node\ (kb\text{-}h\text{-}leaf\ Empty)\ s = 0$

by ($simp\ add: kb\text{-}base.path\text{-}ok\text{-}def$)

then have $e1: 2 * s = 0$

by ($simp\ add: kb\text{-}h\text{-}node\text{-}def\ kb\text{-}h\text{-}leaf\text{-}def$)

from $pok2\ sb$ **have** $kb\text{-}h\text{-}node\ (kb\text{-}h\text{-}leaf\ (Occupied\ 1))\ s = 0$

by ($simp\ add: kb\text{-}base.path\text{-}ok\text{-}def$)

then have $4 + 2 * s = 0$

by ($simp\ add: kb\text{-}h\text{-}node\text{-}def\ kb\text{-}h\text{-}leaf\text{-}def$)

then have $(4 :: koala\text{-}bear) = 0$

by ($simp\ add: e1$)

then have $(2 :: koala\text{-}bear) * 2 = 0$

by $simp$

then have $(2 :: koala\text{-}bear) = 0$

using $mult\text{-}eq\text{-}0\text{-}iff [THEN\ iffD1]$ **by** $blast$

— resolution, not simp: simp folds $2 * 2$ into the numeral 4 before $mult\text{-}eq\text{-}0\text{-}iff$ can see a product, and $4 = 0$ alone yields nothing

with $kb\text{-}two\text{-}ne\text{-}zero$ **show** $False ..$

qed

37.2 Public-input, echo and length premises

lemma $exC\text{-}pub: kb\text{-}pub\text{-}ok\ exC\text{-}op\ 0\ 0\ exC\text{-}vd$

unfolding $kb\text{-}pub\text{-}ok\text{-}def\ kb\text{-}vd\text{-}ok\text{-}def$

by ($simp\ add: exC\text{-}op\text{-}def\ exC\text{-}vd\text{-}def\ kb\text{-}params\text{-}def$)

lemma $exC\text{-}wlen: length\ (snd\ exC\text{-}w) = depth\ kb\text{-}params$

by ($simp\ add: exC\text{-}w\text{-}def\ kb\text{-}params\text{-}def$)

lemma $exC\text{-}echo: kb\text{-}echo\text{-}ok\ exC\text{-}op\ exC\text{-}w$

by ($simp\ add: kb\text{-}echo\text{-}ok\text{-}def\ exC\text{-}op\text{-}def\ exC\text{-}w\text{-}def$)

37.3 Chain shape (the wf premise, discharged by computation)

lemma *exC-descs-up-explicit*:

$kbc\text{-}base.descs\text{-}up = [RDiff\ 0\ 4, RDiff\ 1\ 6, RDiff\ 2\ 5, RDiff\ 3\ 7,$
 $RDiff\ 5\ 9, RDiff\ 7\ 10, RDiff\ 6\ 11, RBool\ 8]$
by (*simp add: kbc-base.descs-up-def upt-rec*)

lemma *exC-res-gates-in*:

$\forall g \in set\ (layer\ gates\ (mk\text{-}res\ layer\ (kbc\text{-}base.descs\ (kind\text{-}of\ exC\text{-}op)) :: koala\text{-}bear\ layer)).$
 $g\text{-}in1\ g < 2 \wedge 12 \wedge g\text{-}in2\ g < 2 \wedge 12$
by (*simp add: mk-res-layer-def exC-descs-up-explicit*)

lemma *exC-q-gates-in*:

$\forall g \in set\ (layer\ gates\ (mk\text{-}q\ layer\ kb\text{-}up\ stack\ desc :: koala\text{-}bear\ layer)).$
 $g\text{-}in1\ g < 2 \wedge 13 \wedge g\text{-}in2\ g < 2 \wedge 13$
by (*simp add: mk-q-layer-def kb-up-stack-desc-def qw-gates-def*)

context

fixes *exC-enc* :: *koala-bear list*
and *exC-bels* :: *koala-bear list list*
defines *exC-enc-def*: $exC\text{-}enc \equiv kbc\text{-}base.encode\text{-}witness\ exC\text{-}op\ 0\ 0\ exC\text{-}vd\ exC\text{-}w$
and *exC-bels-def*:
 $exC\text{-}bels \equiv tl\ (circuit\ values$
 $(mk\text{-}res\ layer\ (kbc\text{-}base.descs\ (kind\text{-}of\ exC\text{-}op)) \# kb\text{-}stack\ (kind\text{-}of\ exC\text{-}op))\ exC\text{-}enc)$

begin

lemma *exC-bels-explicit*:

$exC\text{-}bels = [layer\ eval\ (mk\text{-}q\ layer\ kb\text{-}up\ stack\ desc)\ exC\text{-}enc, exC\text{-}enc]$
by (*simp add: exC-bels-def kb-stack-def Let-def*)

lemma *exC-wf*:

gkr-chain-wf [12, 13]
 $(mk\text{-}res\ layer\ (kbc\text{-}base.descs\ (kind\text{-}of\ exC\text{-}op)) \# kb\text{-}stack\ (kind\text{-}of\ exC\text{-}op))\ exC\text{-}bels$

proof –

have *stack*: $kb\text{-}stack\ (kind\text{-}of\ exC\text{-}op) = [mk\text{-}q\ layer\ kb\text{-}up\ stack\ desc]$
by (*simp add: kb-stack-def*)
have *wb*: $layer\ width\ bits\ (mk\text{-}q\ layer\ kb\text{-}up\ stack\ desc :: koala\text{-}bear\ layer) = 12$
by (*simp add: mk-q-layer-def kb-up-stack-desc-def*)
show *?thesis*
unfolding *stack exC-bels-explicit*
using *exC-res-gates-in exC-q-gates-in*
by (*simp add: stack wb*)

qed

38 Theorem C, activated

All premises of the composed soundness theorem are discharged for the concrete scenario: the chain-shape premise by computation over the actual compiled layers, the representation premise by the mpoly interpolation representative with $dbnd = (\lambda s. 2 * s)$, the public-input/echo/length premises by evaluation, and the semantic unsatisfiability by the root-equation argument above. The bound fact binds the composed soundness error ($|residuals| + \sum (2\ s \cdot 2\ s + 1) / |KoalaBear|$) for the concrete Update against ANY adversary. (Fact

binding: the chain event is a locale constant, so the specialised statement lives in the instance's terms.)

```
lemmas instance-theorem-C-activation =
  kbs.theorem-C-composition[where dbnd =  $\lambda s. 2 * s$ ,
    OF exC-bels-def[unfolded exC-enc-def, THEN meta-eq-to-obj-eq] exC-wf layer-repr-mpoly
    exC-pub exC-wlen exC-echo exC-nosem]

end

end
```